

國立臺北商業大學

資訊管理系

1 1 4 資 訊 系 統 專 案 設 計

系統手冊



組別：第 114409 組

題目：會議寶

指導老師：陳信宏副教授

組長：11146065 姜筱雲

組員：11146067 連湘琪

11146074 邵紫綺

11146079 李曼鈺

11146082 范雅淇

中 華 民 國 1 1 4 年 1 2 月 3 1 日

誌謝

本專題研究得以順利推展，特別感謝臺灣中央研究院數位文化中心的王祥安博士。在專題進行期間，王博士在百忙之中多次撥冗與我們深入交流與討論，並在理論與實務層面提供了寶貴的指導與協助。

王博士憑藉其深厚的學術素養與豐富的實務經驗，為我們的研究帶來全新的視角與啟發，有效協助我們克服研究過程中的諸多挑戰，讓本專題得以更深入且全面地發展。在其悉心指導下，我們不僅加深了對研究領域的理解，也在學術思維與研究方法上獲得顯著提升。

在此，謹向王祥安博士致上最誠摯的感謝。感謝他無私的支持與指導，這份指導對我們的學術旅程具有深遠的影響，必將長存於我們心中。

姜筱雲、連湘琪、邵紫綺、李曼鈺、范雅淇謹上

謹致於國立臺北商業大學

資訊管理系

中華民國 114 年 12 月 31 日

國立臺北商業大學專題課程作品

電子文件上網授權書

本授權書所授權之作品為授權人在 國立臺北商業大學 資訊管理(科/系/所)

114409 組 114 學年度 第 1 學期專題課程之成果報告作品。

專題題目：會議寶

指導教授：陳信宏

☐ 本人 ☒ 團體著作人

☒ 同意 提供讀者基於個人非營利性質之線上檢索、閱覽、下載或列印。

茲同意將授權人擁有著作權之上列論文全文(含摘要)無償授權本人就讀學校(國立臺北商業大學)圖書館，不限地域、時間與次數，以微縮、光碟或其他各種數位化方式將上列作品收錄、重製與利用，並得將數位化之上列作品及其電子檔上載資料庫系統。於著作權法合理使用範圍內，讀者得進行線上檢索、閱覽、下載或列印。

專題課程成果報告作品全文上載網路公開之範圍及時間：

本校區域網路 ☒ 立即公開

☐ 自中華民國 年 月 日公開

校外網際網路 ☒ 立即公開

☐ 自中華民國 年 月 日公開

☐ 不同意 提供讀者基於個人非營利性質之線上檢索、閱覽、下載或列印。

授權人

簽 名

(作品若為團體作品，請每一位團體成員均應簽署)

中 華 民 國 1 1 4 年 1 2 月 3 1 日

目錄

誌謝	II
第 1 章 前言	1
1-1 背景介紹	1
1-3 系統目的與目標	2
1-4 預期成果	3
第 2 章 營運計畫	4
2-1 可行性分析	4
2-2 商業模式 (BusinessModel)	6
2-3 市場分析-STP	7
2-4 競爭力分析-SWOT/TOWS	8
第 3 章 系統規格	10
3-1 系統架構	10
3-2 系統軟、硬體需求與技術平台	11
3-3 開發標準與使用工具	11
第 4 章 專案時程與組織分工	12
4-1 專案時程	12
4-2 專案組織與分工	13
第 5 章 需求模型	17
5-1 使用者需求	17
5-2 使用個案圖 (UseCaseDiagram)	18

5-4 使用個案描述-活動圖	19
5-5 分析類別圖 (AnalysisClassDiagram)	26
第 6 章 設計模型	27
6-1 循序圖 (SequentialDiagram)	27
6-2 類別設計圖	44
第 7 章 實作模型	45
7-1 佈署圖(DeploymentDiagram)	45
7-2 套件圖(PackageDiagram).....	45
7-3 元件圖(ComponentDiagram)	46
7-4 狀態機(StateMachine)	47
第 8 章 資料庫設計	53
8-1 資料庫關聯表	53
8-2 表格及其 MetaData	54
第 9 章 程式	65
9-1 元件清單及其規格描述.....	65
9-2 其他附屬之各種元件	71
第 10 章 測試模型	101
10-1 測試計劃	101
10-2 測試個案與測試結果.....	102
第 11 章 操作手冊	108
11-1 系統元件	108

11-2 系統下載及安裝	108
第 1 2 章 使用手冊	110
第 1 3 章 感想	117
第 1 4 章 參考資料	120
附錄一 評審建議事項	121
附錄二 會議記錄	123

圖目錄

圖 3-1-1 系統架構圖.....	10
圖 3-1-3 系統功能架構圖.....	10
圖 4-1-1 專案時程甘特圖.....	12
圖 4-2-1 進度文件資料上傳 Github 貢獻次數.....	15
圖 5-2-1 使用個案圖.....	18
圖 5-4-1 會員註冊之活動圖.....	19
圖 5-4-2 會員登入之活動圖.....	19
圖 5-4-3 會員忘記密碼之活動圖.....	20
圖 5-4-4 會員資料管理之活動圖.....	20
圖 5-4-5 建立組織之活動圖.....	21
圖 5-4-6 創建議題之活動圖.....	21
圖 5-4-7 發起會議之活動圖.....	22
圖 5-4-8 檔案上傳之活動圖.....	22
圖 5-4-9 語音轉文字之活動圖.....	23
圖 5-4-10 會議回顧之活動圖.....	23
圖 5-4-11 查詢紀錄之活動圖.....	24
圖 5-4-12 正式文件之活動圖.....	24
圖 5-4-13 任務追蹤之活動圖.....	25
圖 5-5-1 分析類別圖.....	26
圖 6-1-1 會員註冊之循序圖.....	27
圖 6-1-2 會員登入之循序圖.....	28
圖 6-1-3 會員資料管理之循序圖.....	29
圖 6-1-4 忘記密碼之循序圖.....	30
圖 6-1-5 建立組織之循序圖.....	31
圖 6-1-6 創建議題之循序圖.....	32
圖 6-1-7 發起會議之循序圖.....	33
圖 6-1-8 檔案上傳之循序圖.....	34
圖 6-1-9 即時轉錄之循序圖.....	35
圖 6-1-10 檔案轉錄之循序圖.....	36
圖 6-1-11 智能查詢之循序圖.....	37
圖 6-1-12 會議回顧之循序圖.....	38
圖 6-1-13 歷史查詢之循序圖.....	39
圖 6-1-14 正式文件之循序圖.....	40
圖 6-1-15 任務追蹤之循序圖.....	41
圖 6-1-16 通知機制之循序圖.....	42
圖 6-1-17 管理者介面之循序圖.....	43
圖 6-2-1 設計類別圖.....	44
圖 7-1-1 佈署圖.....	45
圖 7-2-1 套件圖.....	45
圖 7-3-1 元件圖.....	46

圖 7-4-1 會員登入之狀態機.....	47
圖 7-4-2 會員資料修改登入之狀態機.....	47
圖 7-4-3 檔案上傳之狀態機.....	48
圖 7-4-4 智能查詢之狀態機.....	48
圖 7-4-5 語音轉文字之狀態機.....	49
圖 7-4-6 會議回顧之狀態機.....	49
圖 7-4-7 查詢紀錄之狀態機.....	50
圖 7-4-8 正式文件之狀態機.....	50
圖 7-4-9 任務追蹤之狀態機.....	51
圖 7-4-10 Line Bot 之狀態機.....	52
圖 8-1-1 資料庫關聯表.....	53

表目錄

表 1-3-1 相關系統服務之功能比較表.....	2
表 2-1-2 技術可行性評估表.....	4
表 2-2-1 商業模式-Businessmodel.....	6
表 2-3-1STP 市場分析表.....	7
表 2-4-1SWOT-TOWS 分析表.....	9
表 3-2-1 系統軟、硬體需求.....	11
表 3-3-1 專案開發工具.....	11
表 4-2-1 專題組織分工表.....	13
表 4-2-2 專題工作內容與貢獻度表.....	16
表 5-1-1 使用者功能性需求表.....	17
表 5-1-2 使用者非功能性需求表.....	17
表 8-2-1 使用者資料 user 資料表.....	54
表 8-2-2 組織資料 organizations 資料表.....	54
表 8-2-3 組織成員關係 organization_members 資料表.....	55
表 8-2-4 議題資料 topics 資料表.....	55
表 8-2-5 議題成員關係 topic_participants 資料表.....	56
表 8-2-6 會議資料 meetings 資料表.....	57
表 8-2-7 會議參與者關係 topic_participants 資料表.....	57
表 8-2-8 檔案上傳 files 資料表.....	58
表 8-2-9 會議摘要 summaries 資料表.....	59
表 8-2-10 摘要紀錄 summaries_logs 資料表.....	60
表 8-2-11 問答紀錄 qa_logs 資料表.....	61
表 8-2-12 任務資料 tasks 資料表.....	62
表 8-2-13 逐字稿資料 transcripts 資料表.....	63
表 8-2-14 正式文件 formal_documents 資料表.....	64
表 9-1-1 元件清單與規格描述表(前端)-HTML 介面編寫.....	65
表 9-1-2 元件清單與規格描述表(後端)-語音處理.....	66
表 9-1-3 元件清單與規格描述表(後端)-文字處理.....	67
表 9-1-4 元件清單與規格描述表(後端)-組織/議題/會議管理.....	68
表 9-1-5 元件清單與規格描述表(後端)-AI 與語意分析.....	69
表 9-1-6 元件清單與規格描述表(後端)-系統服務.....	70
表 9-2-1 虛擬碼-meeting_during_transcript_now.html.....	71
表 9-2-2 虛擬碼- meeting_review.html.....	73
表 9-2-3 虛擬碼- task.html.....	75
表 9-2-4 虛擬碼- whisper_handler.py.....	78
表 9-2-5 虛擬碼- openai_srt_punctuator.py.....	80
表 9-2-6 虛擬碼- todos_bp.py.....	81
表 9-2-7 虛擬碼- meeting.py.....	82
表 9-2-8 虛擬碼- permission.py.....	85
表 9-2-9 虛擬碼- qa.py.....	87

表 9-2-10 虛擬碼-task.py	88
表 9-2-11 虛擬碼- rag_pipeline.py	90
表 9-2-12 虛擬碼- flask_blueprint.py	93
表 9-2-13 虛擬碼-formal_doc.py	95
表 9-2-14 虛擬碼- realtime_stt_ws.py	96
表 9-2-15 虛擬碼-rag_utils.py	98
表 10-2-1 功能測試-註冊/登入/登出	102
表 10-2-2 功能測試-修改會員資料	102
表 10-2-3 功能測試-建立組織/邀請成員	102
表 10-2-4 功能測試-創建議題/創建會議	103
表 10-2-5 功能測試-設定會議權限	103
表 10-2-6 功能測試-發送會議通知	103
表 10-2-7 功能測試-即時轉錄	104
表 10-2-8 功能測試-檔案轉錄	104
表 10-2-9 功能測試-檔案上傳	104
表 10-2-10 功能測試-檔案區瀏覽/智能查詢	105
表 10-2-11 功能測試-生成摘要	105
表 10-2-12 功能測試-修改/複製摘要/檢視修改紀錄	105
表 10-2-13 功能測試-查詢紀錄檢視與修改	106
表 10-2-14 功能測試-逐字稿整理→正式文件	106
表 10-2-15 功能測試-新增/編輯/刪除任務	106
表 10-2-16 功能測試-發送任務提醒	107
表 10-2-17 功能測試-帳號維護/檔案維護	107
表 10-2-18 功能測試-資料庫管理/AI 模型維護	107
表 11-1 系統安裝元件資訊	108

第1章 前言

1-1 背景介紹

隨著數位轉型與遠距協作的普及，會議已成為企業內部資訊傳遞、團隊溝通與決策制定的核心環節。然而，隨著會議頻率增加，資訊與決策往往堆疊雜亂，難以有效整理重點或追蹤執行。長期下來容易造成資訊保存不完整、檢索困難，甚至出現「開會等於沒開」的情況，嚴重影響組織效率與決策品質。

為解決這些問題，我們推出以組織為單位的「智能會議助理」系統——會議寶。它不僅能輕鬆建立組織、議題與會議，支援會議檔案上傳並進行智能查詢，還能將語音錄音即時或檔案自動轉文字，生成會議摘要，快速掌握討論重點與決策內容。此外，會議寶可將會議紀錄轉換為正式文件，並提供任務追蹤功能，確保決策落地。透過語音轉文字、智能分析與任務管理，會議寶全面提升資訊管理效率與團隊協作能力，讓每一次會議都能真正產生價值。

1-2 動機

在現今高度協作的工作環境中，會議已成為專案推動與決策制定的關鍵環節。然而，多數市面上的會議助理工具僅具備語音轉文字功能，對內容的理解與後續應用整合不足。實際情況中，會議紀錄常分散於錄音檔、簡報、手寫筆記與群組對話之間，缺乏統一的平台進行彙整與關聯。成員在會後常需耗費大量時間手動蒐集、比對與回顧，不僅降低效率，也容易造成資訊誤解與決策延宕。例如，新進團隊成員若需接手進行中的專案，往往必須翻閱冗長無序的錄音、版本繁多的簡報、分散難尋的筆記，以及凌亂的聊天紀錄，難以快速掌握專案脈絡，甚至可能重複討論既有議題，最終拖延團隊進度。這並非個別案例，而是許多組織中普遍存在的溝通瓶頸。

為解決這些痛點，我們打造了更全面的智能會議助理「會議寶」系統。它除了具備語音轉文字功能外，更整合了智能查詢，幫助快速找到會議中的關鍵資訊；透過歷史會議摘要，協助成員迅速掌握討論重點；並能自動產出正式文件，提升紀錄專業度與可用性。同時，系統還具備任務追蹤功能，將討論中的待辦事項明確化並持續提醒，確保決策落實執行。會議寶全面提升組織資訊整合能力、協作效率與決策品質，讓每一次會議都真正發揮價值。

1-3 系統目的與目標

本專題旨在開發一套以組織為單位的智能會議助理系統——「會議寶」，以使用者需求為出發點，提供完整且智慧化的會議紀錄與知識管理服務。具體目標如下：

- 資料統整與檢索：透過智能查詢與歷史摘要，快速整理、查找會議資訊。
- 語音轉文字：即時轉錄會議內容，降低紀錄負擔，確保資訊完整且可追溯。
- 智能查詢：針對會議內容快速回應提問，提升資訊獲取效率。
- 正式文件生成：產出專業會議紀錄及決策文件，提高紀錄的專業度與可用性。
- 任務與決策追蹤：跨會議場次關聯待辦事項並提供提醒，確保決策脈絡執行。

▼表 1-3-1 相關系統服務之功能比較表

功能特色/系統	 會議寶	 Otter.ai	 NotebookLM	 SeaMeet
會議提醒通知	✓	✓		
檔案上傳與管理	✓	✓	✓	✓
會議內容智能查詢	✓	✓	✓	
歷史會議決策查詢	✓	✓	✓	
語音轉文字	✓	✓	✓	✓
AI 會議/摘要記憶	✓	✓	✓	✓
正式文件生成	✓			
任務追蹤	✓			

1-4 預期成果

本系統預期能有效整合分散的會議紀錄與討論內容，幫助使用者迅速掌握會議中的關鍵資訊，解決資訊遺漏、重複討論與責任不清等常見問題。透過語音轉文字功能，會議發言可自動生成逐字稿，搭配會議回顧與自動生成的會議摘要，使用者能快速掌握每場會議的重點內容，節省重新閱讀冗長紀錄的時間。

系統的任務追蹤功能可明確紀錄各項決策及其形成會議，並追蹤後續執行進度，使責任分工更清楚、執行更高效。正式文件生成功能則可將會議討論整理成標準化報告或決議文件，方便共享與歸檔。

此外，系統整合智能查詢機制，使用者可透過自然語言直接提問，迅速取得來自歷史會議資料的精準回應，大幅降低資料查找與溝通成本。這些功能不僅提升資訊在不同會議間的串聯與延續性，也增強跨會議的決策協調能力，使會議資料不再只是紀錄，而能轉化為具備知識價值的資產。最終，我們期望實現「資訊有記憶，會議不白開」的目標，讓每一次會議的討論與決策都能被完整累積與應用。

第2章 營運計畫

2-1 可行性分析

可行性分析包含四個要素：技術可行性、經濟可行性、法律可行性與操作可行性。

本組將依據這四個要素對本系統進行分析。

1.技術可行性

在技術可行性方面，我們主要評估團隊是否具備足夠的資源與能力完成系統開發。本系統需要整合語音辨識、自然語言處理、資料庫管理及前端框架等技術，以支援語音轉文字、智能查詢、會議回顧、任務追蹤與正式文件生成等功能。團隊成員具備相關技術經驗，能夠有效運用軟硬體設備進行模型運算、資料存取與系統部署，確保系統各項功能的穩定與可行性。整體而言，本系統的技術資源與能力充分滿足開發需求，具備良好的技術可行性。

▼表 2-1-1 技術可行性評估表

評估項目	評估結果	說明
技術資源	符合需求	採用開源資源及前端框架，支援語音轉文字、智能查詢、會議回顧、文件生成及任務追蹤功能
技術能力	符合需求	團隊具備語音辨識、NLP、資料庫及前端開發能力，能完成核心功能開發
軟硬體設備	符合需求	擁有可支援語音模型運算、資料存取及系統部署的設備，滿足多用戶操作需求

2.經濟可行性

本系統主要採用免費或開源資源開發，成本包含開發人力、資料收集與系統維運費用。預期收益來自企業、學校及團隊使用會議寶進行會議紀錄與任務追蹤，可透過訂閱費獲得收入。初期採用開源工具與模擬數據驗證功能，降低軟體購置成本，整體開發與維運費用可控，經濟可行性良好。

3.法律可行性

本系統將遵守相關資訊與隱私法規，如《個人資料保護法》及《通訊保障及監察法》，確保所有會議錄音與資料的使用皆建立在使用者知情同意的前提下。系統開發與運作符合資訊倫理原則，會議資料的處理、保存與應用均在合理且合規的範圍內進行。此外，系統亦遵守所整合平台的使用政策與服務條款，保障使用者權益，確保系統運作合法合規。

4.操作可行性

本系統介面設計簡潔明確，操作流程直覺，提供使用者友善的操作體驗。系統功能整合會議錄音轉文字、內容摘要、智能查詢與任務追蹤等核心功能，提升操作便利性與整體使用效率。此外，系統適用於學術、企業及團隊合作等多種場景，操作流程通用且具實用價值，能滿足不同使用者的需求。

2-2 商業模式 (BusinessModel)

▼表 2-2-1 商業模式-Businessmodel

關鍵合作夥伴 ◆ 資料庫 ◆ 語音辨識平台	關鍵活動 ◆ 智能查詢 ◆ 語音轉錄 ◆ 會議摘要 ◆ 決策查詢 ◆ 任務追蹤 關鍵資源 ◆ 自然語言處理 ◆ 網站開發技術 ◆ AI 模型訓練 ◆ 語音模型能力	價值主張 ◆ 語音轉文字 ◆ 會議摘要 ◆ 智能查詢 ◆ 任務追蹤 ◆ 正式文件生成 ◆ 決策查詢 ◆ 多使用者共用 ◆ 跨會議資料查詢	顧客關係 ◆ 會議提醒 ◆ 任務提醒 通路 ◆ 網站 ◆ API 串接	目標客戶 ◆ 需要多人同步管理會議的機構
成本結構 ◆ 技術開發成本 ◆ 伺服器開發成本 ◆ AI 模型訓練與人員維護 ◆ 營運管理費用		收益流 ◆ 訂閱費		

2-3 市場分析-STP

▼表 2-3-1STP 市場分析表

市場區隔 SEGMENTATION	◆ 頻繁開會者 ◆ 高度重視會議紀錄者 ◆ 尋求會議效率提升者
目標市場 TARGETING	主要鎖定需要多人同步管理會議的機構。
產品定位 POSITIONING	會議寶—智能會議助理，整合語音轉文字、自動摘要與任務追蹤，提升團隊溝通效率與決策速度。

1. 市場區隔 (Segmentation)

- 頻繁開會者：經常參與會議，需要高效記錄、管理與追蹤會議內容的人群。
- 高度重視會議紀錄者：對會議資料的保存、整理與後續追蹤有強烈需求的人群。
- 尋求會議效率者：希望提升會議效率、縮短討論時間、優化溝通流程的人群。

2. 目標市場 (Targeting)

- 需要管理多人同步會議資料、追蹤決策脈絡的組織。
- 重視溝通透明度與知識整合效率的企業與機構。
- 需即時查詢過往會議資訊的部門或團隊。

3. 產品定位 (Positioning)

會議寶—智能會議助理，提供「紀錄+分析+記憶」功能，整合語音轉錄、重點摘要與決策追蹤，串聯歷次會議內容，快速查詢關鍵資訊，提升決策與會議效率。

2-4 競爭力分析-SWOT/TOWS

SWOT 分析用於評估企業、專案或個人面臨的內外部環境因素，並結合 TOWS 分析提出策略，以有效利用優勢與機會，克服弱點與威脅。

本系統致力於開發具智慧記憶與分析能力的會議助理，強調跨會議資料整合搜尋與長期內容追蹤，透過即時語音轉錄與重點摘要，協助使用者快速掌握會議重點、追蹤任務進度，提升知識管理效率與決策品質。

優勢(Strengths)：

- 專注會議管理，整合語音轉錄、摘要與決策追蹤。
- 支援歷史資料整合與長期追蹤。
- 多元資料輸入輸出，提升工作效率。
- 模組化設計，適用多種組織類型。

劣勢(Weaknesses)：

- 依賴完整會議資料，效果受限。
- 長期記憶需持續累積資料。
- AI 回覆結果偶有偏差。
- 功能專注於會議，不適用其他場景。
- 初期知名度低，需加強推廣。

機會(Opportunities)：

- 企業與組織對會議管理與資料整合需求增加。
- 智慧助理與自動化整理接受度提升，應用場景廣泛。
- 現有系統流程尚未有效整合，具開發空間。

威脅(Threats)：

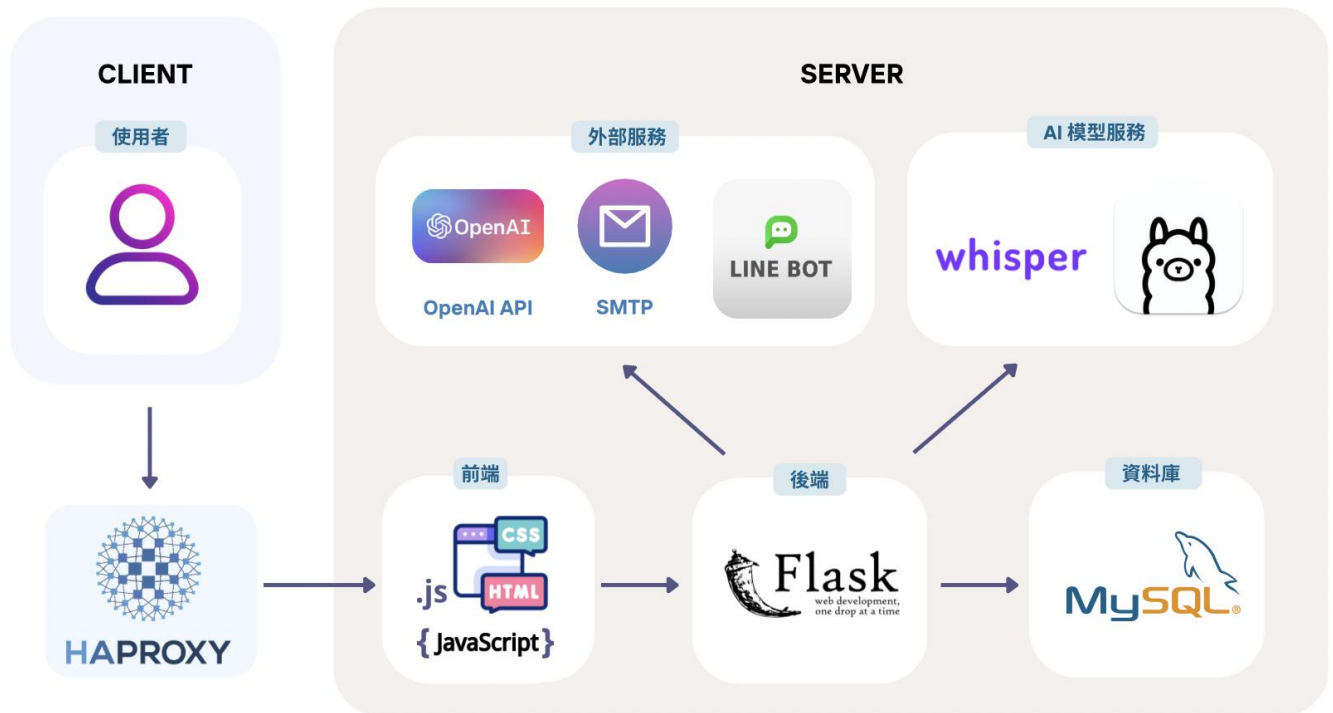
- 市面上有類似產品，競爭激烈。
- AI 摘要可能出現錯誤，影響信任度。
- 資安與法規要求嚴格，需符合合規標準。
- SaaS 價格敏感，需控制成本並提供誘因。

▼表 2-4-1SWOT-TOWS 分析表

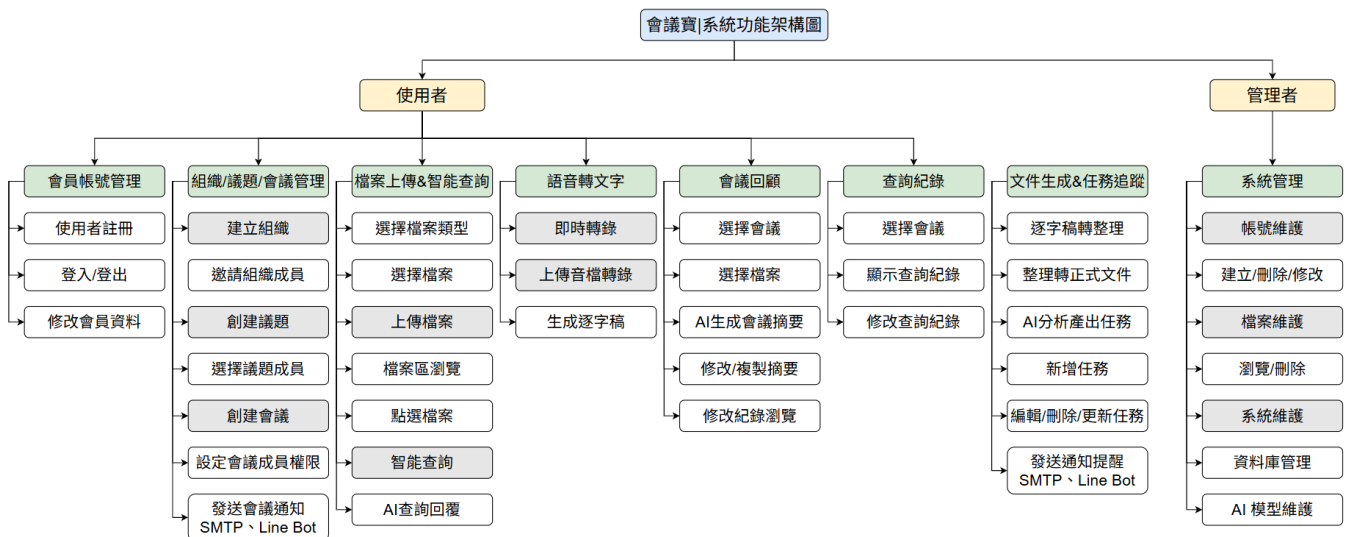
外部 內部	優勢(Strengths)	劣勢 (Weaknesses)
機會 (Opportunity)	◆ 利用會議管理功能和多種資料格式，滿足企業需求。 ◆ 模組化設計＋長期追蹤功能，開發會議智慧助理。 ◆ 把歷史會議資料整合，創造差異化服務。	◆ 鼓勵使用者持續上傳資料，強化長期記憶功能。 ◆ 提升 AI 回覆精準度，建立信任度。 ◆ 強化推廣，增加知名度。
威脅 (Threats)	◆ 用專業的會議管理功能，在競爭中脫穎而出。 ◆ 強化模組化與多元輸入輸出功能，應對價格敏感與多樣化需求。 ◆ 符合資安和法規要求，增加使用者信任。	◆ 改進 AI 與系統功能，減少錯誤和風險。 ◆ 專注會議功能，避免浪費資源在其他場景。 ◆ 控制成本並設優惠方案，吸引使用者並應對競爭。

第3章 系統規格

3-1 系統架構



▲圖 3-1-1 系統架構圖



▲圖 3-1-2 系統功能架構圖

3-2 系統軟、硬體需求與技術平台

▼表 3-2-1 系統軟、硬體需求

電腦	
作業系統	Windows10/11
中央處理器 CPU	4 核心以上
記憶體 RAM	8GB 系統記憶體
磁碟可用空間	100MB 以上可用空間
網路卡	Wi-Fi 無線網路卡

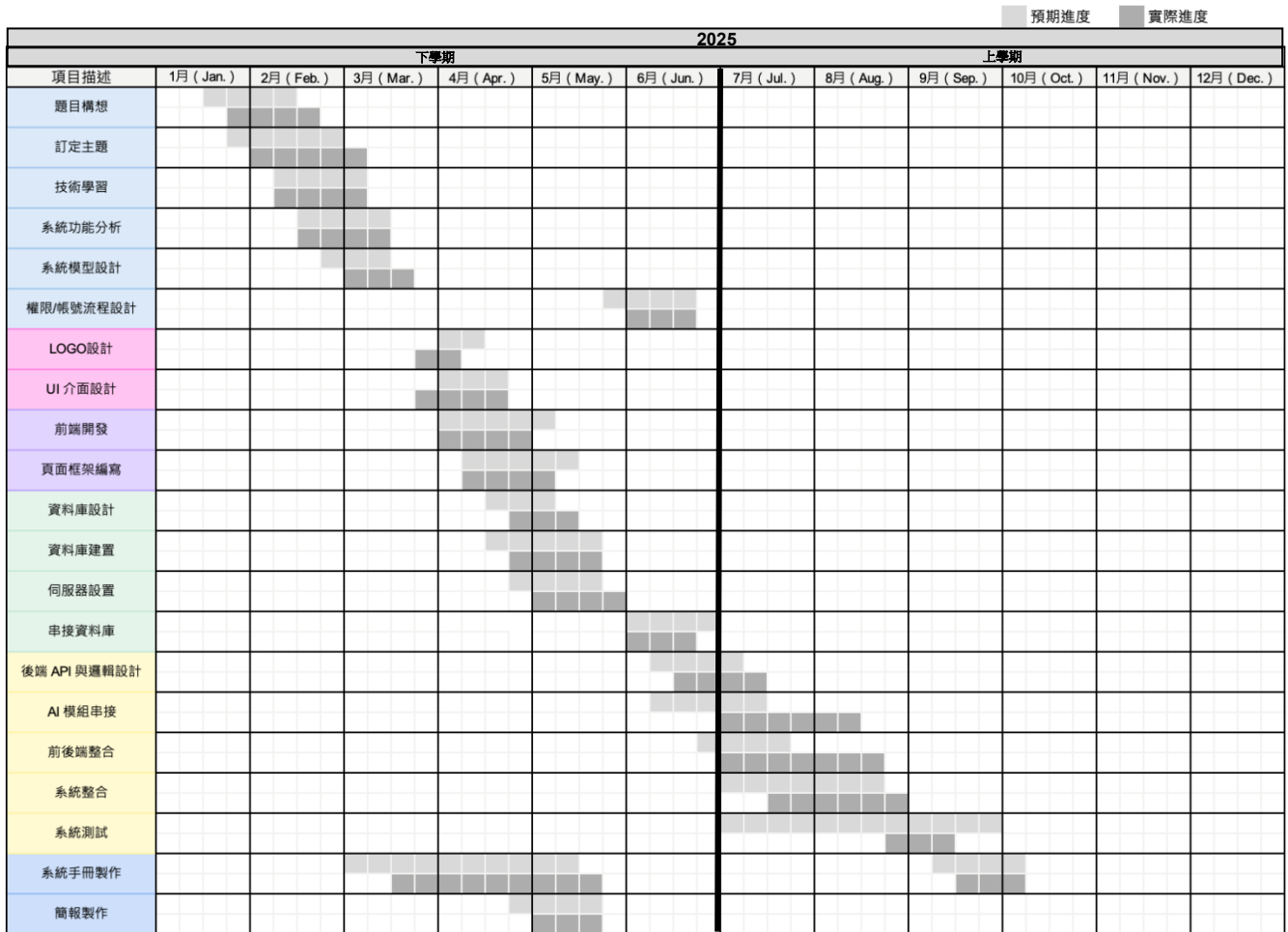
3-3 開發標準與使用工具

▼表 3-3-1 專案開發工具

系統開發環境	
作業系統	Windows11
資料庫	MySQL
伺服器	HAProxy
程式開發技術	
前端	HTML、JavaScript、CSS
後端	Python、Flask
編輯器	Visual Studio Code
通知發送	SMTP、LINE Bot
向量檢索	Embedding+FAISS
問答生成	RAG
語音處理	Whisper、OpenAI
大語言模型	Ollama、Llama
專案管理平台	
版本管理	Fork、GitHub
設計工具	
圖檔	Procreate
UML 圖	Draw.io
文件	Microsoft Word
簡報	Microsoft PowerPoint、Canva

第 4 章 專案時程與組織分工

4-1 專案時程



▲圖 4-1-1 專案時程甘特圖

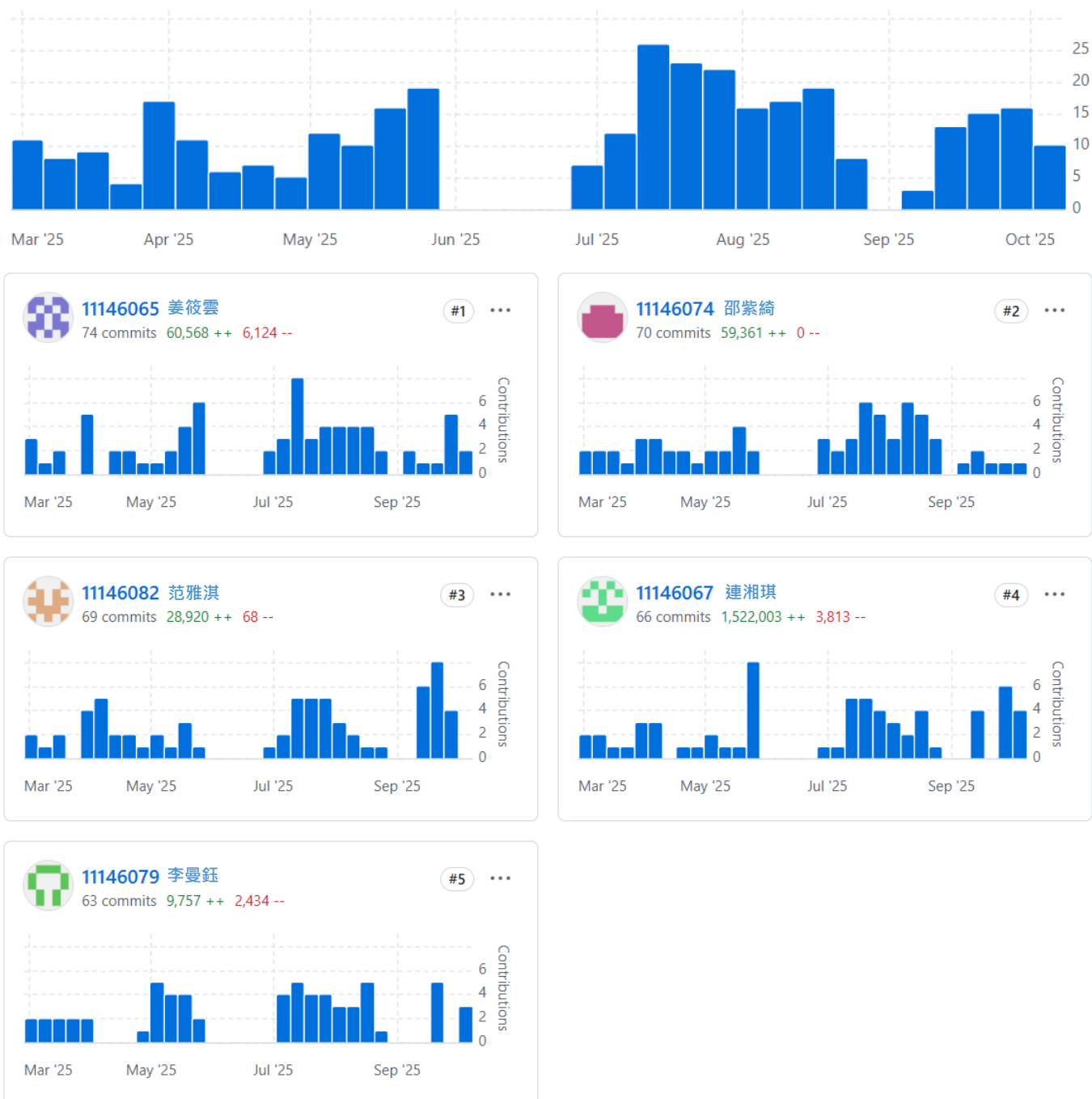
4-2 專案組織與分工

▼表 4-2-1 專題組織分工表

●主要負責人○次要負責人(每一項只能有 1 位主要負責人，次要負責人最多 2 位)

項目/組員		11146065 姜筱雲	11146067 連湘琪	11146074 邵紫綺	11146079 李曼鈺	11146082 范雅淇
後端 開發	資料庫建置		●			
	資料庫串接		●			
	Flask API		●	○		
	SMTP(郵件發送)	○			●	
	LINE Bot	●				
	Whisper+OpenAI(語音轉文字)				●	
	Ollama/Llama-智能查詢		●			○
	OpenAI-正式文件生成					●
	OpenAI-任務追蹤	●				○
	後端程式設計(權限設定)		○	●		
	組織/議題/會議管理			●		
前端 開發	Web 介面整合	●				
	會員註冊頁面	●				
	會員登入頁面	●				
	忘記密碼頁面	●				
	會員專區頁面					●
	建立組織頁面	●				
	組織成員列表頁面	●				
	創建議題頁面	●				
	議題成員列表頁面			●		
	創建會議頁面	●				
	會議成員列表頁面	●				
	檔案上傳&智能查詢頁面	●		●		
	語音轉文字頁面				●	
	會議回顧頁面					●
	查詢紀錄頁面		●			
	正式文件頁面		●			
	任務追蹤頁面	●				
美術 設計	Logo 設計		●			
	色彩設計				●	

文件 撰寫	統整		●			
	第 1 章 前言		○			●
	第 2 章 營運計畫			●	○	○
	第 3 章 系統規格	●			○	
	第 4 章 專題時程與組織分工	○			●	
	第 5 章 需求模型	●				
	第 6 章 設計模型		○	●		○
	第 7 章 實作模型			○		●
	第 8 章 資料庫設計		●			
	第 9 章 程式	○			●	○
	第 10 章 測試模型				●	
	第 11 章 操作手冊			●		
	第 12 章 使用手冊		●			
	第 13 章 感想	●				
	第 14 章 參考資料			●		
報告	附錄	●		○		
	簡報製作			●		
	影片製作			●		



▲圖 4-2-1 進度文件資料上傳 Github 貢獻次數

▼表 4-2-2 專題工作內容與貢獻度表

序號	姓名	工作內容<各限 100 字以內>	貢獻度
1	組長 <u>姜筱雲</u>	負責串接 SMTP、LINE BOT 及 OpenAI 任務追蹤功能，前端負責大部分頁面開發及全頁面整合，撰寫文件 3、4、5、9、13 章以及附錄二，並完成競賽報名資料填寫及報名作業。	<u>20</u> %
2	組員 <u>連湘琪</u>	負責建置資料庫及其串接，後端程式設計與 Ollama/Llama 智能查詢功能開發，前端負責少數頁面開發，LOGO 設計，撰寫文件 1、6、8、12 章並統整全份文件，完成競賽報名資料填寫。	<u>20</u> %
3	組員 <u>邵紫綺</u>	負責後端程式設計及前端少數頁面開發，撰寫文件 2、6、7、11 章，完成會議紀錄重點整理、簡報製作及影片製作，以支援專案整體呈現與報告。	<u>20</u> %
4	組員 <u>李曼鈺</u>	負責 SMTP 郵件發送與 Whisper+OpenAI 語音轉文字功能開發，前端負責單頁面，撰寫文件 2、3、4、9、10 章，並完成競賽報名資料填寫作業。	<u>20</u> %
5	組員 <u>范雅淇</u>	負責 OpenAI 正式文件生成功能及前端少數頁面開發，撰寫文件 1、2、6、7、9 章，協助專案文件整理與內容產出，並完成競賽報名資料填寫。	<u>20</u> %
			總計:100%

第5章 需求模型

5-1 使用者需求

1.功能性需求 (FunctionalRequirement)

▼表 5-1-1 使用者功能性需求表

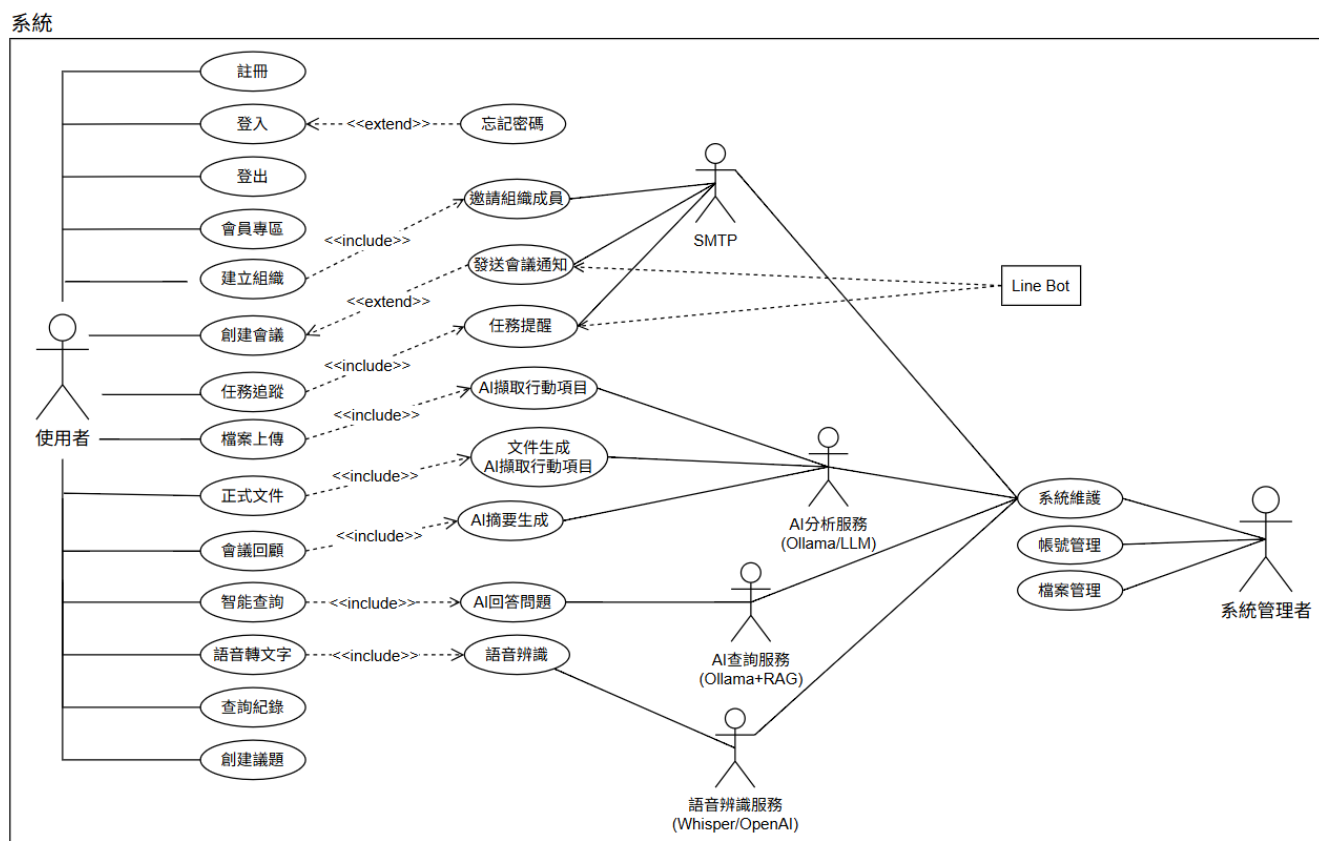
功能性需求	描述
註冊與登入	使用者可以註冊帳號成為會員並登入系統。
	使用者可以編輯及更新個人資料。
會議提醒通知	使用者可於會議前發送會議提醒通知，包括會議日期、名稱及議程。
建立組織	使用者可以建立組織，設定組織名稱並邀請成員加入。
創建議題	使用者可以在組織中創建議題，方便後續討論與追蹤。
創建會議	使用者可以在議題內發起會議，選擇參與成員並設定操作權限。
檔案上傳與智能查詢	使用者可以上傳與會議相關的檔案以便後續檢視。
	使用者可針對會議內容提出問題，系統提供即時回覆與說明。
語音轉文字	使用者可於會議中即時錄音，系統同步轉換為文字並整理成逐字稿。
	使用者可於會議後上傳錄音檔，系統將轉換為文字並整理成逐字稿。
會議回顧	使用者可利用會議回顧功能生成歷史會議摘要，並提供複製與修改功能。
正式文件生成	使用者可將會議逐字稿整理轉換成正式文件，方便後續存檔或分享。
任務追蹤	使用者可建立任務並追蹤進度，確保會議後工作落實，並可發送任務提醒。

2.非功能性需求 (Non-FunctionalRequirement)

▼表 5-1-2 使用者非功能性需求表

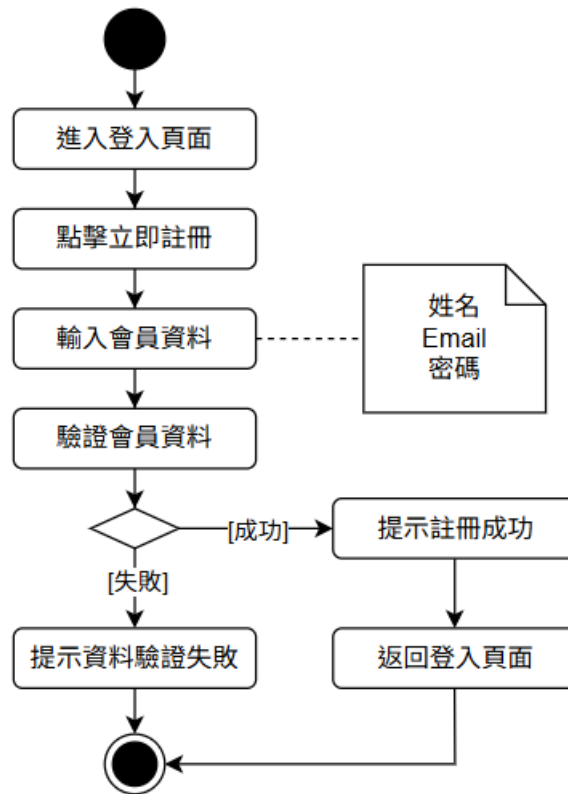
非功能性需求	描述
效能	系統能快速回應使用者查詢及操作請求。
	系統能處理並儲存會議紀錄與檔案，確保即時回顧與搜尋功能順暢。
準確度	語音轉文字功能精準轉錄會議內容，並正確識別發言者。
	會議摘要與任務追蹤資料準確反映會議內容，減少人工修改。
使用者介面	系統介面直覺簡單，使用者能快速創建議題、會議及管理檔案。
	功能設計符合人機互動原則，使用者操作流暢、易上手。
擴展性	系統架構具彈性，可整合更多第三方工具與資料來源。
	系統能應對資料量增加及功能需求變化，保持長期穩定運作。

5-2 使用個案圖（UseCaseDiagram）

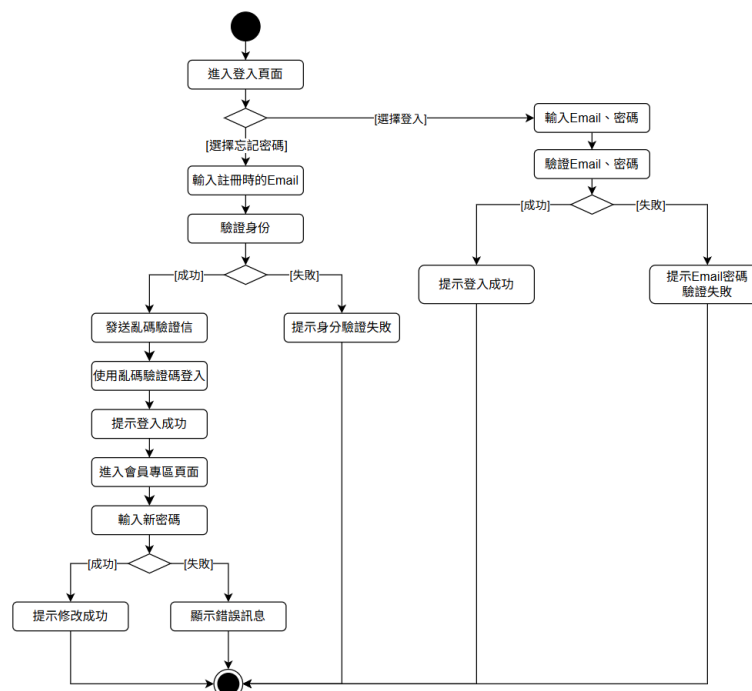


▲圖 5-2-1 使用個案圖

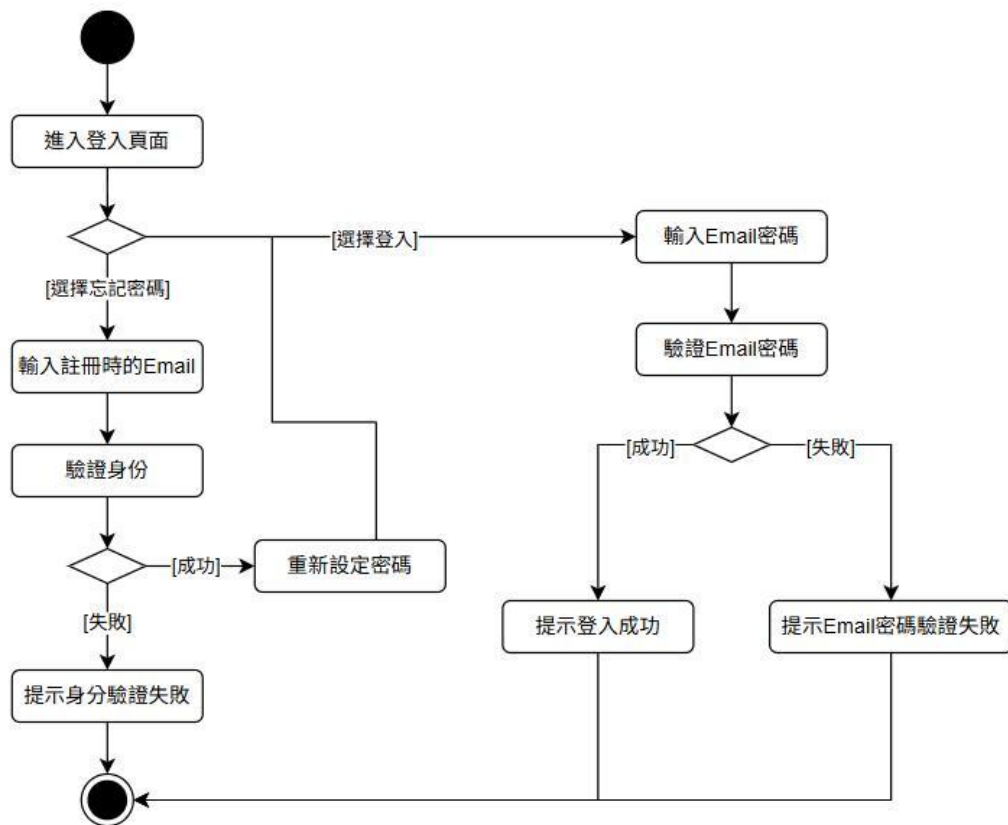
5-4 使用個案描述-活動圖



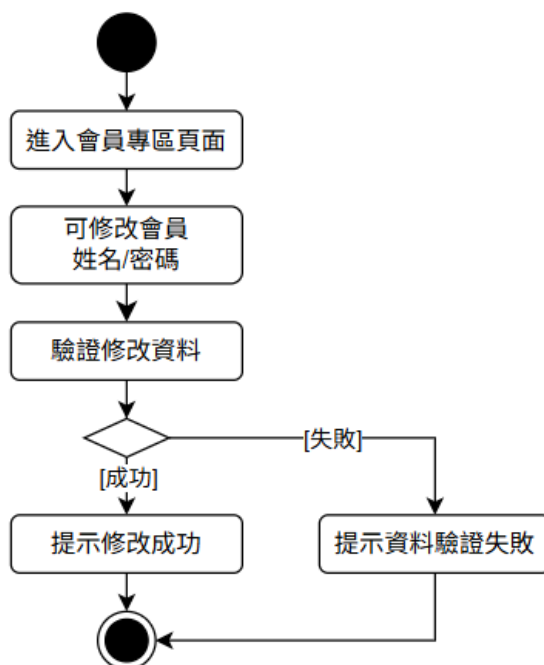
▲圖 5-4-1 會員註冊之活動圖



▲圖 5-4-2 會員登入之活動圖



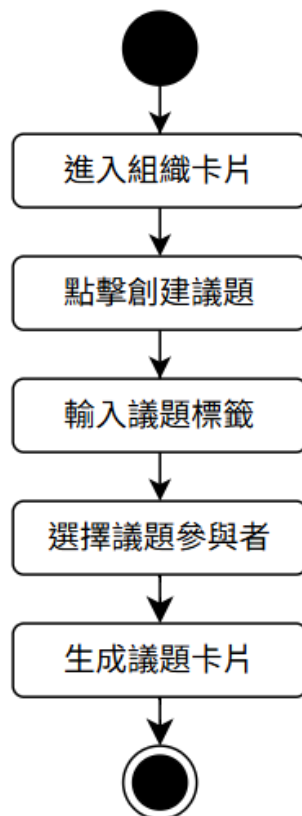
▲圖 5-4-3 會員忘記密碼之活動圖



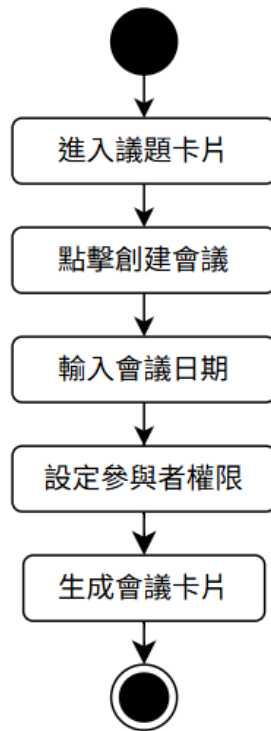
▲圖 5-4-4 會員資料管理之活動圖



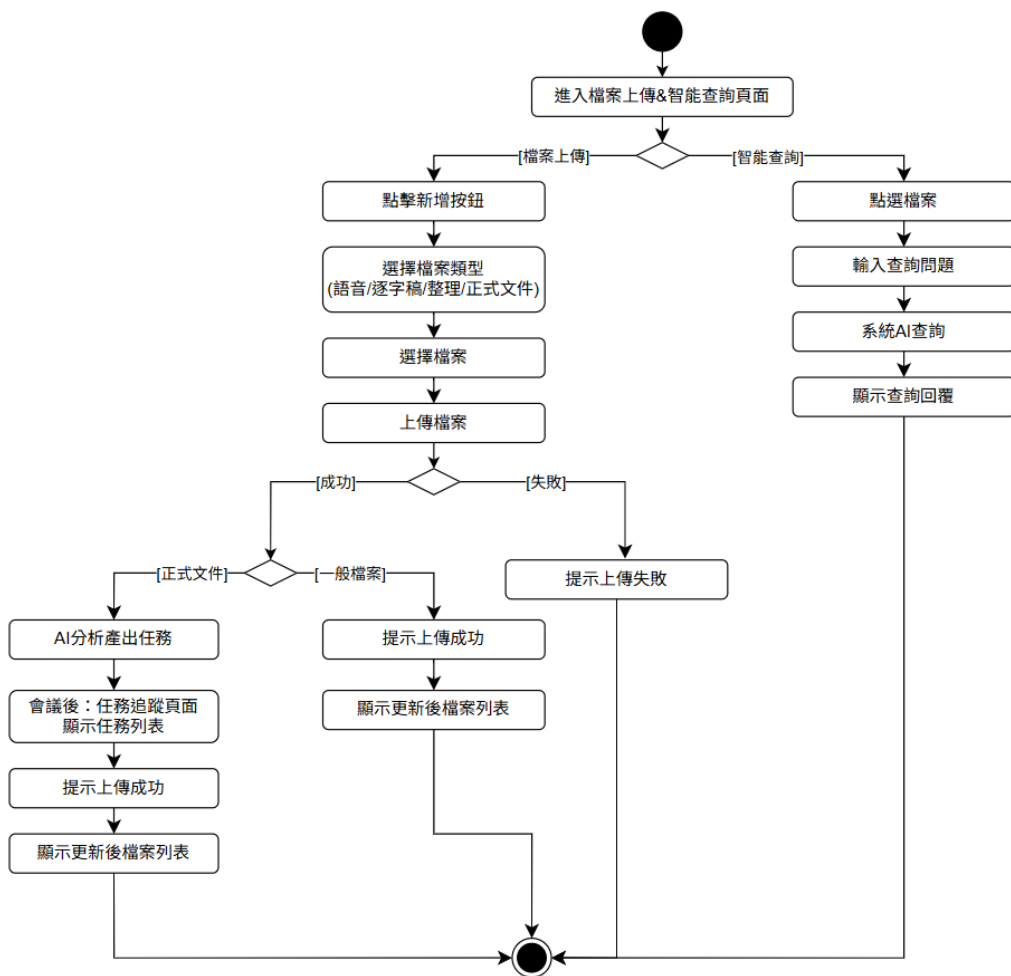
▲圖 5-4-5 建立組織之活動圖



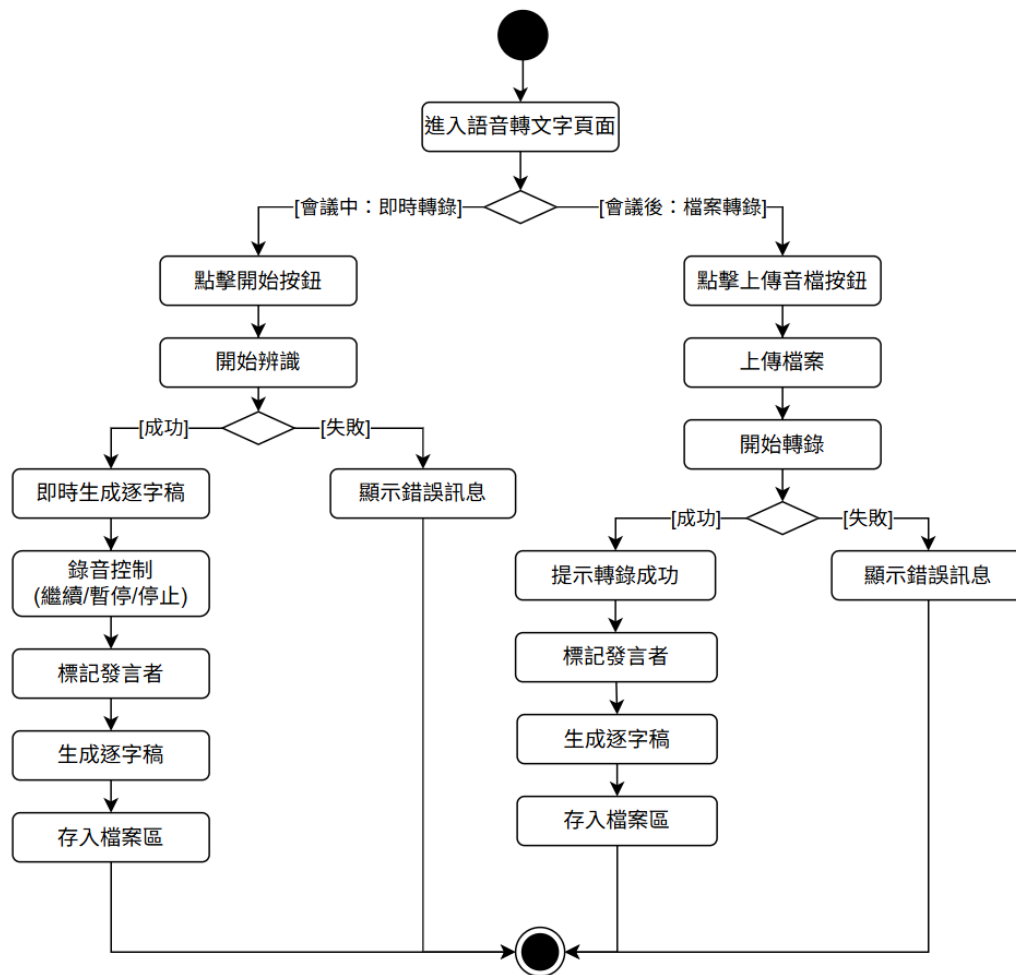
▲圖 5-4-6 創建議題之活動圖



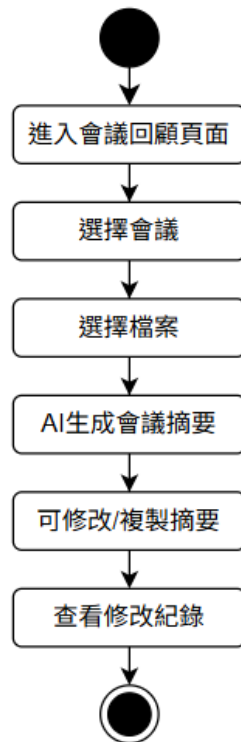
▲圖 5-4-7 發起會議之活動圖



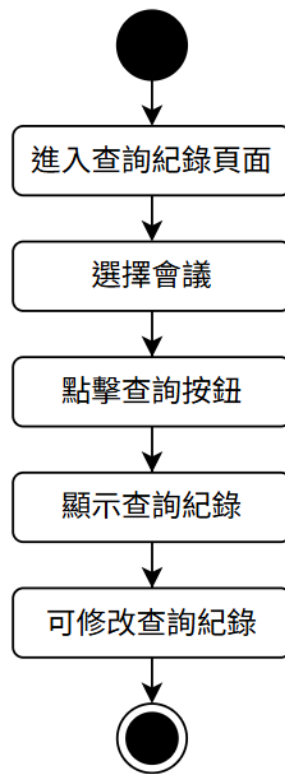
▲圖 5-4-8 檔案上傳之活動圖



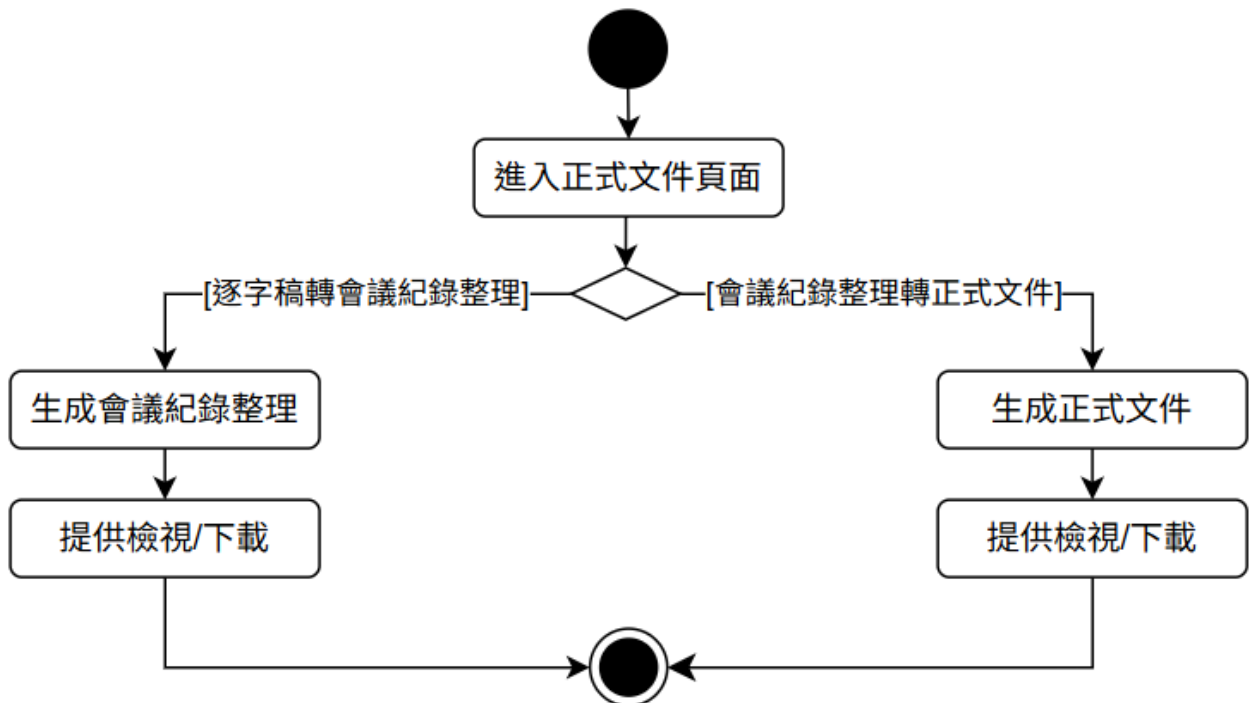
▲圖 5-4-9 語音轉文字之活動圖



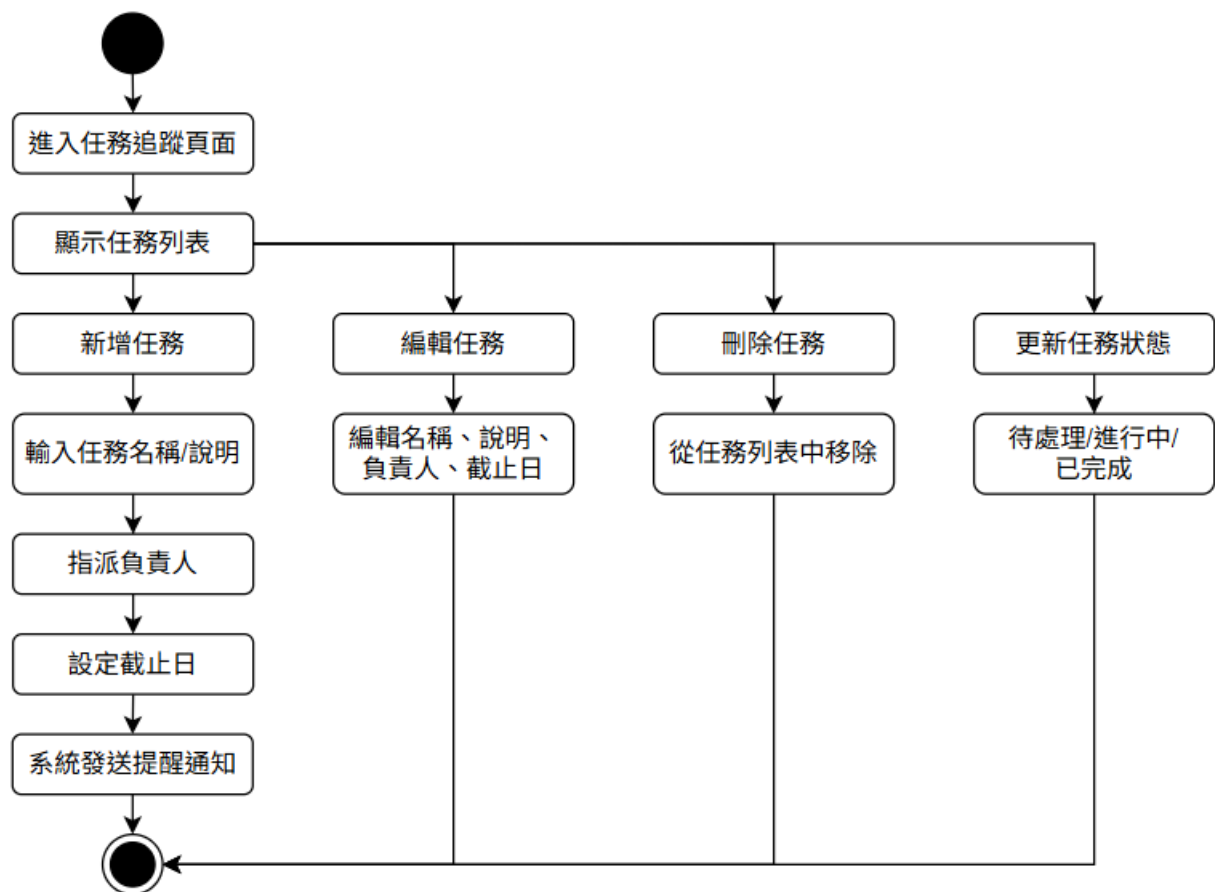
▲圖 5-4-10 會議回顧之活動圖



▲圖 5-4-11 查詢紀錄之活動圖

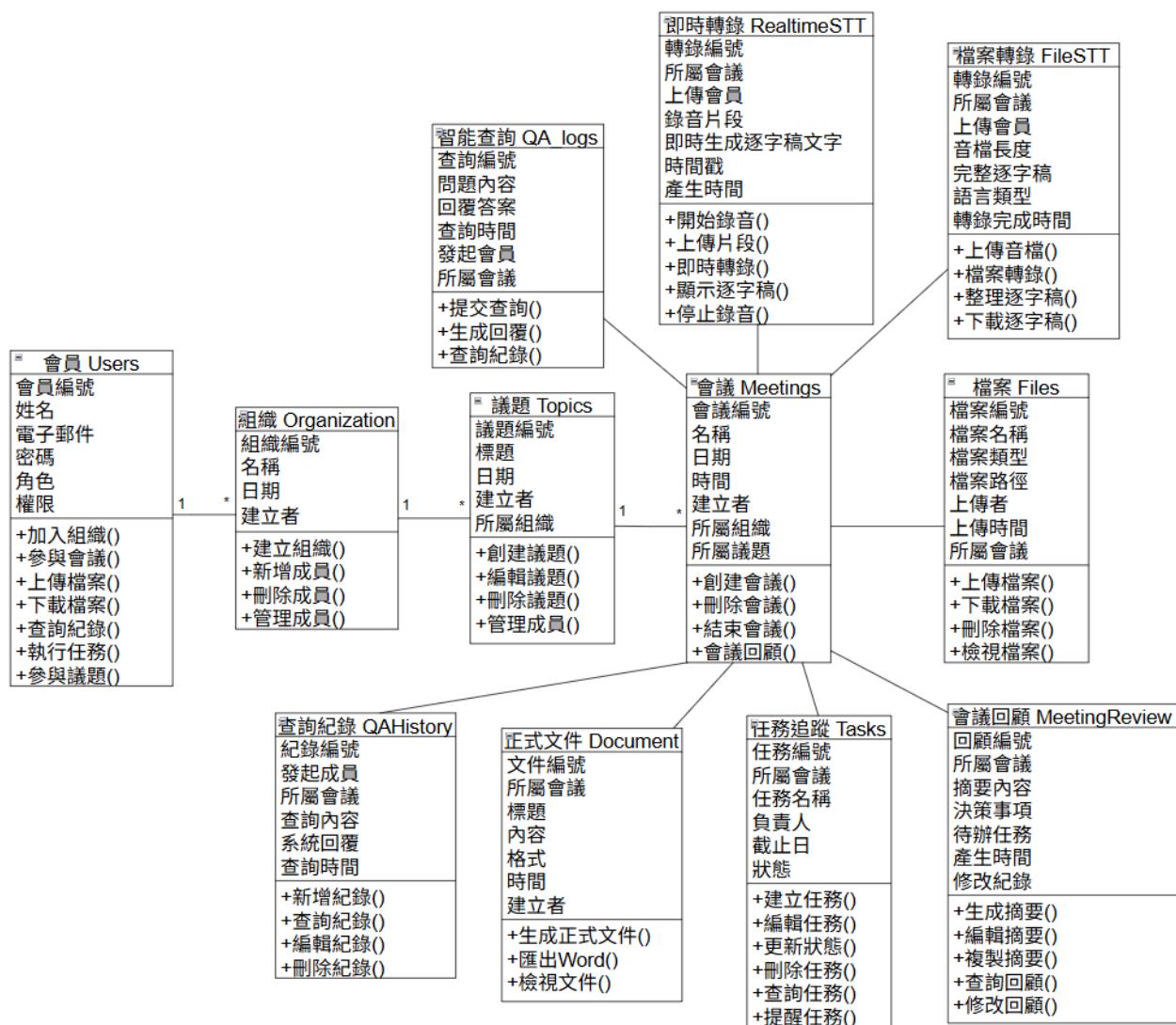


▲圖 5-4-12 正式文件之活動圖



▲圖 5-4-13 任務追蹤之活動圖

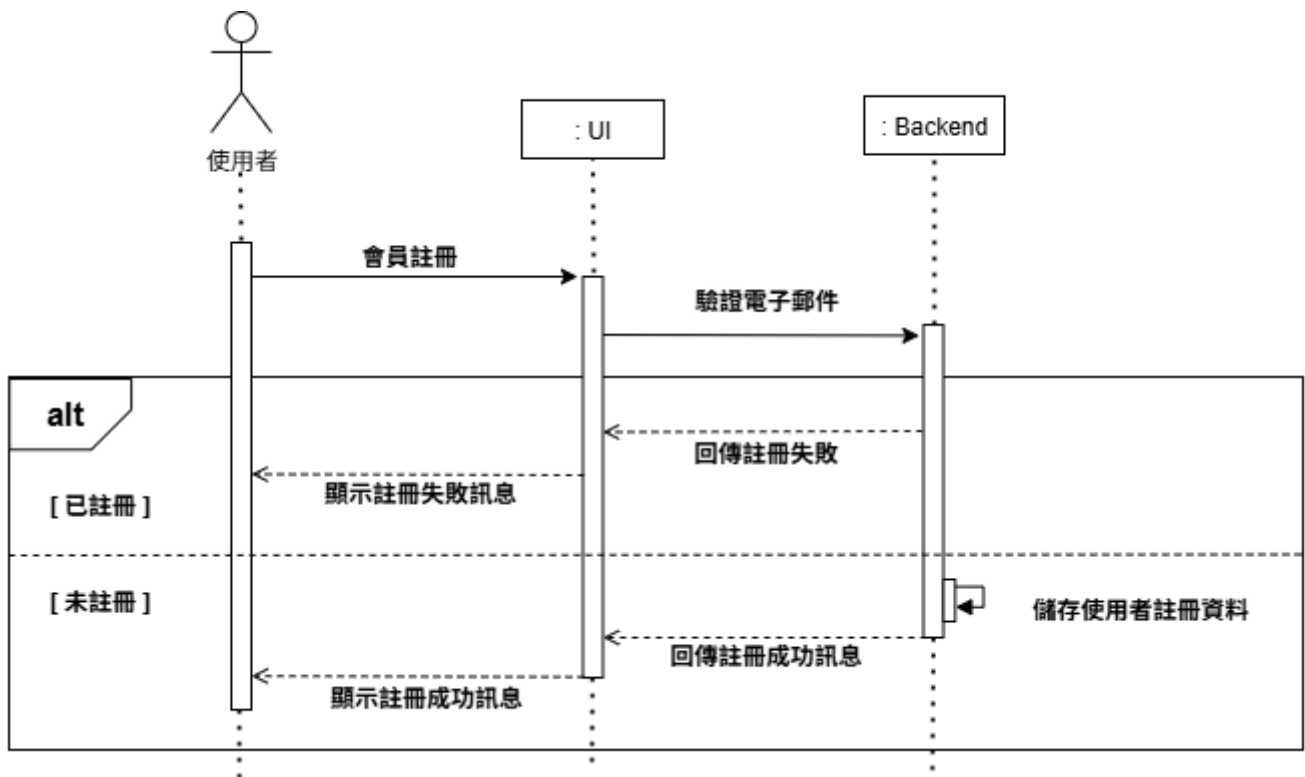
5-5 分析類別圖 (AnalysisClassDiagram)



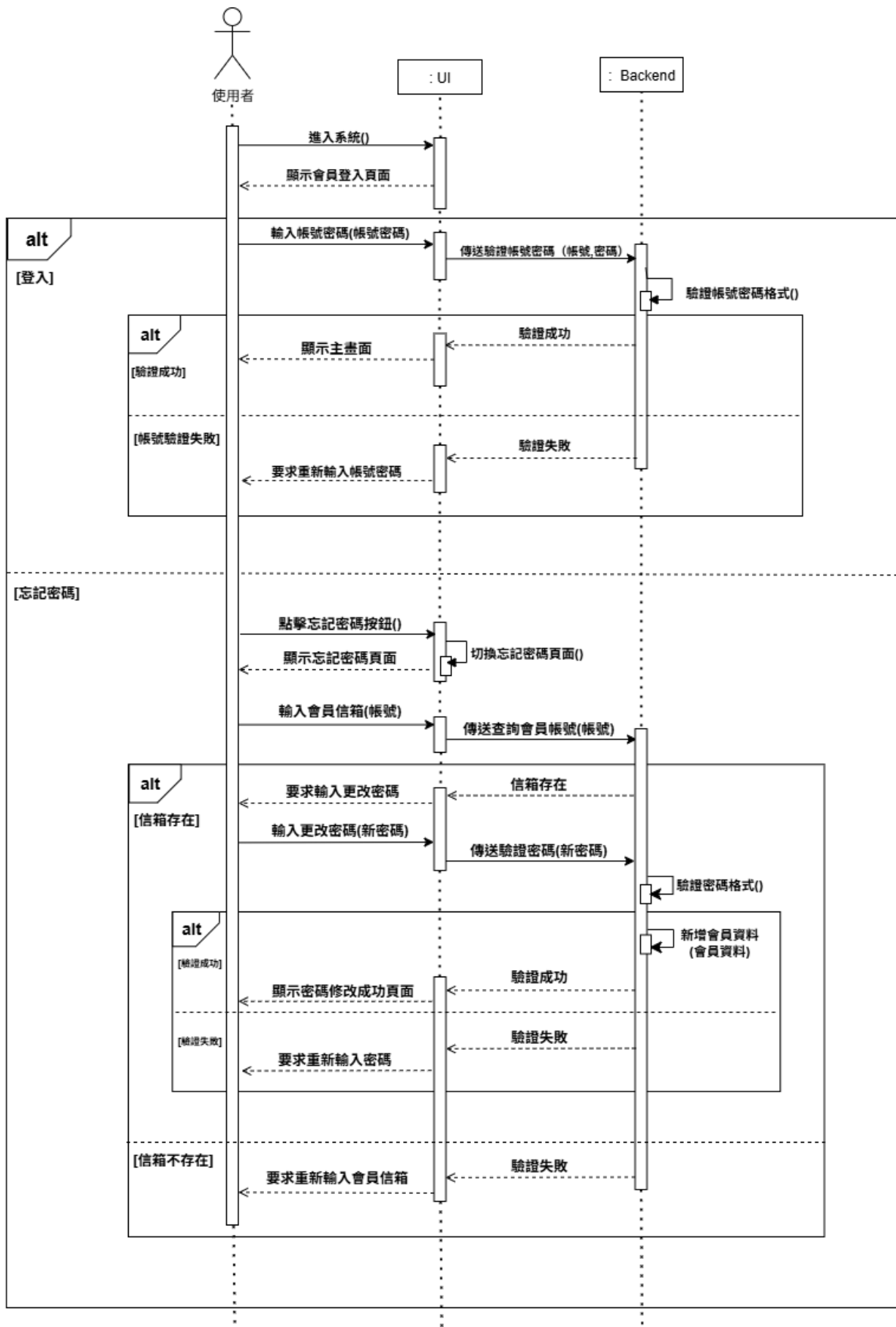
▲圖 5-5-1 分析類別圖

第6章 設計模型

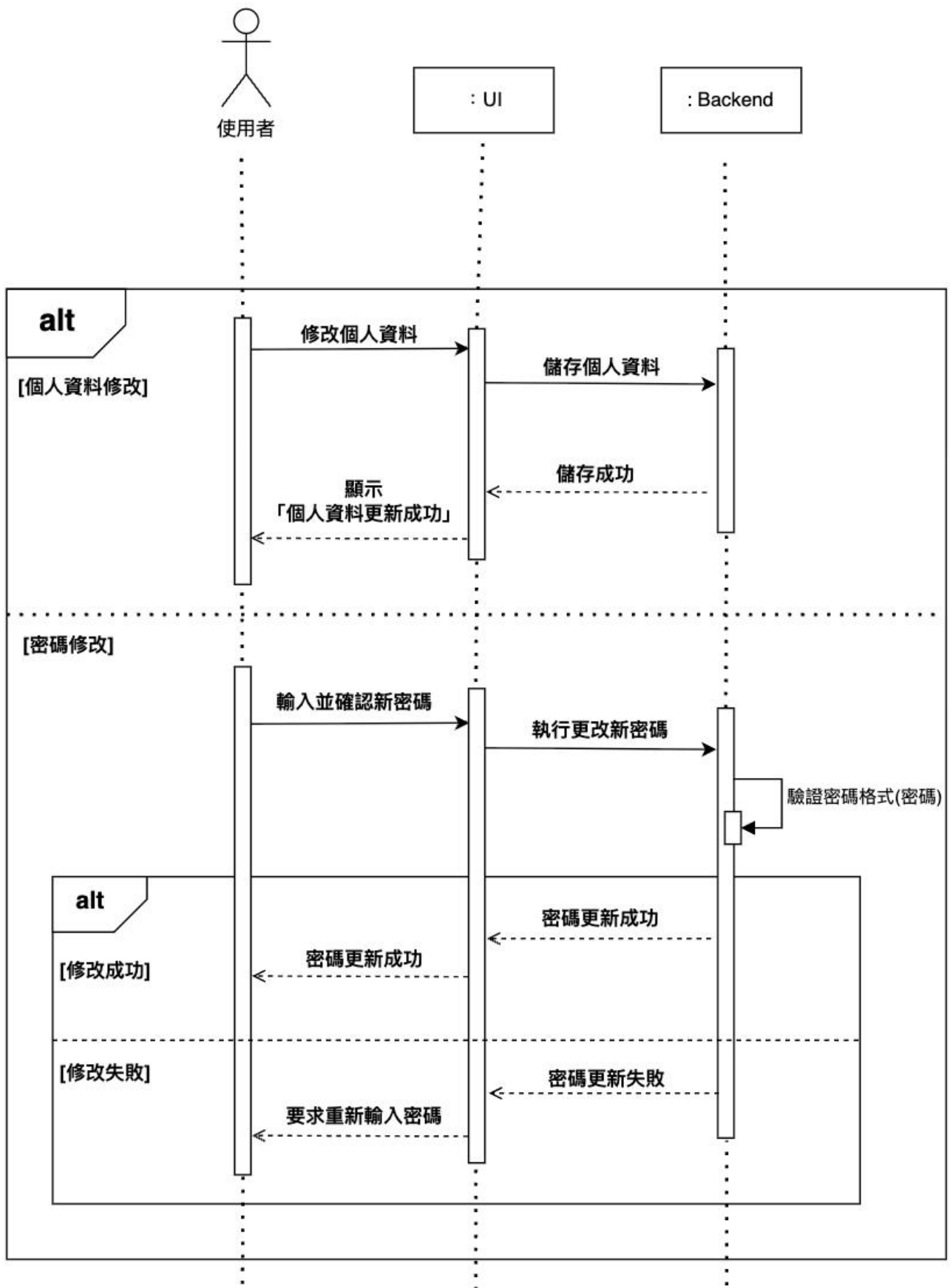
6-1 循序圖 (SequentialDiagram)



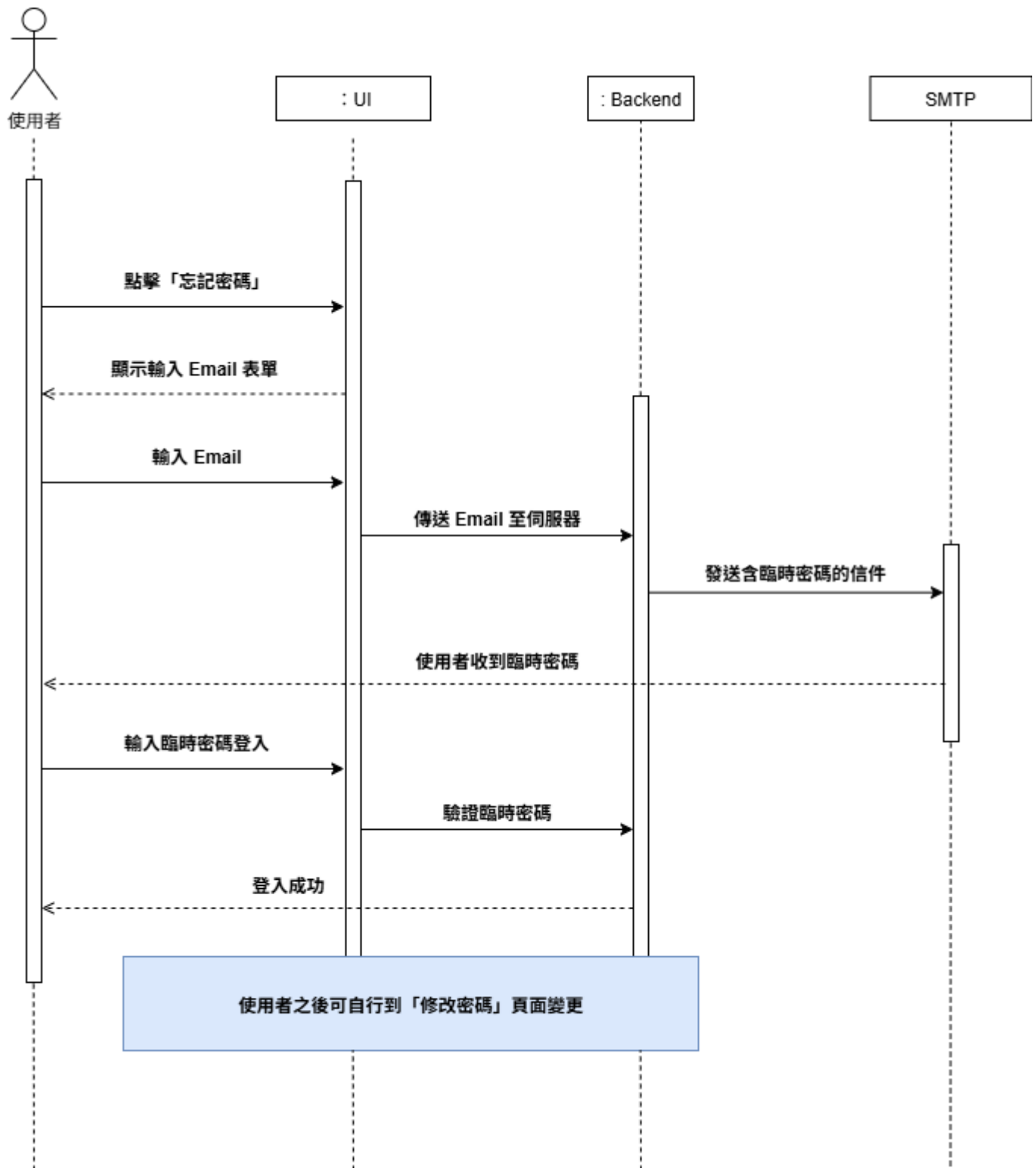
▲圖 6-1-1 會員註冊之循序圖



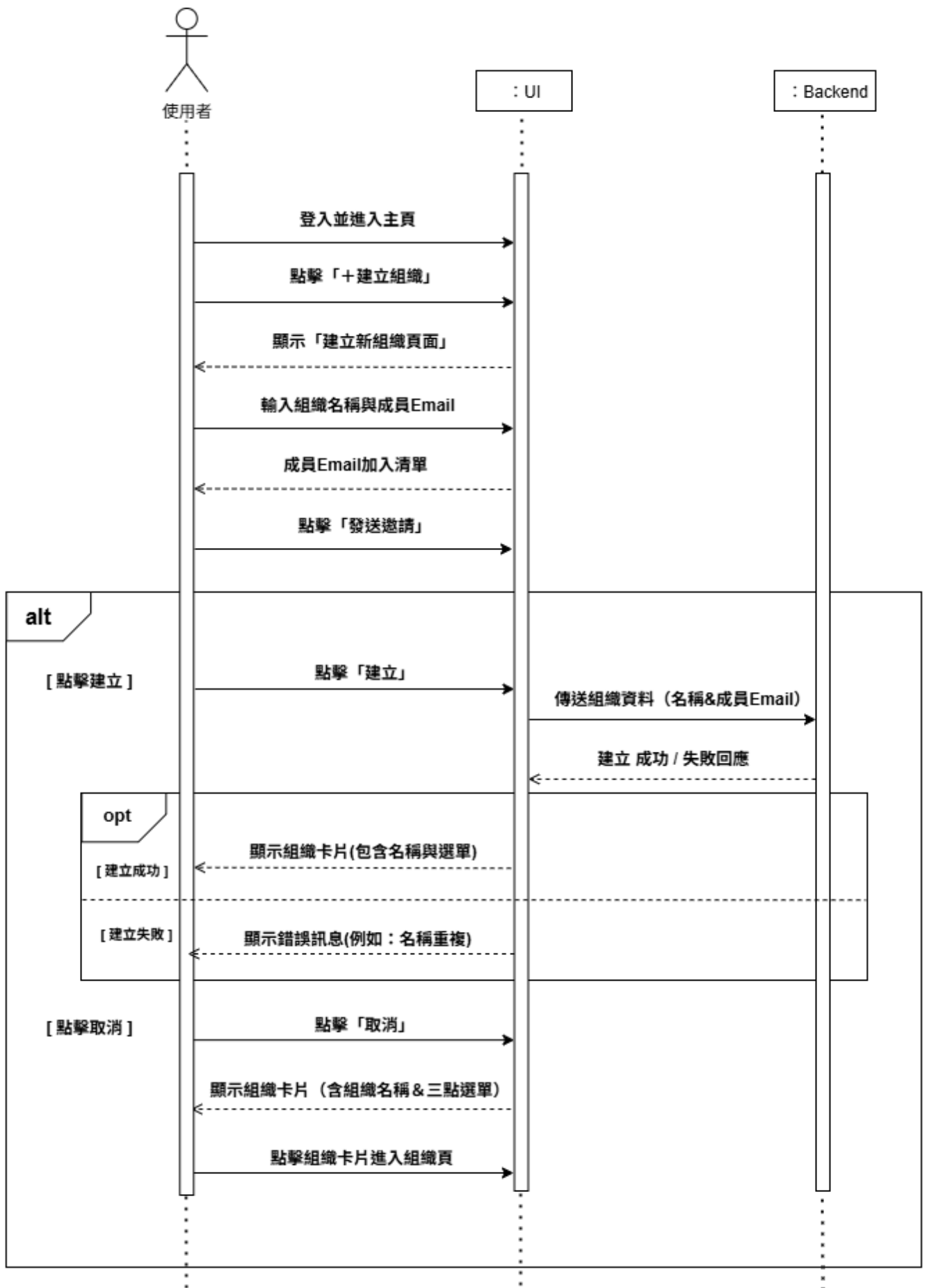
▲圖 6-1-2 會員登入之循序圖



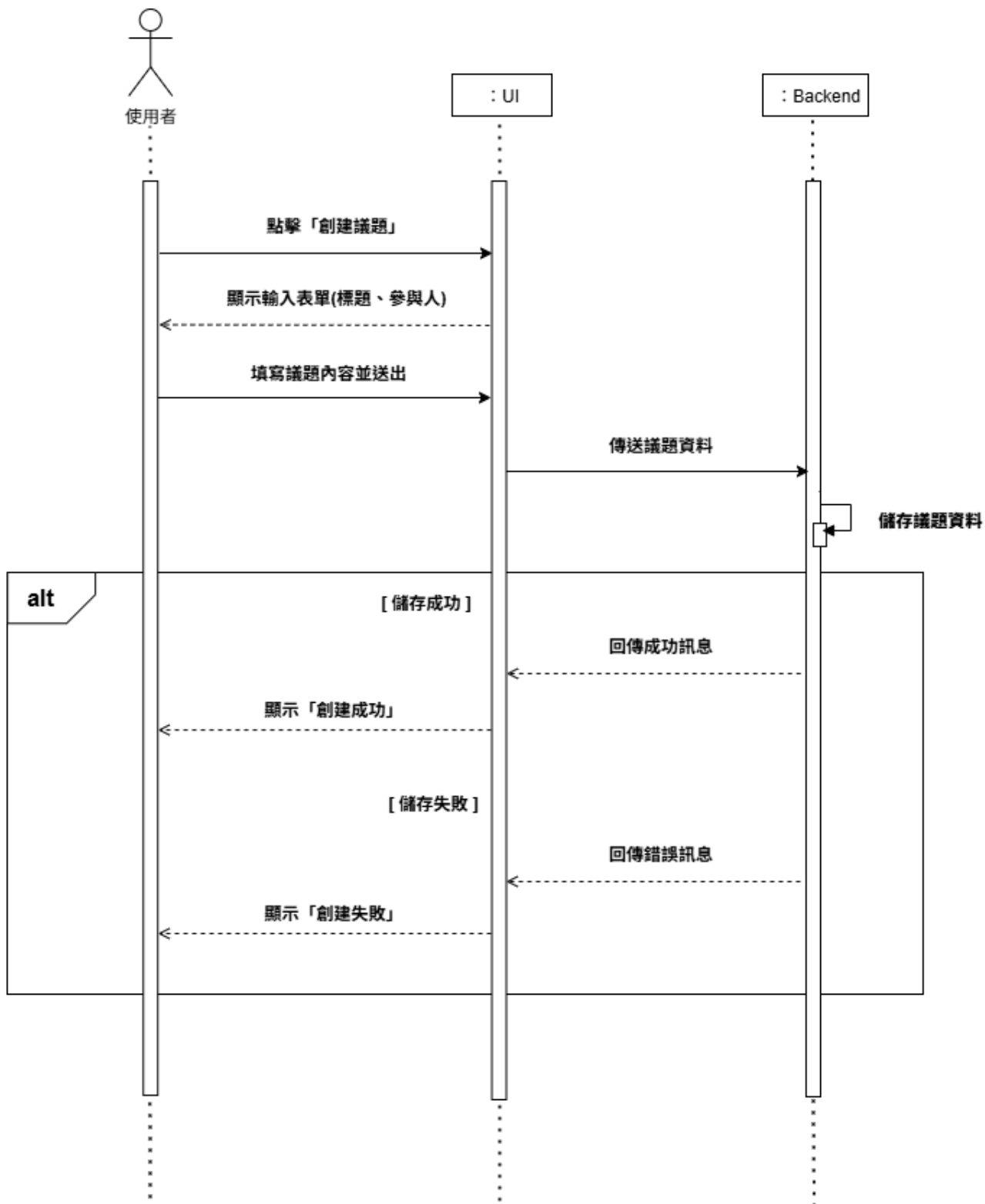
▲圖 6-1-3 會員資料管理之循序圖



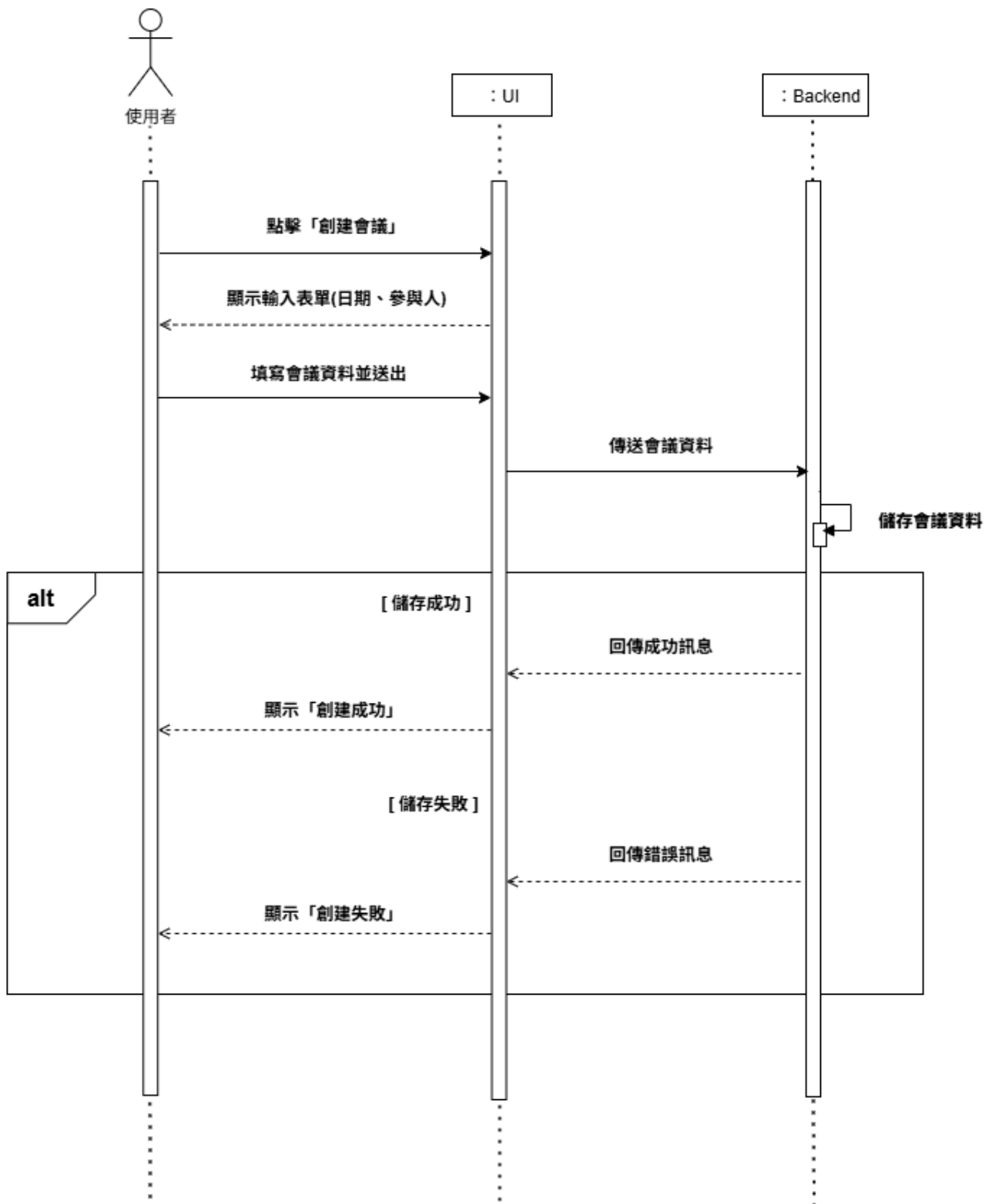
▲圖 6-1-4 忘記密碼之循序圖



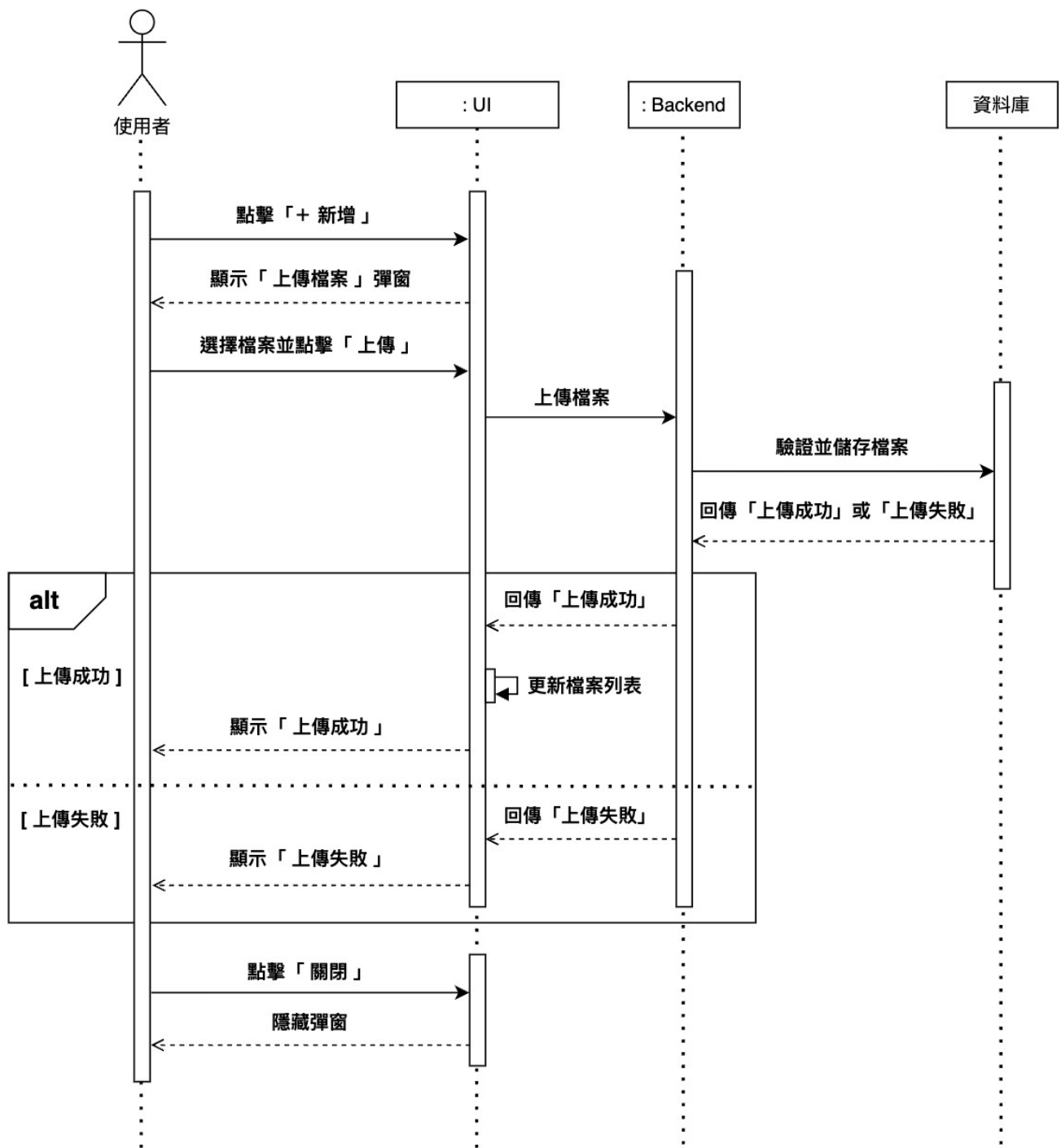
▲圖 6-1-5 建立組織之循序圖



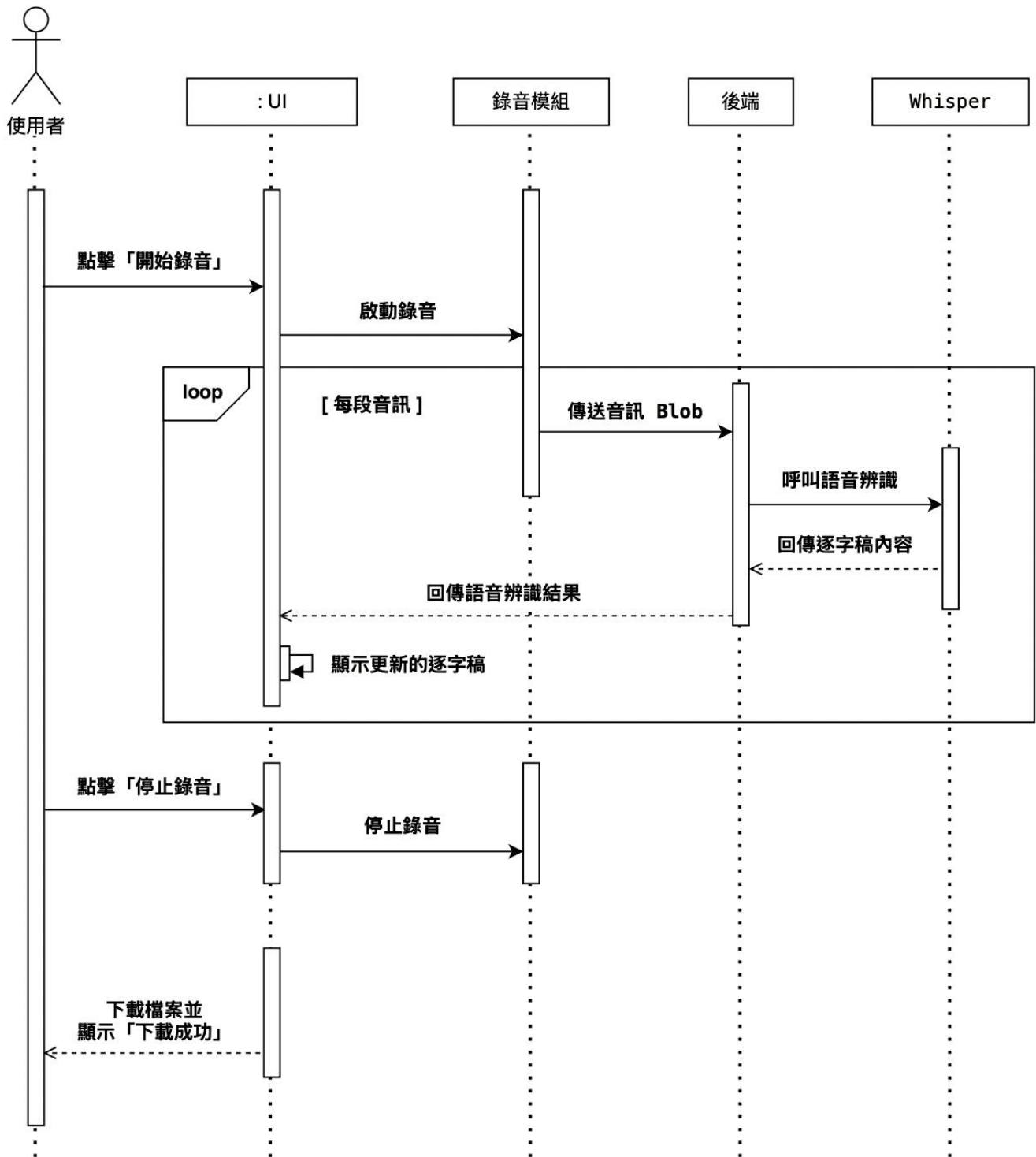
▲圖 6-1-6 創建議題之循序圖



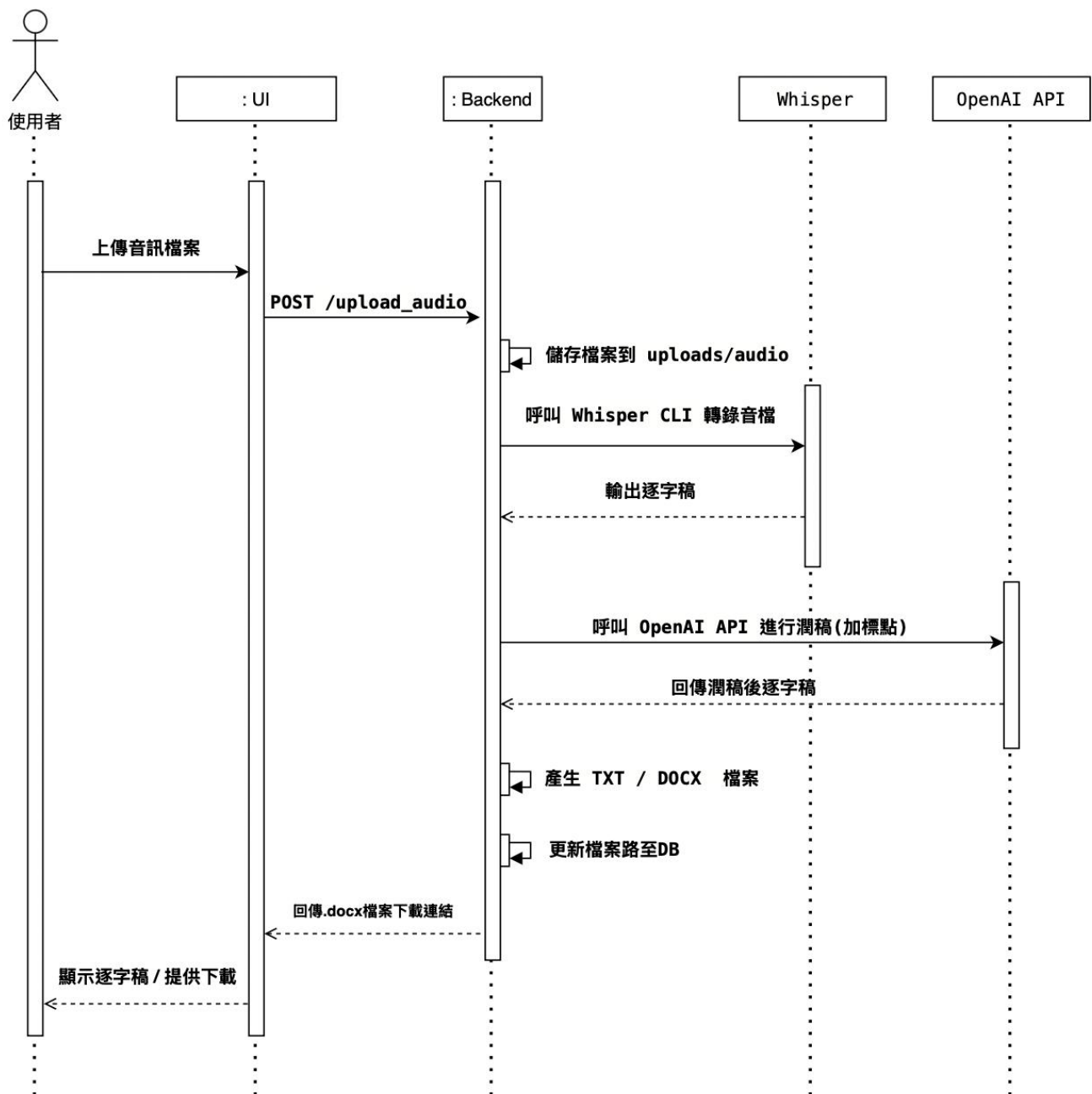
▲圖 6-1-7 發起會議之循序圖



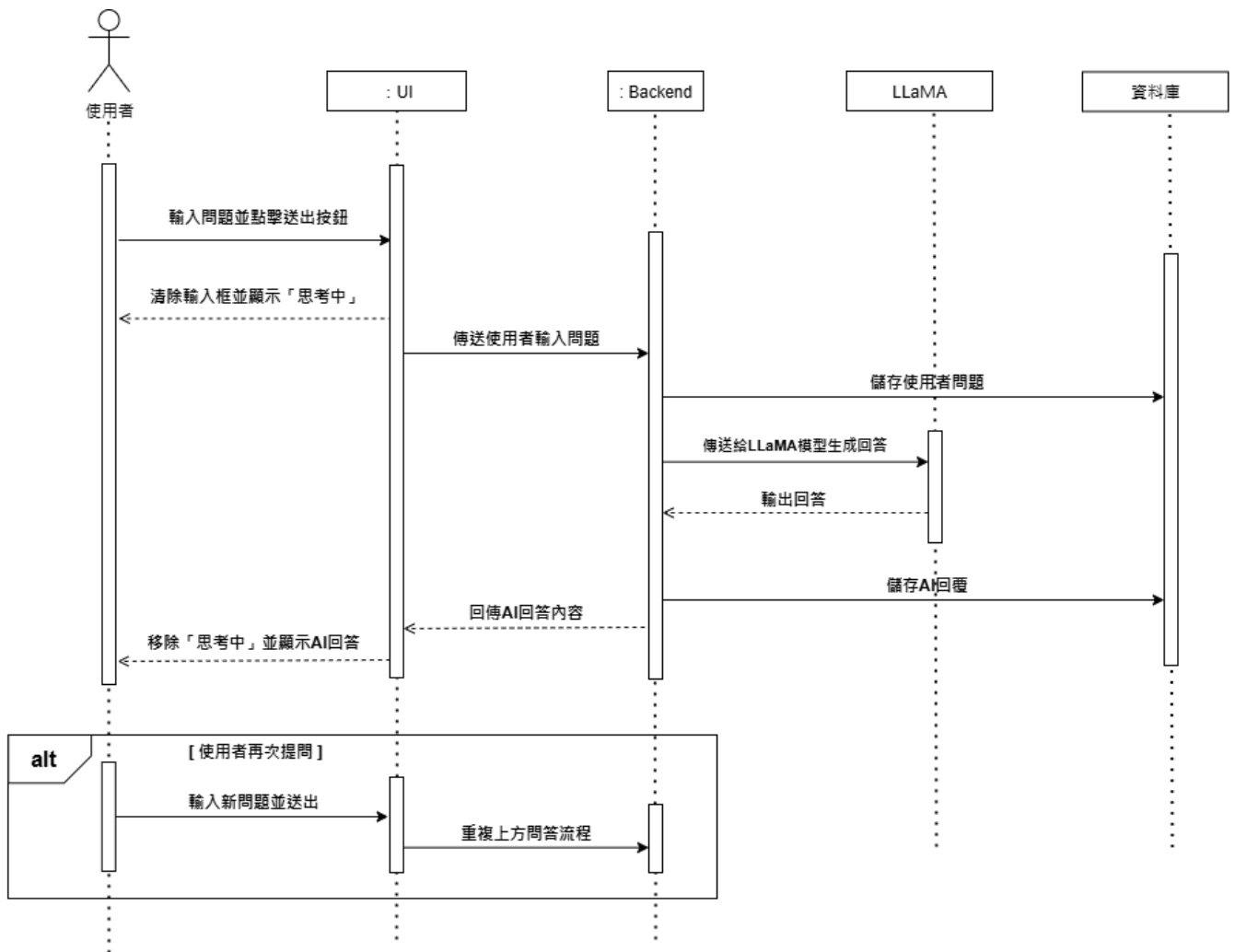
▲圖 6-1-8 檔案上傳之循序圖



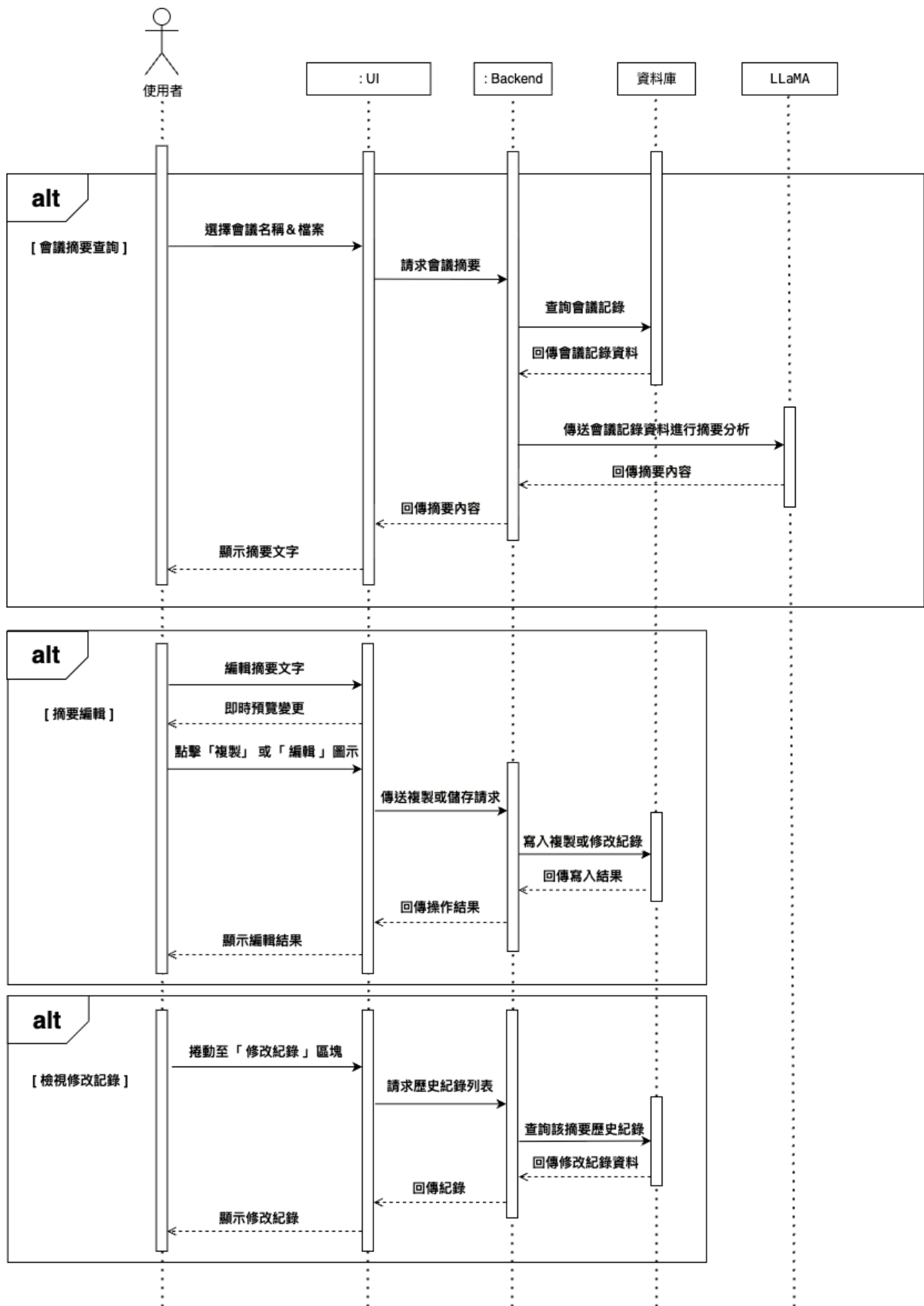
▲圖 6-1-9 即時轉錄之循序圖



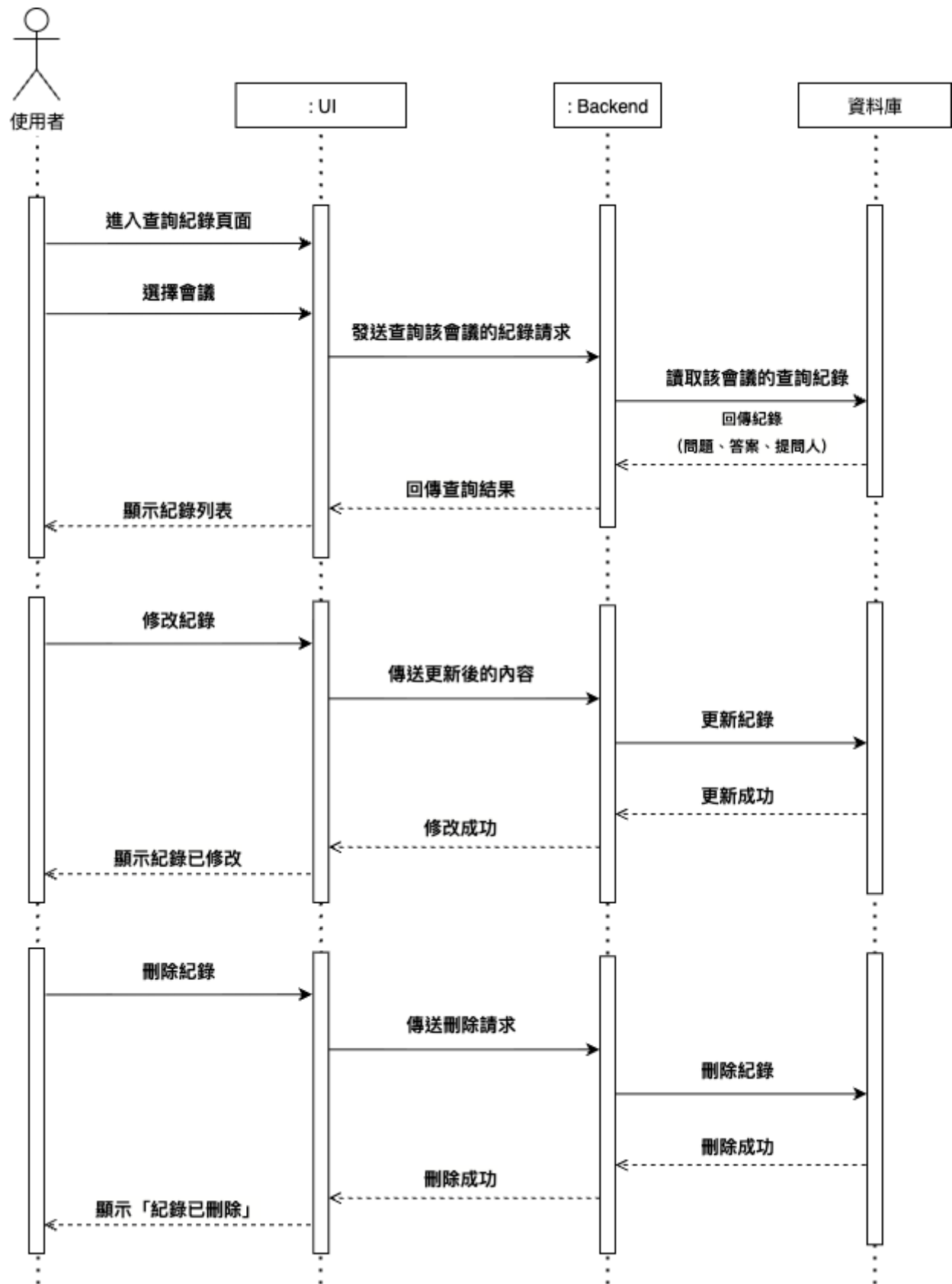
▲圖 6-1-10 檔案轉錄之循序圖



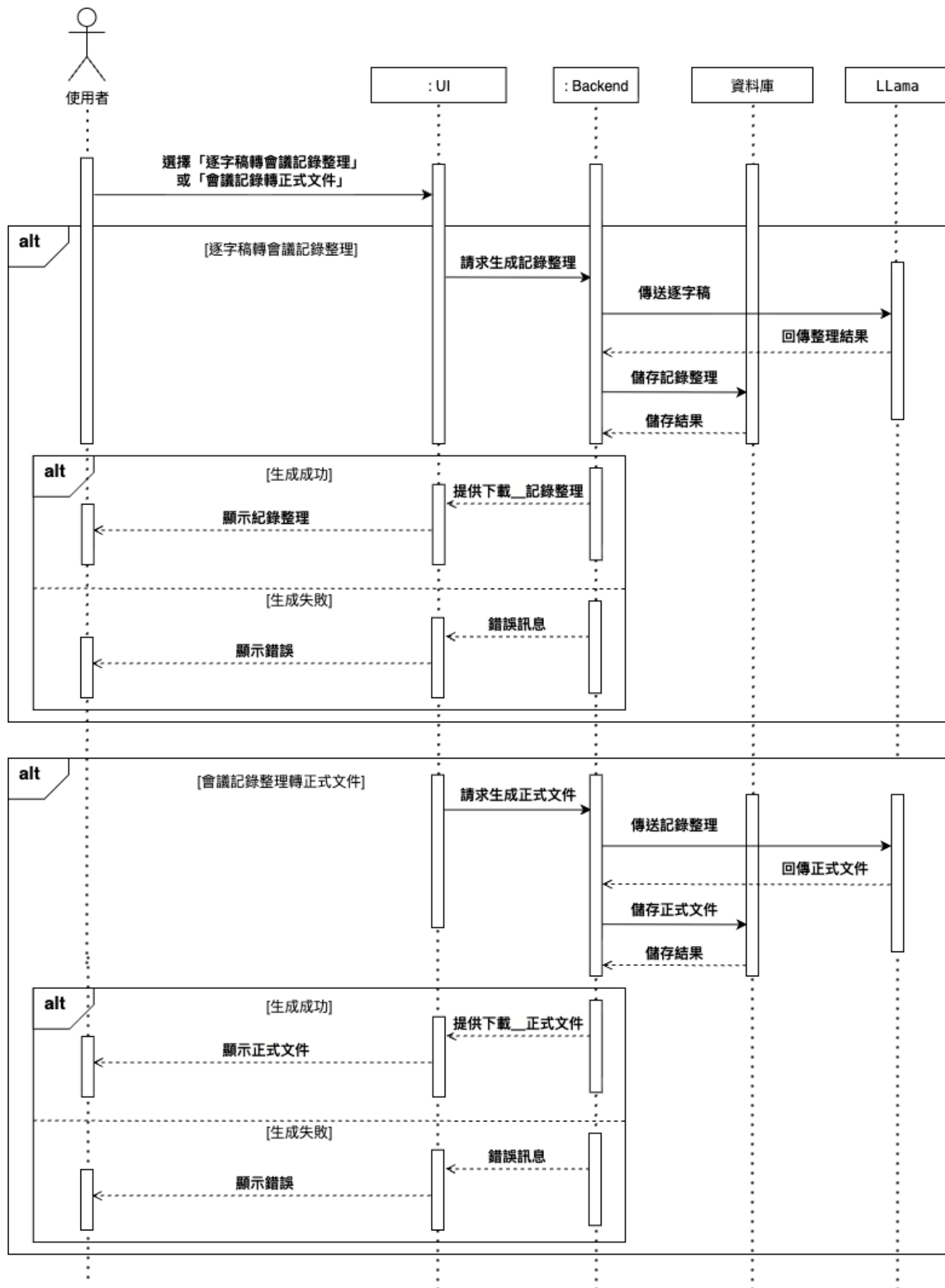
▲圖 6-1-11 智能查詢之循序圖



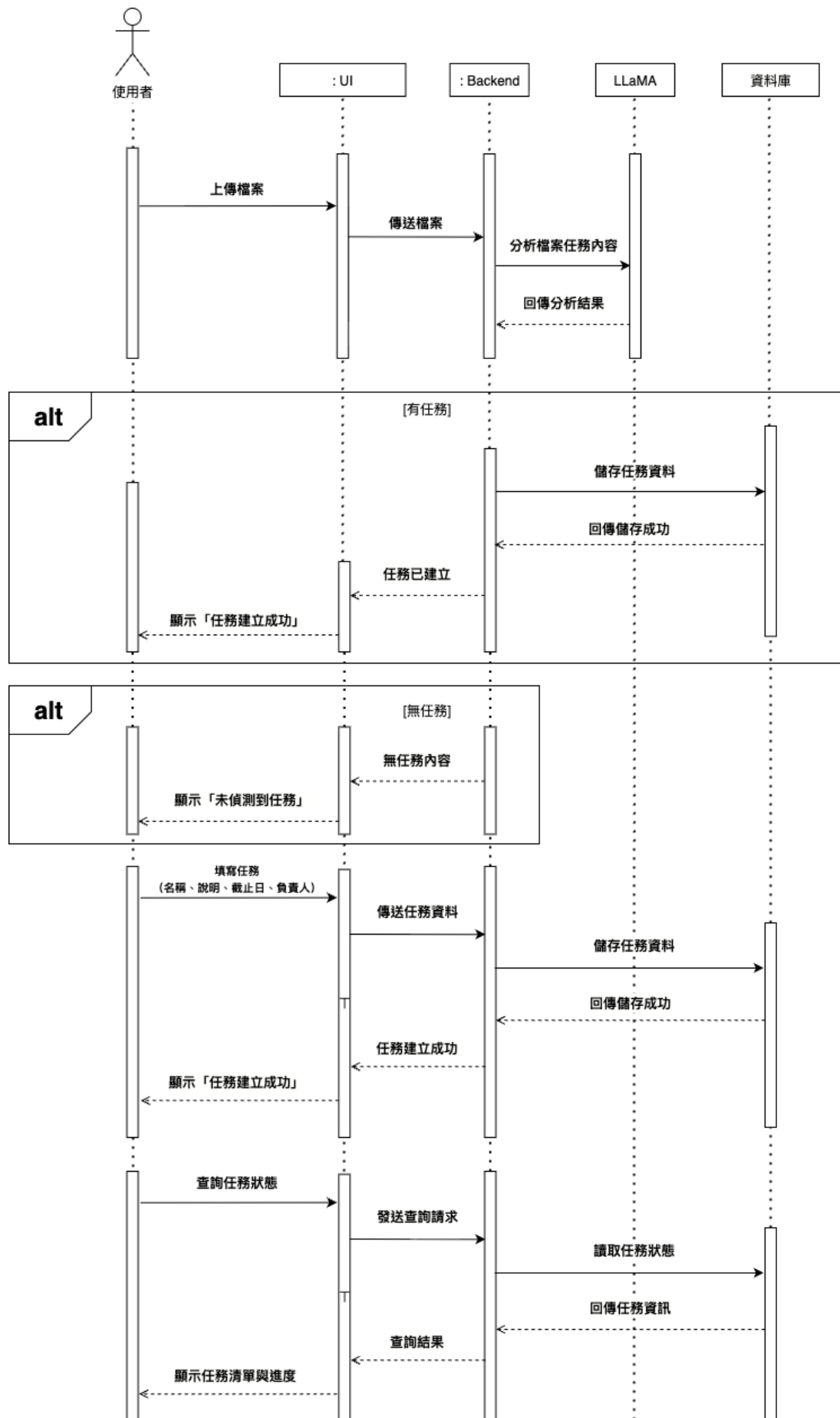
▲圖 6-1-12 會議回顧之循序圖



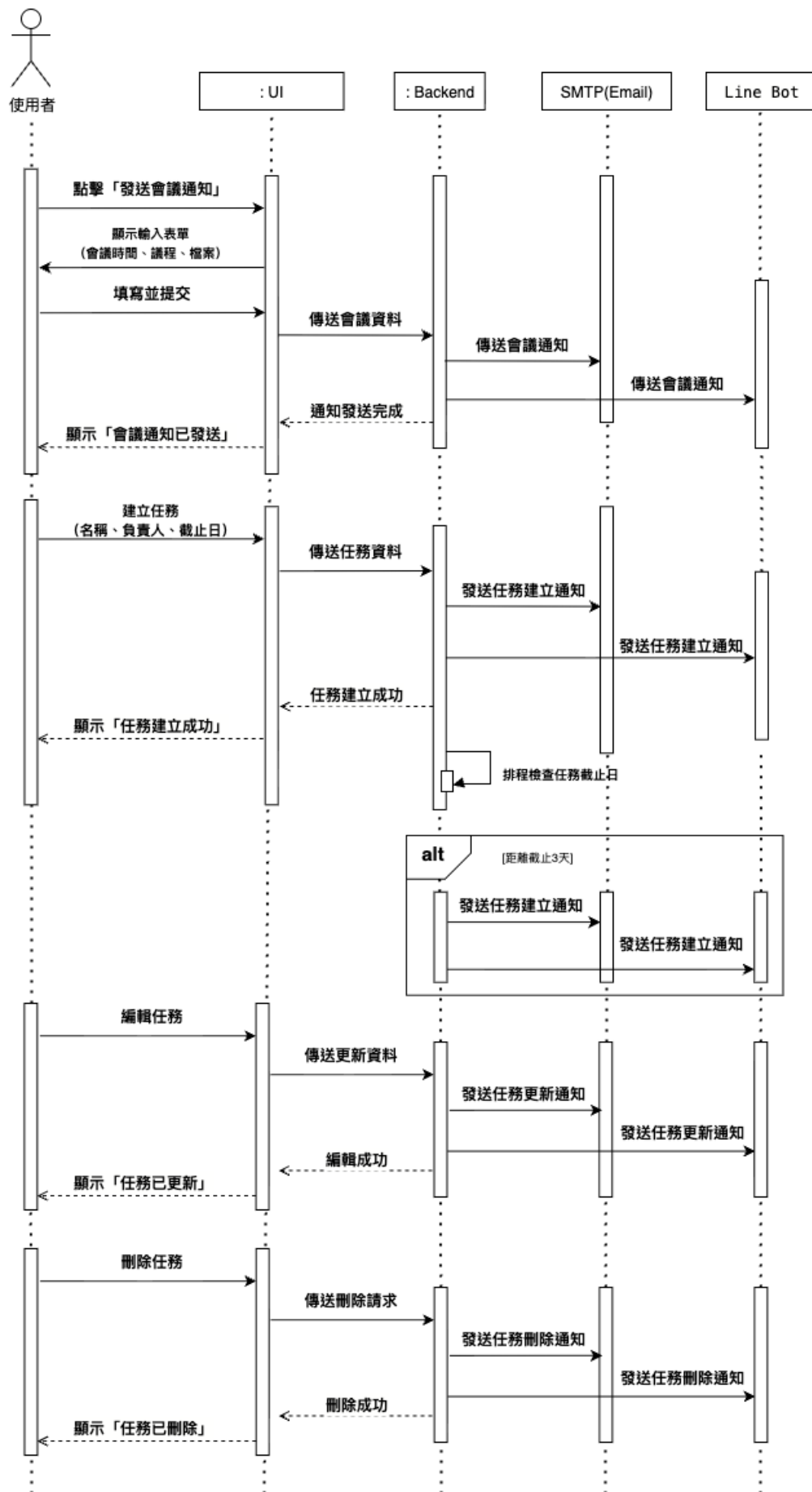
▲圖 6-1-13 歷史查詢之循序圖



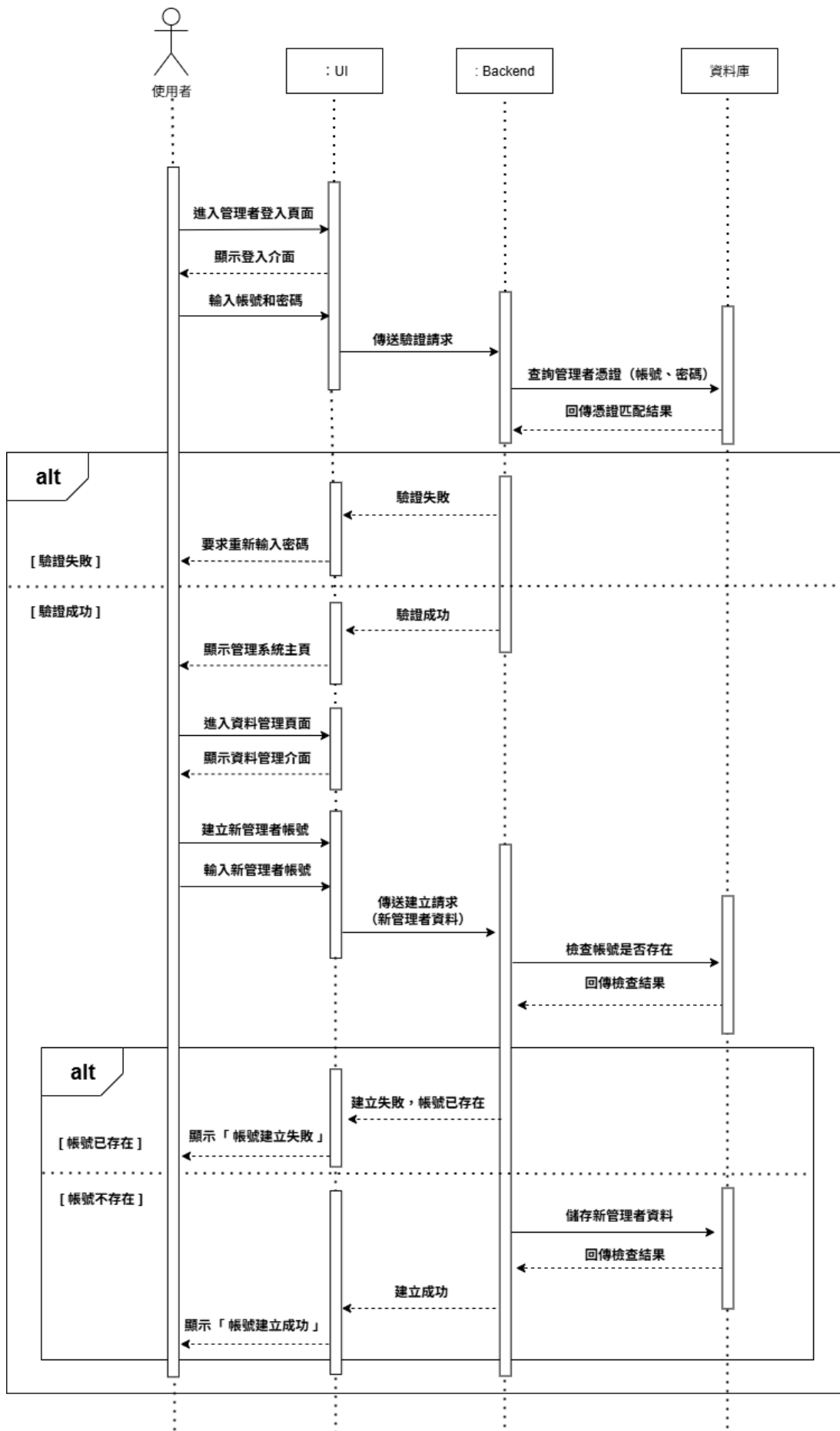
▲圖 6-1-14 正式文件之循序圖



▲圖 6-1-15 任務追蹤之循序圖

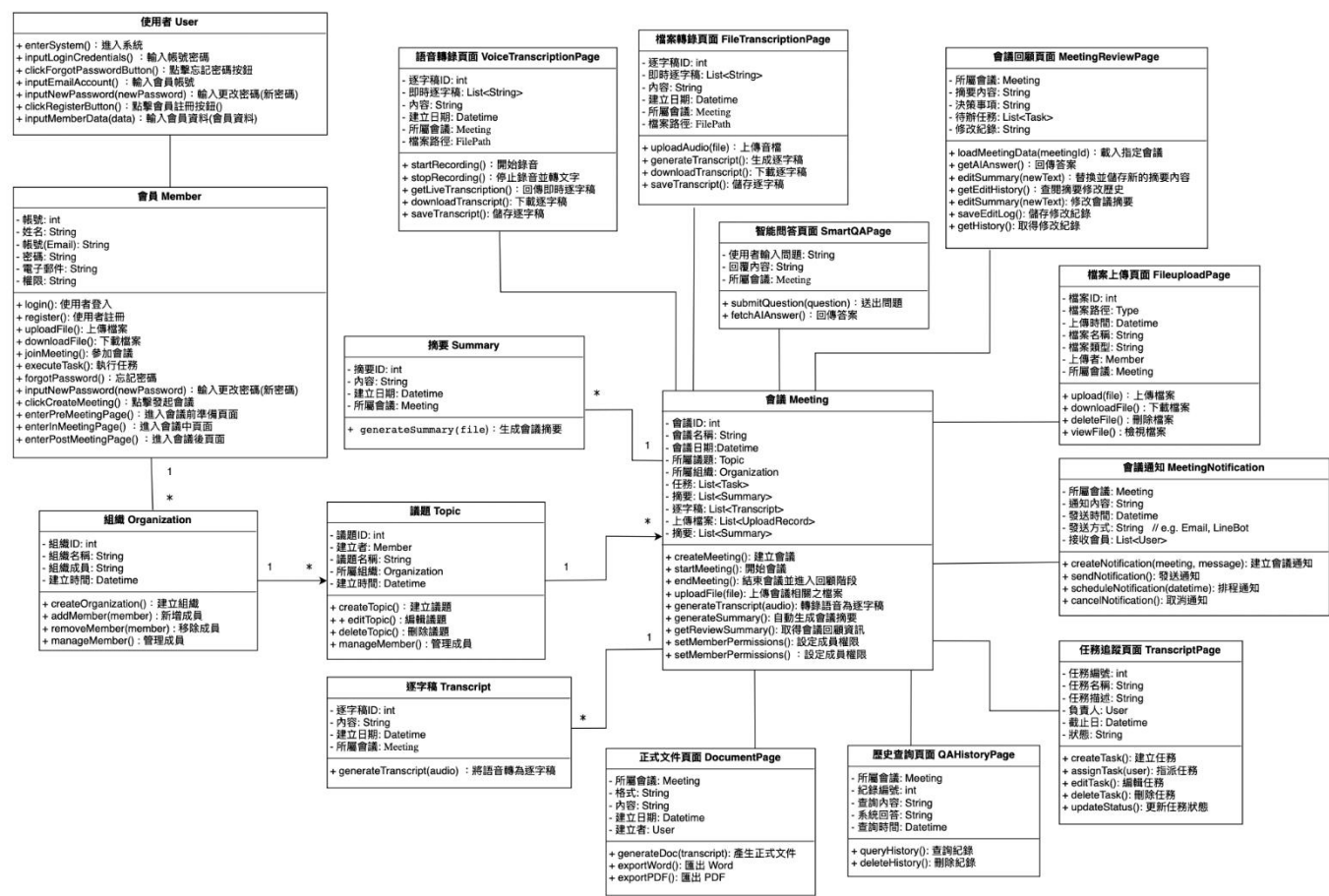


▲圖 6-1-16 通知機制之循序圖



▲圖 6-1-17 管理者介面之循序圖

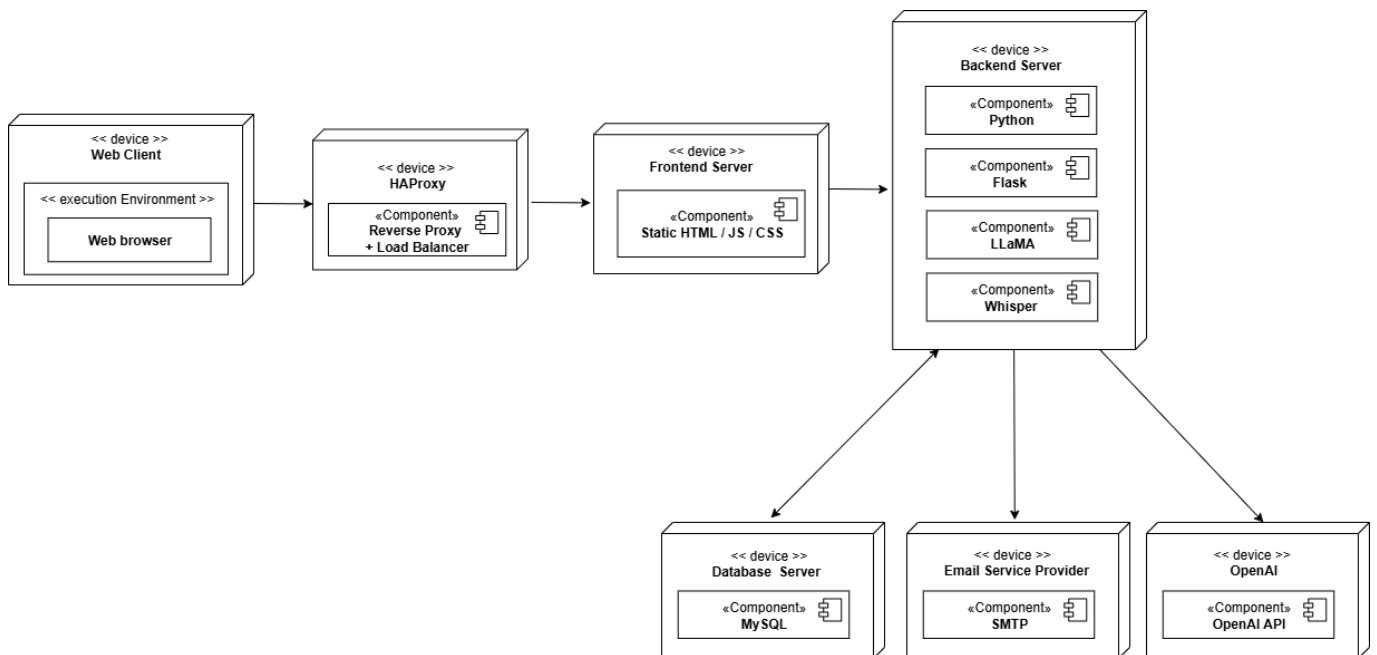
6-2 類別設計圖



▲圖 6-2-1 設計類別圖

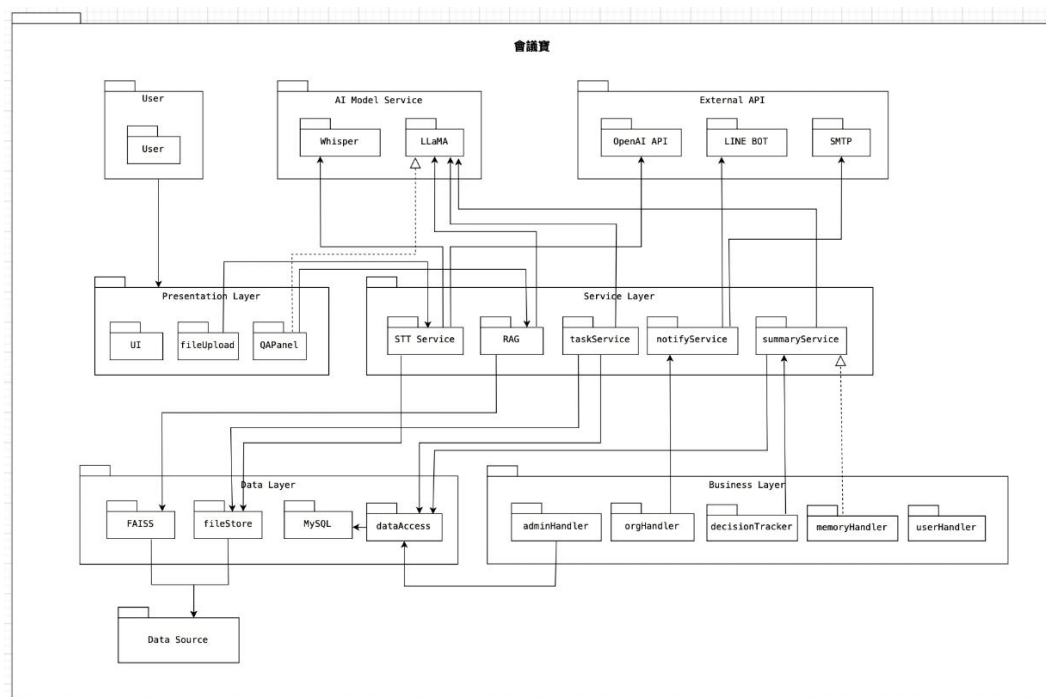
第7章 實作模型

7-1 佈署圖(DeploymentDiagram)



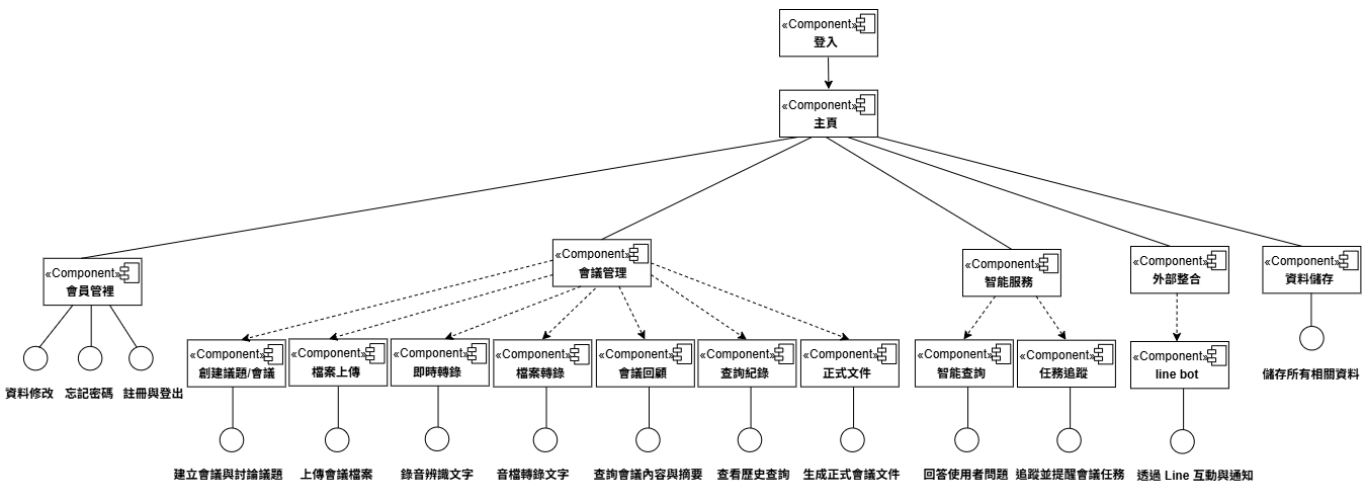
▲圖 7-1-1 佈署圖

7-2 套件圖(PackageDiagram)



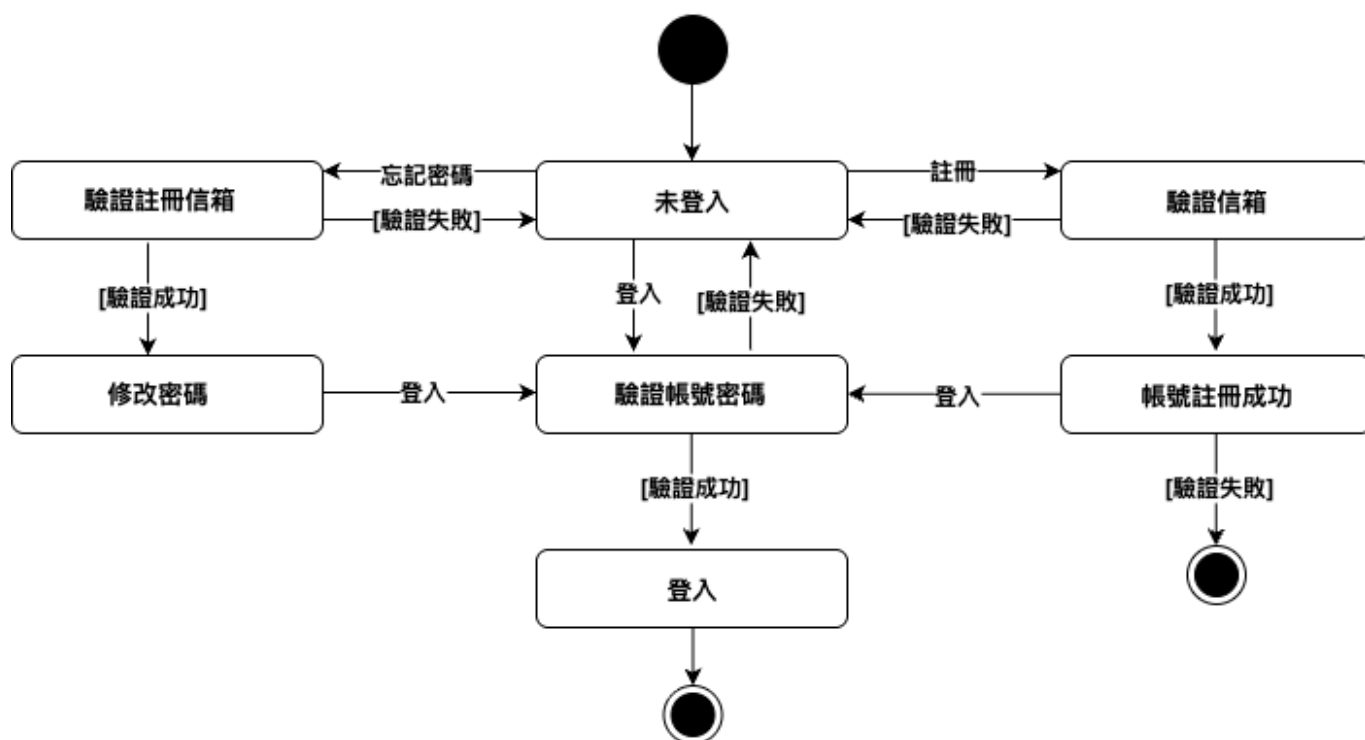
▲圖 7-2-1 套件圖

7-3 元件圖(ComponentDiagram)

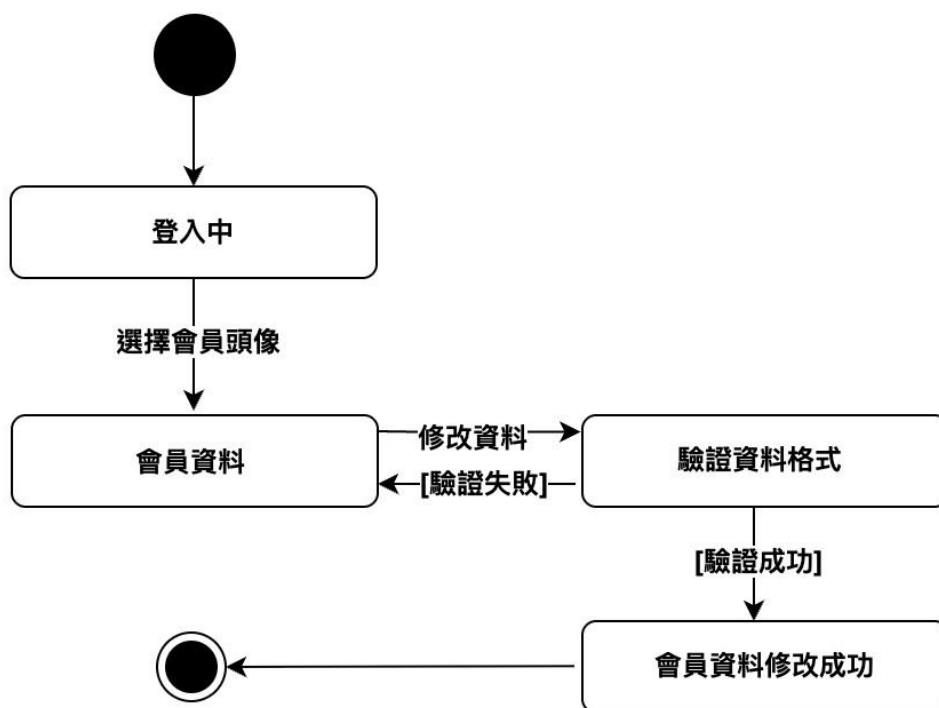


▲圖 7-3-1 元件圖

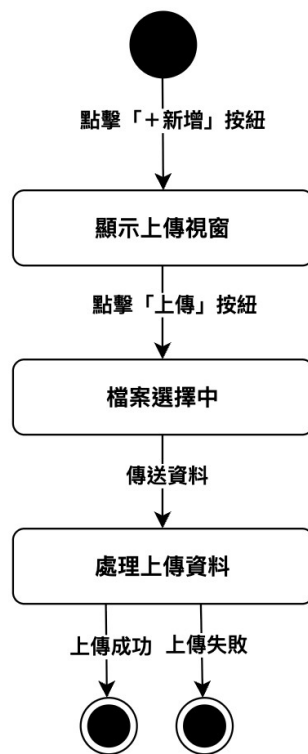
7-4 狀態機(StateMachine)



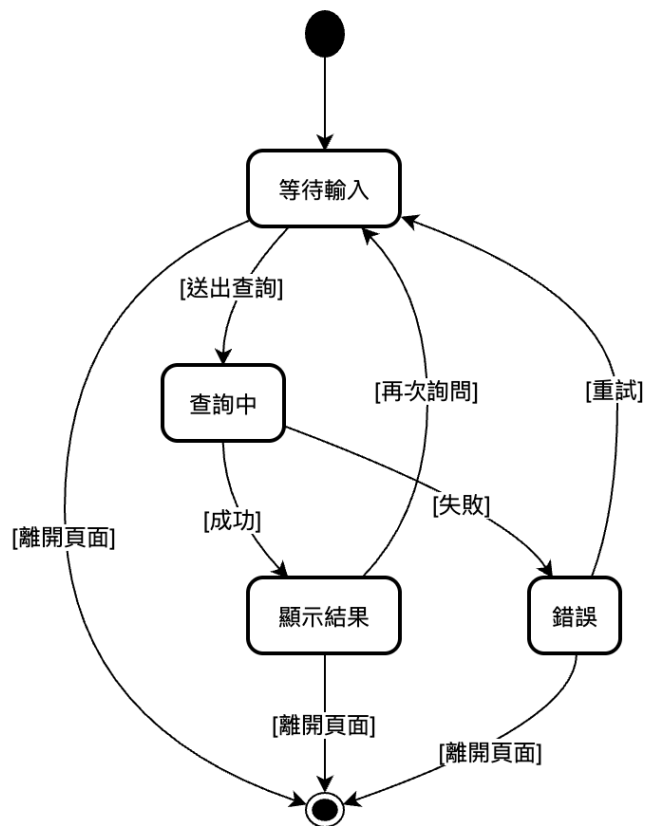
▲圖 7-4-1 會員登入之狀態機



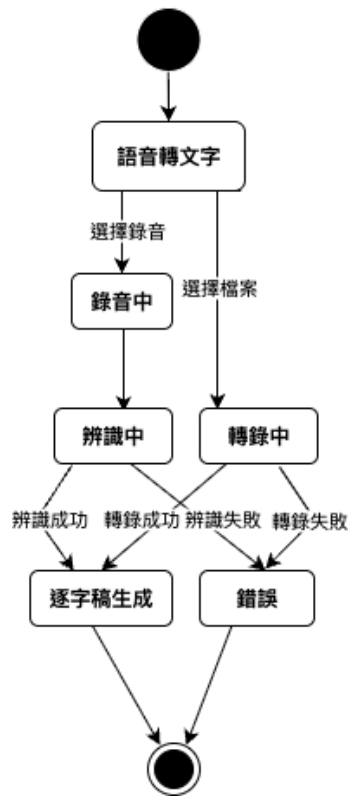
▲圖 7-4-2 會員資料修改登入之狀態機



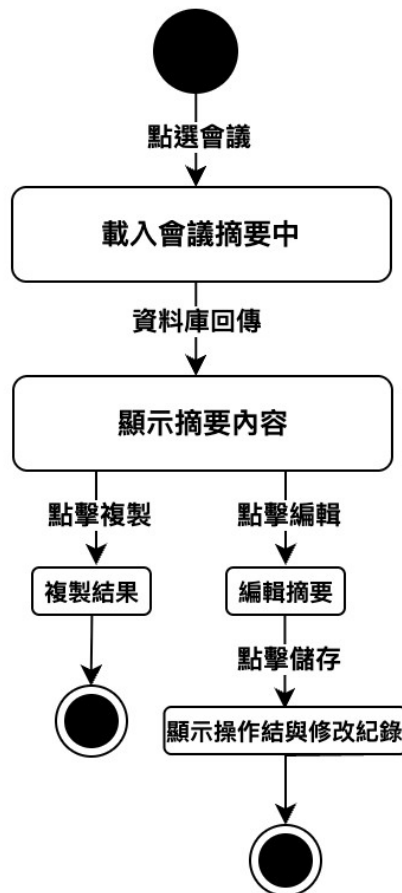
▲圖 7-4-3 檔案上傳之狀態機



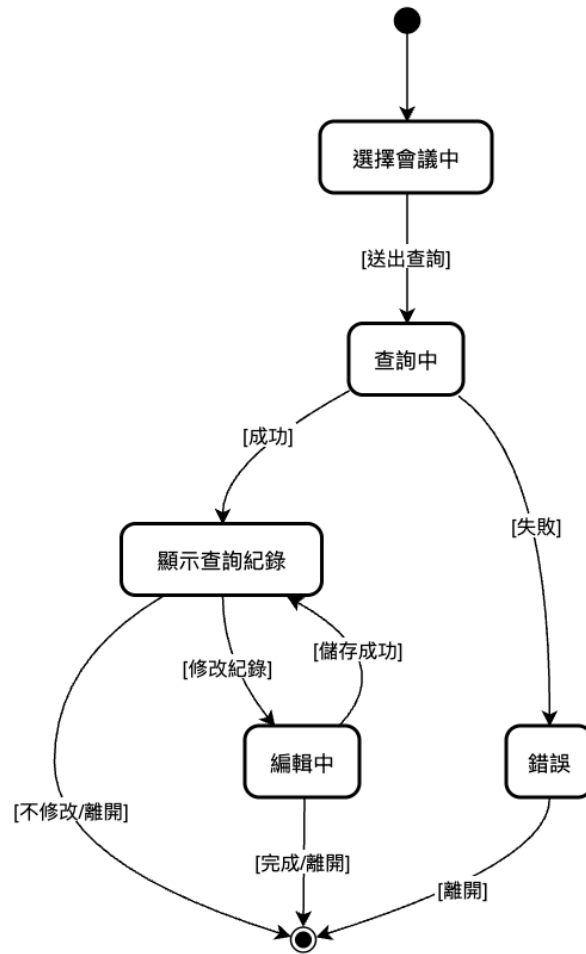
▲圖 7-4-4 智能查詢之狀態機



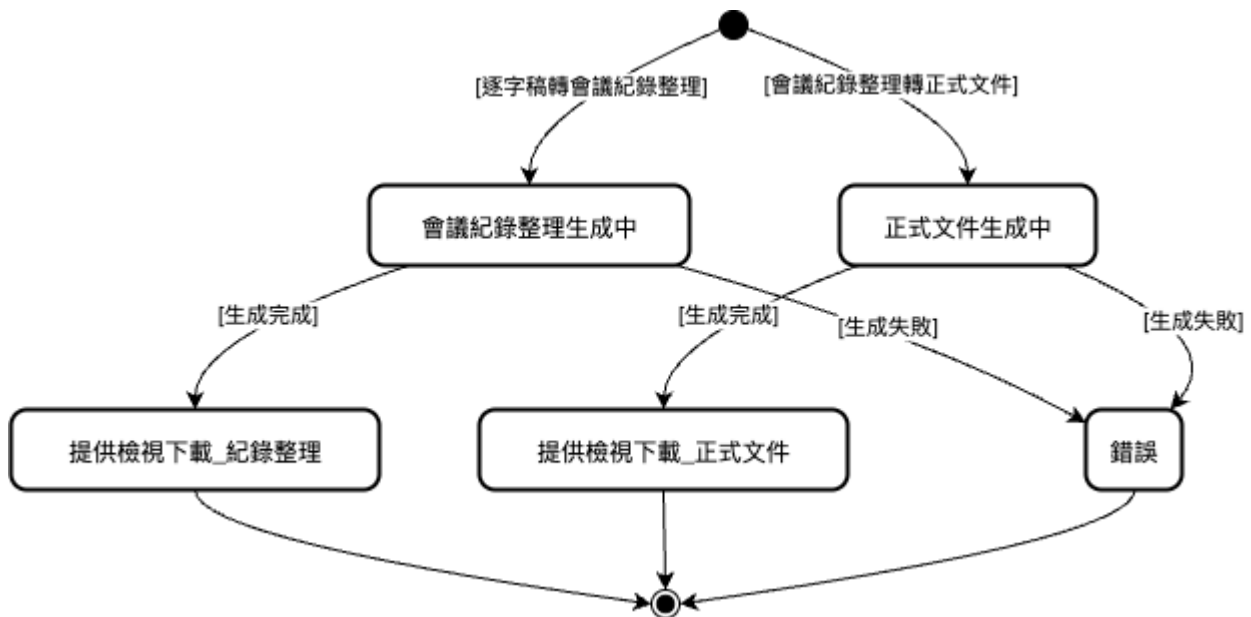
▲圖 7-4-5 語音轉文字之狀態機



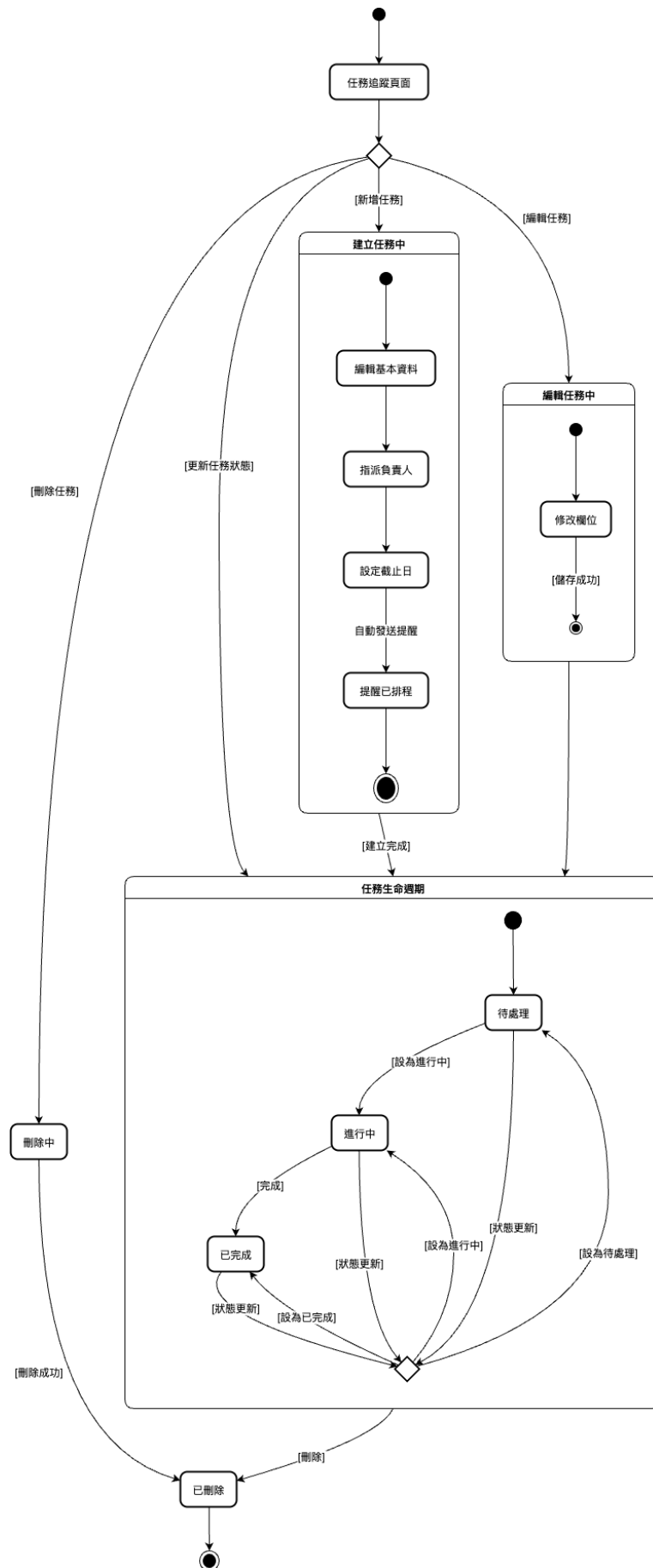
▲圖 7-4-6 會議回顧之狀態機



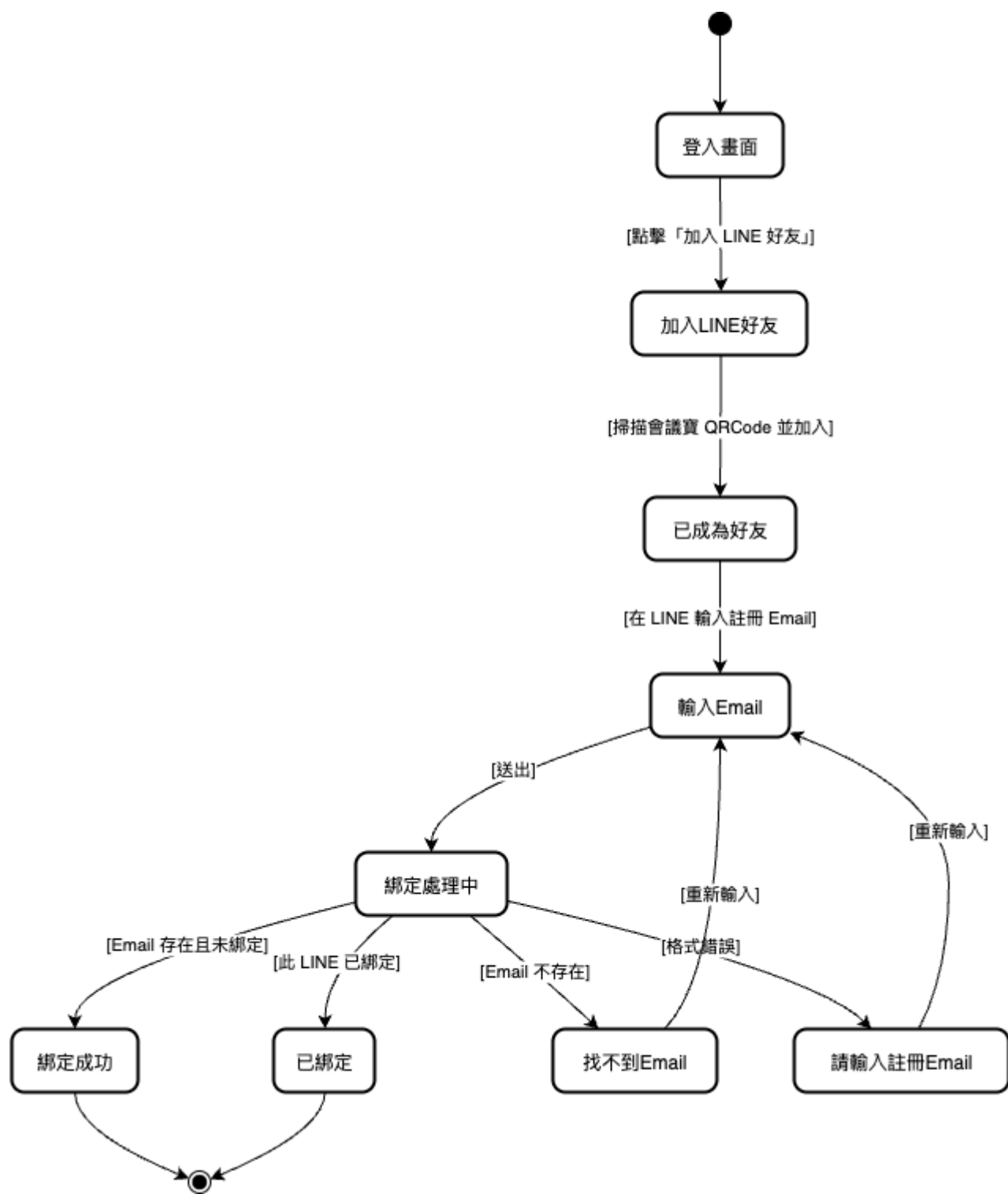
▲圖 7-4-7 查詢紀錄之狀態機



▲圖 7-4-8 正式文件之狀態機



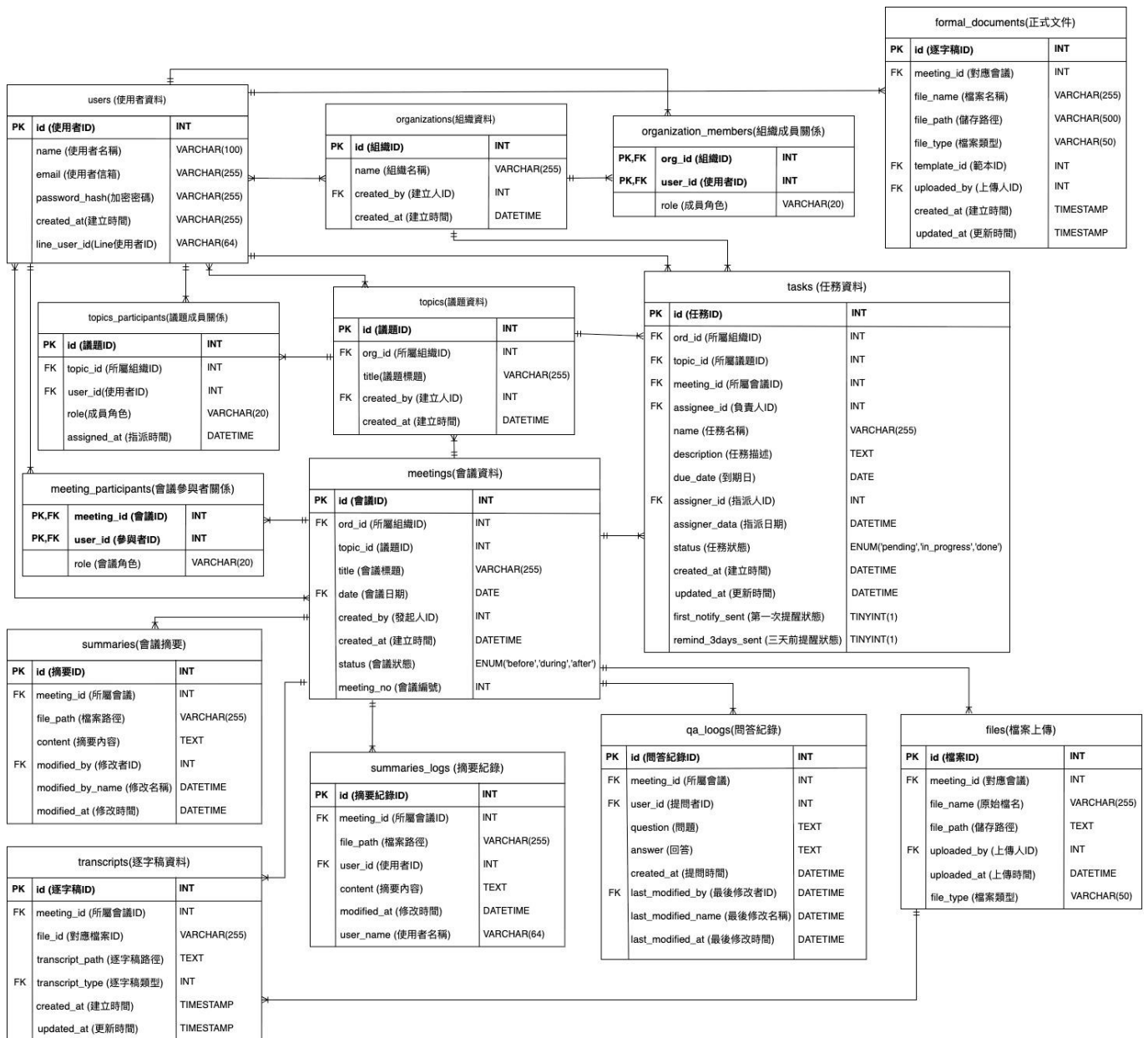
▲圖 7-4-9 任務追蹤之狀態機



▲圖 7-4-10 Line Bot 之狀態機

第8章 資料庫設計

8-1 資料庫關聯表



▲圖 8-1-1 資料庫關聯表

8-2 表格及其 MetaData

▼表 8-2-1 使用者資料 user 資料表

英文名稱	users	資料表編號	01		
中文名稱	使用者資料	主索引	id		
資料檔陳述	儲存系統使用者的註冊資訊與登入身份				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	使用者 ID	INT	v	v	
name	使用者名稱	VARCHAR(100)		v	
email	使用者信箱	VARCHAR(255)	v	v	
password_hash	加密密碼	VARCHAR(255)		v	
created_at	建立時間	DATETIME			
line_user_id	Line 使用者 ID	VARCHAR(64)	v		

▼表 8-2-2 組織資料 organizations 資料表

英文名稱	organizations	資料表編號	02		
中文名稱	組織資料	主索引	id		
資料檔陳述	紀錄每個使用者所創立或所屬的組織單位資料				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	組織 ID	INT	v	v	
name	組織名稱	VARCHAR(255)		v	
created_by	建立人 ID	INT		v	v
created_at	建立時間	DATETIME			

▼表 8-2-3 組織成員關係 organization_members 資料表

英文名稱	organization_members	資料表編號	03		
中文名稱	組織成員關係	主索引	org_id、user_id		
資料檔陳述	建立使用者與組織之間的成員隸屬關係與角色身份				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
org_id	組織 ID	INT		v	v
user_id	使用者 ID	INT		v	v
role	成員角色	VARCHAR(20)			

▼表 8-2-4 議題資料 topics 資料表

英文名稱	topics	資料表編號	04		
中文名稱	議題資料	主索引	Id		
資料檔陳述	儲存組織內的議題資訊				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	議題 ID	INT	V	V	
org_id	所屬組織 ID	INT		V	V
title	議題標題	VARCHAR(255)		V	
creator_id	建立者__ID	INT		V	V
created_at	建立時間	DATETIME			

▼表 8-2-5 議題成員關係 topic_participants 資料表

英文名稱	topic_participants	資料表編號	05		
中文名稱	議題成員關係	主索引	id		
資料檔陳述	紀錄議題與使用者之間的參與關係				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	議題 ID	INT	v	v	
topic_id	參與者 ID	INT		v	v
user_id	會議角色	INT		v	v
role	成員角色	VARCHAR(20)		v	
assigned_at	指派時間	DATETIME			

▼表 8-2-6 會議資料 meetings 資料表

英文名稱	meetings		資料表編號	06	
中文名稱	會議資料		主索引	id	
資料檔陳述	儲存每一場由使用者發起的會議基本資料與時間設定				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	會議 ID	INT	V	V	
org_id	所屬組織	INT		V	V
title	會議標題	VARCHAR(255)			
date	會議日期	DATE		V	
created_by	發起人 ID	INT		V	V
created_at	建立時間	DATETIME			
Status	會議狀態	ENUM('before','during','after')			
Meeting_no	會議編號	INT		V	

▼表 8-2-7 會議參與者關係 topic_participants 資料表

英文名稱		topic_participants	資料表編號	07	
中文名稱		會議參與者關係	主索引	meeting_id,user_id	
資料檔陳述		紀錄會議與使用者之間的參與關係			
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
meeting_id	會議 ID	INT	V	V	V
user_id	使用者 ID	INT	V	V	V
role	參與者角色	VARCHAR(20)			

▼表 8-2-8 檔案上傳 files 資料表

英文名稱	files	資料表編號	08		
中文名稱	檔案上傳	主索引	id		
資料檔陳述	管理會議中上傳的檔案資料與對應的會議與上傳者				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	檔案 ID	INT	v	v	
meeting_id	對應會議	INT		v	v
file_name	原始檔名	VARCHAR(255)		v	
file_path	檔案路徑	TEXT		v	
uploaded_by	上傳人 ID	INT		v	v
uploaded_at	上傳時間	DATETIME			
file_type	檔案類型	VARCHAR(50)		v	

▼表 8-2-9 會議摘要 summaries 資料表

英文名稱	summaries	資料表編號	09		
中文名稱	會議摘要	主索引	id		
資料檔陳述	儲存由 AI 或使用者撰寫的會議內容簡要摘要				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	摘要 ID	INT	v	v	
meeting_id	所屬會議	INT		v	v
file_path	檔案路徑	VARCHAR(255)		v	
content	摘要內容	TEXT			
modified_by	修改者 ID	INT			v
modified_by_name	修改者名稱	VARCHAR(64)			
modified_at	修改時間	DATETIME			

▼表 8-2-10 摘要紀錄 summaries_logs 資料表

英文名稱	summaries_logs	資料表編號	10		
中文名稱	摘要紀錄	主索引	id		
資料檔陳述	紀錄會議摘要的歷程				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	摘要紀錄 ID	INT	v	v	
meeting_id	所屬會議 ID	INT		v	v
file_path	檔案路徑	VARCHAR(255)		v	
user_id	使用者 ID	INT		v	v
content	摘要內容	TEXT			
modified_at	修改時間	DATETIME			
user_name	使用者名稱	VARCHAR(64)			

▼表 8-2-11 問答紀錄 qa_logs 資料表

英文名稱	qa_logs	資料表編號	11		
中文名稱	問答紀錄	主索引	id		
資料檔陳述	記錄會議期間使用者與 AI 的問答內容				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	問答紀錄 ID	INT	v	v	
meeting_id	所屬會議	INT		v	v
user_id	提問者 ID	INT		v	v
question	問題	TEXT			
answer	回答	TEXT			
created_at	提問時間	DATETIME			
last_modified_by	最後修改者 ID	INT			v
last_modified_name	最後修改者	VARCHAR(100)			
last_modified_at	最後修改時間	DATETIME			

▼表 8-2-12 任務資料 tasks 資料表

英文名稱	tasks	資料表編號			12
中文名稱	任務資料	主索引			id
資料檔陳述	儲存與會議、議題或組織相關的任務資訊				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
id	任務 ID	INT	v	v	
org_id	所屬組織 ID	INT		v	v
topic_id	所屬議題 ID	INT			v
meeting_id	所屬會議 ID	INT			v
assignee_id	負責人 ID	INT		v	v
name	任務名稱	VARCHAR(255)		v	
description	任務描述	TEXT			
due_date	到期日	DATE			
assigner_id	指派人 ID	INT			v
assign_date	指派日期	DATETIME			
status	任務狀態	ENUM('pending', 'in_progress','done')			
created_at	建立時間	DATETIME			
updated_at	更新時間	DATETIME			
first_notify_sent	第一次提醒狀態	TINYINT(1)		v	
remind_3days sent	三天前提醒狀態	TINYINT(1)		v	

▼表 8-2-13 逐字稿資料 transcripts 資料表

英文名稱	transcripts	資料表編號	13		
中文名稱	逐字稿資料	主索引	id		
資料檔陳述	儲存會議逐字稿的相關資訊				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
Id	逐字稿 ID	INT	v	v	
meeting_id	所屬會議 ID	INT		v	v
file_id	對應檔案 ID	INT			v
transcript_path	逐字稿檔案路徑	VARCHAR(500)		v	
transcript_type	逐字稿類型	VARCHAR(50)			
created_at	建立時間	TIMESTAMP			
updated_at	更新時間	TIMESTAMP			

▼表 8-2-14 正式文件 formal_documents 資料表

英文名稱	formal_documents	資料表編號	14		
中文名稱	正式文件	主索引	id		
資料檔陳述	存放會議相關的正式文件資料				
欄位名稱	意義	資料型態	唯一性	不為空值	外鍵
Id	逐字稿 ID	INT	v	v	
meeting_id	所屬會議 ID	INT		v	v
file_name	檔案名稱	VARCHAR(255)		v	
file_path	檔案路徑	VARCHAR(500)		v	
file_type	檔案類型	VARCHAR(50)			
template_id	範本 ID	INT			v
uploaded_by	上傳者 ID	INT			v
created_at	建立時間	TIMESTAMP			
updated_at	更新時間	TIMESTAMP			

第9章 程式

9-1 元件清單及其規格描述

▼表 9-1-1 元件清單與規格描述表(前端)-HTML 介面編寫

HTML 檔案(前端)-網頁介面編寫		
編號	檔案名稱	功能
1-1-1	member_sign_up.html member_sign_in.html	會員註冊與登入頁面
1-1-2	forgot_password.html	忘記密碼頁面，寄送驗證信
1-1-3	member_profile.html	會員資訊頁面，可查看與修改會員資料
1-1-4	build_organization.html	建立組織頁面，含邀請、編輯與刪除
1-1-5	organization_member.html	組織成員頁面，可新增與移除
1-1-6	topics.html	創建議題頁面，新增議題與管理成員
1-1-7	topics_member.html	議題成員頁面，管理成員新增與移除
1-1-8	start_meeting.html	創建會議頁面，新增會議並設定權限
1-1-9	meeting_member.html meeting_member2.html meeting_member3.html	會議前/中/後成員頁面，可新增、刪除、修改權限
1-1-10	meeting_before_file_upload.html meeting_during_file_upload.html meeting_after_file_upload.html	會議前/中/後檔案上傳，支援分類、刪除、智能查詢
1-1-11	meeting_during_transcript_now.html	會議中即時語音轉錄頁面，支援即時錄音轉文字，提供編輯、說話者標註，並可下載逐字稿或上傳至檔案區

1-1-12	transcript_file.html	會議後檔案轉錄頁面，上傳音檔/影片，自動轉為逐字稿，支援說話者標註、下載與儲存至檔案區
1-1-13	meeting_qa_history.html	會議後查詢紀錄頁面，查詢並檢視所有會議成員的歷史提問與解答，可進行搜尋、編輯與刪除
1-1-14	meeting_review.html	會議後會議回顧頁面，可透過 AI 產生會議摘要，並提供檢視、編輯及追蹤修改紀錄功能
1-1-15	meeting_formal_doc.html	會議後正式文件頁面，將逐字稿整理為會議紀錄，並可套用模板產出正式文件
1-1-16	task.html	會議後任務追蹤頁面，可新增、指派、編輯與管理會議任務

▼表 9-1-2 元件清單與規格描述表(後端)-語音處理

Python 檔案(後端)-語音處理		
編號	檔案名稱	功能
2-1-1	blueprints/gpt_punctuate.py	GPT 自動補標點與分段修飾
2-1-2	blueprints/realtime_stt_ws_old.py	Whisper 即時語音轉文字辨識
2-1-3	blueprints/server.py	WebSocket 即時語音轉譯服務
2-1-4	blueprints/speech_transcribe.py	音訊上傳轉譯與字幕產生儲存
2-1-5	blueprints/whisper_core.py	WhisperCLI 模型封裝與降級
2-1-6	blueprints/whisper_handler.py	上傳音訊即時轉文字加標點
2-1-7	blueprints/whisper_stream.py	音訊片段轉譯並檢查清理輸出

▼表 9-1-3 元件清單與規格描述表(後端)-文字處理

Python 檔案(後端)-文字處理		
編號	檔案名稱	功能
2-2-1	embedding_utils.py	抽取文件文字並生成向量嵌入
2-2-2	utils/text_extract.py	從文件抽取純文字並清理格式
2-2-3	blueprints/openai_tools/srt_utils.py	字幕解析與 SRT 格式轉換工具
2-2-4	blueprints/cleaner.py	清理逐字稿雜訊產生乾淨文字
2-2-5	blueprints/punctuate.py	中文逐字稿自動加標點符號
2-2-6	blueprints/punctuation_utils.py	基本標點補齊與自動換行處理

▼表 9-1-4 元件清單與規格描述表(後端)-組織/議題/會議管理

Python 檔案(後端)-組織/議題/會議管理		
編號	檔案名稱	功能
2-3-1	blueprints/services/minutes_fs.py	處理會議逐字稿與紀錄存取
2-3-2	blueprints/todos_bp.py	抽取行動項目並建立任務
2-3-3	blueprints/file.py	管理會議檔案與權限控制
2-3-4	blueprints/meeting.py	會議建立刪除與流程管理
2-3-5	blueprints/todos_api.py	提供會議紀錄摘要與下載
2-3-6	blueprints/organization.py	組織建立刪除與成員管理
2-3-7	blueprints/permission.py	會議階段權限與成員管理
2-3-8	blueprints/qa.py	會議逐字稿問答與摘要管理
2-3-9	blueprints/task.py	任務建立查詢與通知寄送
2-3-10	blueprints/topic.py	議題建立刪除與成員管理

▼表 9-1-5 元件清單與規格描述表(後端)-AI 與語意分析

Python 檔案(後端)-AI 與語意分析		
編號	檔案名稱	功能
2-4-1	gpt_punctuate.py	GPT 自動補標點與修飾
2-4-2	_openai.py	串接 llama.cpp 即時回應
2-4-3	utils/todo_extractor.py	抽取逐字稿行動項目清理
2-4-4	utils/rag_pipeline.py	建立或載入向量索引流程
2-4-5	utils/rag_utils.py	文字分塊嵌入 FAISS 索引
2-4-6	blueprints/openai_tools/flask_blueprint.py	標點 API 與健康檢查
2-4-7	blueprints/openai_tools/gpt_punctuate.py	GPT 中文文本補標點處理
2-4-8	blueprints/openai_tools/openai_punctuator.py	批次補標點保留行數順序
2-4-9	blueprints/openai_tools/ openai_srt_punctuator.py	字幕逐行補標點保留時間碼
2-4-10	blueprints/openai_tools/punctuate_srt.py	命令列字幕補標點工具
2-4-11	blueprints/openai_tools/punctuate.py	規則式與後備標點工具
2-4-12	blueprints/services/llm_client.py	llama.cpp 模型封裝與快取
2-4-13	blueprints/services/punctuation_utils.py	基礎標點補全自動加句號
2-4-14	blueprints/formal_doc.py	LLM 生成正式會議文件
2-4-15	blueprints/llm_constants.py	定義 LLM 常數與模板 schema
2-4-16	blueprints/llm_utils.py	LLM 初始化與逐字稿轉 JSON
2-4-17	blueprints/semantic_classifier.py	LLM 逐字稿分類成會議紀錄

▼表 9-1-6 元件清單與規格描述表(後端)-系統服務

Python 檔案(後端)-系統服務		
編號	檔案名稱	功能
2-5-1	a.py	檢視 pickle 檔案內容結構
2-5-2	app.py	Flask 主入口整合模組路由
2-5-3	db.py	MySQL 存取與資料管理
2-5-4	email_utils.py	郵件通知與邀請信功能
2-5-5	blueprints/auth.py	會員註冊登入與身分管理
2-5-6	blueprints/email.py	郵件 API 與通知重設流程
2-5-7	blueprints/realtime_stt_ws.py	WebSocket 代理 OpenAI 即時連線
2-5-8	blueprints/linebot.py	透過 LINEBot 發送會議通知與任務提醒

9-2 其他附屬之各種元件

▼表 9-2-1 虛擬碼-meeting_during_transcript_now.html

編號	1-1-11	程式名稱	meeting_during_transcript_now.html
目的	提供使用者在「會議中」進行即時錄音與轉錄，包含音訊擷取、分段上傳、即時文字顯示、逐字稿下載與上傳		
虛擬碼			
1. 初始化與設置 ● 初始化屬性： ➤ MEETING_ID：目前會議 ID（由 Jinja 傳入） ➤ FILES_URL：檔案上傳頁面網址 ➤ API 端點清單：/whisper_stream、/api/whisper_stream ➤ 狀態參數：錄音中斷點、時間游標 timelineSec、完整逐字稿字串 fullTranscript ➤ 閾值設定：最小音訊長度、最小檔案大小、靜音判斷分貝數 ● UI 元件： ➤ sidebar：側邊欄（顯示會議資訊與功能選單） ➤ liveText：即時轉錄顯示區 ➤ hintsBox：專有名詞詞庫輸入框 ➤ 控制按鈕：開始、暫停、停止、下載逐字稿、上傳逐字稿 ➤ 選項：切片秒數下拉選單、編輯模式勾選、自動捲動勾選 2. 函數與功能 ● 側邊欄與導覽 ➤ toggleSidebar：展開或收合側邊欄 ➤ goToNext：導向會議後頁面（審查） ➤ openHelpModal：顯示錄音與切片流程說明 ● 錄音與即時轉錄流程 ➤ start：啟用麥克風，開始錄音，定時匯出音訊切片並送至 API ➤ pauseOrResume：暫停或恢復錄音 ➤ stopAll：停止所有錄音流程，釋放資源並允許下載/上傳逐字稿			

- cleanupRecorders：清理錄音器與音訊串流

- API 上傳與文字處理

- resolveAPI：測試可用的 API 端點
- postChunk：將音訊切片 POST 至 API，處理錯誤（404 換端點、413 縮短切片、重試等）
- analyzeBlobWav：分析音訊片段（長度與音量），過短或過小會忽略
- appendSegment：新增文字片段至即時轉錄區（支援合併或跳過重複片段）
- mergeOrAppend：判斷新片段是否覆蓋、跳過或新增
- filterClientSide：過濾掉黑名單提示詞
- createSeg：建立單一逐字稿片段（含時間範圍、說話者下拉、可編輯文字）

- 逐字稿輸出與上傳

- buildExportText：組合完整逐字稿，包含編號、時間範圍、說話者、文字
- downloadTxt：將逐字稿下載為 TXT 檔案
- uploadTranscript：上傳逐字稿至檔案區（POST 至/api/files/transcript）

- 互動功能

- 捲動 liveText：若非底部，顯示「跳到最新」按鈕並關閉自動捲動
- jumpLatestBtn：按下後捲動至最底並開啟自動捲動
- editModeChk：切換逐字稿片段是否可編輯
- 控制按鈕事件綁定：
 - startBtn→start()
 - pauseBtn→pauseOrResume()
 - stopBtn→stopAll()
 - downloadBtn→downloadTxt()
 - uploadBtn→uploadTranscript()

3. 頁面佈局與建構

- 側邊欄(Sidebar)：

- 顯示目前會議名稱與日期

- 功能連結：檔案上傳、語音轉文字、使用說明
- 主內容(MainContent)：
 - 頁首：Logo、流程進度條（前/中/後）、頭像
 - 錄音狀態區：錄音狀態提示、切片秒數選擇、編輯模式、自動捲動、詞庫輸入
 - 即時轉錄區：顯示轉錄片段，支援時間標記與說話者選擇
 - 控制按鈕：開始、暫停、停止、下載逐字稿、上傳逐字稿

▼表 9-2-2 虛擬碼- meeting_review.html

編號	1-1-14	程式名稱	meeting_review.html
目的	提供使用者於會議後檢視、產生、編輯與管理會議摘要，並支援檔案選擇、AI 摘要生成、手動修改與歷史紀錄查詢。		
虛擬碼			
一、 初始化與設置 1. 全域變數（由後端模板注入） ○ MEETING_ID：會議識別碼 ○ TOPIC_ID：議題識別碼 ○ MEETING_TOPIC_ID：會議對應的議題 ID ○ PERMISSION：當前使用者對摘要的權限(edit/view) ○ CURRENT_USER_ROLE：目前使用者角色（主持人、成員等） 2. 樣式與套件 ○ TailwindCSS：頁面布局與樣式 ○ Lucide：SVG 圖示庫 ○ navbar_a.css：自訂導覽列樣式 3. 權限控制 ○ canEdit()：依 PERMISSION 或 CURRENT_USER_ROLE 判斷是否能修改摘要 ○ 若無權限，摘要編輯按鈕顯示灰階、點擊提示「沒有權限」 二、 函數與功能 1. 工具函數 ○ normalize(v)：將字串正規化（小寫、去空白） ○ renderSummary(obj)：將結構化 JSON(時間/出席人員/討論事項/決議/待辦事項)渲染為文字段落 ○ tryParseSummaryString(s)：嘗試將字串轉為 JSON，支援舊/新欄位			

- `extractSummaryObject(apiResp)`：從 API 回傳取出結構化摘要物件

2. 摘要操作

- 複製摘要：copy-summary 按鈕→將文字區塊內容複製到剪貼簿
- AI 生成摘要：generate-summary 按鈕→呼叫
`POST/qa/api/llama_summarize` 產生新摘要，顯示於文字區塊
- 編輯摘要：
 - edit-summary 按鈕→切換 textarea 為可編輯狀態
 - 失焦事件→呼叫 `POST/qa/api/summary/{meeting_id}` 儲存修改後內容，並記錄到修改紀錄

3. 會議與檔案載入

- 初始化會議清單：`initMeetingSelect()`
 - 若有 TOPIC_ID→載入該議題下所有會議
(`/qa/api/meetings_by_topic/{topic_id}`)
 - 若無 TOPIC_ID→僅顯示當前會議
- 選擇會議→載入檔案清單(`/api/meeting_files/{meeting_id}`)
- 選擇檔案→載入該檔案的摘要
(`/qa/api/summary/{meeting_id}?file_path=...`)與修改紀錄

4. 摘要修改紀錄

- `loadSummaryLogs(meetingId,filePath)`：呼叫
`/qa/api/summary_logs/{meeting_id}` 取得修改紀錄，渲染於歷史區塊
- `toggleLogDetail(id)`：展開或收合單筆修改紀錄

5. 會議控制

- `endMeeting(e)`：主持人可呼叫 `POST/api/meeting/{meeting_id}/end` 結束會議，並導回會議清單
- `getCurrentMeetingId()`：從「結束會議」按鈕取會議 ID

6. 使用說明 Modal

- `openHelpModal()`、`closeHelpModal()`：控制 Modal 顯示/隱藏

三、 頁面佈局與建構

1. 側邊欄 Sidebar

- 顯示目前會議名稱(meeting-name)與日期(meeting-date)
- 功能選單：檔案上傳、語音轉文字、會議回顧、查詢紀錄、正式文件、任務追蹤、使用說明
- 僅主持人會看到「結束會議」按鈕(`endMeetingBtn`)

2. 頁首 Header

- 左：Logo (返回組織頁面)
- 中：會議進度條 (前/中/後)
- 右：使用者頭像 (連結到成員名單頁)

3. 主要內容區 MainContent

- 摘要區塊
 - 會議下拉(meetingTitle)：載入議題下所有會議
 - 檔案下拉(fileSelect)：顯示所選會議的檔案清單
 - 生成摘要按鈕(generate-summary)：觸發 AI 摘要
 - 摘要文字框：顯示/編輯摘要
 - 操作按鈕：複製(copy-summary)、編輯(edit-summary)
- 修改紀錄區塊
 - history-log：顯示所有修改紀錄，可展開查看詳細內容

4. 使用說明 Modal

- 說明操作方式：
 - 選擇會議與檔案
 - AI 生成摘要
 - 摘要操作（複製、編輯、自動儲存）
 - 修改紀錄（可展開查看詳細內容）
 - 結束會議（主持人限定）

▼表 9-2-3 虛擬碼- task.html

編號	1-1-16	程式名稱	task.html
目的	供使用者在「會議後」進行任務追蹤，包括建立、檢視、編輯、刪除任務，以及由受指派者更新任務狀態		
虛擬碼			
1. 任務管理 ● 載入任務 ➤ loadTasks： ◦ 呼叫/api/tasks?org_id=...&meeting_id=...&topic_id=... ◦ 成功→呼叫 renderTaskRows 渲染表格 ◦ 失敗→顯示錯誤提示 ➤ renderTaskRows(list)： ◦ 逐列渲染任務內容（名稱、說明、負責人、截止日、狀態、指派者、指派日期、操作） ◦ 狀態欄： ■ 若為受指派者→顯示下拉選單可修改狀態 ■ 其他人→顯示 badge			

- 操作欄：
 - 指派者→可使用「編輯」「刪除」
 - 非指派者→按鈕灰化、不可操作
- 新增/編輯任務
 - openTaskModal(task)：
 - task=null→新增模式
 - task!=null→編輯模式（會帶入任務資料）
 - 欄位權限：
 - 指派者→可改全部
 - 受指派者→僅能改狀態
 - 其他人→無權限
 - closeTaskModal：關閉視窗
 - taskFormsubmitHandler：
 - 新增（POST/api/task）
 - 驗證必填欄位（名稱、負責人、截止日）
 - 傳送 org_id、meeting_id、topic_id、name、description、due_date、assignee_id
 - 編輯（PUT/api/task/{id}）
 - 指派者→可改全部欄位
 - 受指派者→僅能更新 status
 - 成功後關閉 modal 並重新載入列表
- 任務操作
 - onEditTask(id)：
 - 從 API 重新抓任務資料，打開編輯 modal
 - onDeleteTask(id)：
 - DELETE/api/task/{id}
 - 成功→從表格移除該列（若空→顯示「目前沒有任務」）
 - onStatusChange(id,select)：
 - 受指派者可直接在表格下拉改狀態，PUT/api/task/{id}
 - 成功→更新狀態；失敗→還原原本值

2. 其他功能

- loadAssigneesForMeeting(meetingId)：呼叫/api/participants?meeting_id=...載入會議成員，填充「負責人」下拉選單
- escapeHTML：輸出表格內容時避免 XSS
- ensureEndButtonVisibility：檢查會議狀態，決定是否顯示「結束會議」按

鈕

3. 頁面佈局與建構

- 側邊欄(Sidebar)：顯示目前會議日期與名稱，含多個導覽項目，會依會議狀態決定是否顯示「結束會議」
- 主內容(MainContent)：
 - 頁首：Logo、會議流程進度條（前/中/後）、頭像（連結至成員頁面）
 - 任務追蹤區塊：
 - 「+新增任務」按鈕（開啟 modal）
 - 任務表格：含欄位（名稱、說明、負責人、截止日、狀態、指派者、指派日期、操作）
- Modal（taskModal）：新增/編輯任務視窗，包括：
 - 任務名稱（必填）
 - 任務說明
 - 截止日（必填，不得早於今天）
 - 負責人下拉選單
 - 狀態選單（僅編輯模式，且受指派者可修改）
 - 按鈕：取消、提交

▼表 9-2-4 虛擬碼- whisper_handler.py

編號	2-1-6	程式名稱	whisper_handler.py
目的	提供即時音訊上傳與轉錄服務，並自動偵測硬體選擇最佳 Whisper 推理組合。		
虛擬碼			
4. 初始化與設置 ● 匯入模組： ■ Flask (Blueprint, request, jsonify) ■ faster_whisper.WhisperModel (模型推理) ■ punctuation_utils.basic_punctuation (基本標點) ■ opencc.OpenCC (簡轉繁) ■ ctranslate2 (CUDA/CPU 裝置檢查) ■ os, time ● 路徑： ■ uploads/temp → 儲存臨時音訊檔 ● 常數： ➢ MODEL_NAME = "large-v3" ➢ MODEL, DEVICE, COMPUTE_TYPE → 全域模型狀態 5. 函數與功能 ● init_whisper_model() ◆ 功能：根據環境自動選最快可行的推理組合，避免崩潰。 ◆ 嘗試順序： ● CUDA：float16 → int8_float16 → bfloat16 → int8_bfloat16 ● CPU：int8_float32 → int8 → float32。 ◆ 流程： ● 偵測是否有 CUDA (ctranslate2.get_cuda_device_count)。 ● 逐一嘗試載入模型。 ● 成功 → 設定全域 MODEL, DEVICE, COMPUTE_TYPE。 ● 全失敗 → RuntimeError。 ● whisper_info() ◆ 路由：GET /whisper_info ◆ 功能：回傳目前使用的模型資訊。 ◆ 回傳 JSON： { "success": true,			

```
"device": "cuda",  
"compute_type": "float16",  
"model_name": "large-v3"  
}
```

- whisper_stream()

- ◆ 路由：POST /whisper_stream

- ◆ 輸入：

- 表單上傳 file（音訊檔）
- 可選 prompt（專有名詞提示）

- ◆ 流程：

1. 驗證檔案存在與檔名有效。
2. 儲存到 uploads/temp/{timestamp}_{filename}。
3. 呼叫 MODEL.transcribe()：
 - 語言：中文 (zh)
 - 任務：轉錄 (transcribe)
 - beam_size=5, best_of=5
 - 溫度：[0.0, 0.2, 0.4]
 - vad_filter=True（靜音過濾）
 - 其他安全閾值設定
4. 合併 segments 文字 → raw_text。
5. 若無文字 → 回傳空結果。
6. 有文字 → 進行：
 - 簡轉繁 (OpenCC)
 - 基本標點 (basic_punctuation)
7. 回傳 JSON：

```
{  
  "success": true,  
  "device": "cuda",  
  "compute_type": "float16",  
  "raw_text": "轉換後繁體文字",  
  "polished_text": "帶標點的文字"  
}
```

▼表 9-2-5 虛擬碼- openai_srt_punctuator.py

編號	2-4-9	程式名稱	openai_srt_punctuator.py
目的	使用 OpenAI API 對字幕檔（SRT 單行格式）補標點，保留時間碼與逐行對應。		
虛擬碼			
6. 初始化與設置 ● 匯入：os, time, List, dotenv.load_dotenv, OpenAI ● 匯入 split_time_and_text, join_time_and_text ● 載入 .env，預設模型 gpt-4o-mini ● 定義 _SYSTEM：標點規則（逐行補標點，不可合併/刪除行） 7. 函數與功能 ● _build_user_prompt(lines) ◆ 將字幕文字轉成 L1: ... 格式，作為 prompt。 ● _call_openai_on_lines(lines, model, max_retries=3, sleep_s=1.5) ◆ 呼叫 OpenAI API，逐行解析 Lk:。 ◆ 失敗時 fallback：按行數對齊或回傳原句。 ● punctuate_srt_lines(single_lines, batch_size=30, model=None) ◆ 輸入格式："00:...,000 --> 00:...,000 文字" ◆ 流程： A. 抽取字幕的 start/end/time B. 文字分批丟給 _call_openai_on_lines C. 將補標點結果與原時間碼合併 → join_time_and_text D. 回傳與輸入行數一致的列表 8. 錯誤處理 ● API 呼叫失敗：重試最多 3 次 ● 仍失敗：保留原字幕文字			

▼表 9-2-6 虛擬碼- todos_bp.py

編號	2-3-2	程式名稱	todos_bp.py
目的	提供 API 從會議紀錄或索引檔中抽取待辦事項，並直接建立到資料庫。		
虛擬碼			
1. 初始化與設置 - 載入模組：flask(Blueprint, request, jsonify) - 載入 DB 函式：create_task_simple - 載入工具模組： - utils.rag_pipeline(build_index_from_meeting_folder) - utils.todo_extractor(retrieve_context, compose_prompt, force_json) - 載入 OpenAI：OpenAI - 設定 LLM 模型與參數：OPENAI_MODEL, LLM_CTX, RESERVE_TOKENS, RAG_TOKEN_LIMIT - 建立 Blueprint：todos_bp 2. 工具函式 _generate_short_title(content)： - 呼叫 OpenAI 產生簡短標題（<=20 字） - 若失敗則取 content 前 20 字 _compose_direct_prompt(text)： - 建立 LLM 提示詞，要求輸出格式： {"todos": [{"content": "string"}]} _call_llm_json(prompt, max_tokens, model)： - 呼叫 OpenAI 產生輸出 - 強制 JSON 格式 - 若解析失敗則呼叫 force_json 修復 3. 主要流程 - todos_extract_internal(meeting_id, text, topic_id=None, assigner_id=None)： - 呼叫 call_llm_json 取得 todos JSON			

- 逐一建立任務：
 - 過濾空白 content
 - 呼叫 `_generate_short_title` 產生簡短名稱
 - 呼叫 `create_task_simple` 建立 task (含 `meeting_id / topic_id / uploader_id`)
- 回傳結果：`{"ok": True, "used_rag": bool, "todos": [...], "raw": ...}`

4. API Routes

- POST `/api/todos_extract`
 - 參數：`meeting_id`、`topic_id`、`text/file_text`
 - 驗證 `meeting_id` 必填
 - 呼叫 `todos_extract_internal`
 - 回傳 JSON 結果，並附加 `debug_text_preview` (前 120 字)

▼表 9-2-7 虛擬碼- meeting.py

編號	2-3-4	程式名稱	meeting.py
目的	提供會議全生命週期的 API 與頁面路由，包括建立、查詢、刪除、更新會議及成員管理。支援自動產生會議名稱、權限控管、會前/會中/會後不同狀態的頁面切換，並維護與會者角色與檔案清單。		
虛擬碼			
1. 會議主頁 ○ 驗證登入狀態，若未登入導向 /signin。 ○ 讀取 topic_id 與 org_id。 ○ 從資料庫撈取議題資訊與該議題下的所有會議（依建立時間排序）。 ○ 回傳 HTML 頁面，帶上 topic、meetings、user 資訊。 2. 建立會議 API ○ 接收參數：date、creator_id、org_id、topic_id、participants。 ○ 驗證必填欄位。 ○ 使用 MySQL 建議鎖 GET_LOCK 確保會議次數不重複。 ○ 查詢同一 org/topic 下已存在會議標題解析出最大會議次數。			

- 自動產生「第 N 次會議」標題。
- 插入新會議資料，並把發起人設為主持人，其他成員依 email 新增。
- 提交並回傳成功訊息（含 meeting_id、title、next_no）。

3. 刪除會議 API

- 依序刪除會議相關子表資料 (files, voices, tasks 等)。
- 刪除會議主紀錄 (meetings)。
- 提交後刪除伺服器上的實體檔案夾 (uploads, embeddings)。
- 回傳成功或錯誤訊息。

4. 取得會議成員 API

- 連線資料庫，查詢該會議所有參與者資訊 (id, name, email, role)。
- 回傳 JSON。

5. 批次新增會議成員 API

- 接收 email 陣列與角色 role。
- 驗證格式，逐一查詢使用者是否存在並尚未加入。
- 新增至 meeting_participants，並回傳新增成員清單。

6. 移除會議成員 API

- 接收 email。
- 呼叫 delete_meeting_member_from_db 刪除該成員。
- 回傳成功或錯誤訊息。

7. 更新會議成員權限 API

- 驗證登入狀態與角色（必須是主持人）。
- 接收 email 與新 role，呼叫 update_meeting_member_role_db 更新資料庫。
- 回傳成功或錯誤訊息。

8. 查詢會議檔案清單（依類型）API

- 查詢指定會議、指定檔案類型的檔案資訊並回傳 JSON。

9. 查詢所有會議 API

- 撈取全部會議的 id、date、title。
- 回傳 JSON。

10. 查詢我的會議 API

- 撈取全部會議的 id、date、title。
- /api/my_meetings：取得使用者在某組織發起的所有會議。
- /api/my_meetings_simple：取得使用者參與過的所有會議（精簡版）。

11.查詢議題底下的會議 API

- 根據 topic_id + org_id + user_id 撈取會議清單，包含角色資訊。

12.自動產生預設會議名稱 API

- 查詢指定 org/topic 下的會議，計算最大次數。
- 自動生成下一次會議名稱「第 N 次會議」。
- 回傳 title 與 next_no。

13.結束會議 API

- 驗證登入狀態。
- 更新會議狀態為 after。
- 查詢 org_id 與 topic_id，回傳 JSON。

14.會議入口

- 依照會議狀態（before/during/after）跳轉到不同頁面：
 - before → /meeting_before
 - during → /meeting_during
 - after → /meeting_review

15.會前/會中/會後頁面路由

- 撈取全部會議的 id、date、title。
- /meeting_before/<meeting_id>：會前頁面，若狀態不是 before → 跳轉。
- /meeting_during/<meeting_id>：會中頁面，若狀態不是 during → 跳轉。
- /meeting_review/<meeting_id>：會後頁面，顯示會議回顧。

16.取得會議資訊 API

- 查詢指定會議的基本資料 (org_id, topic_id, status, title, date)。
- 回傳 JSON（不需登入）。

17.before_app_request Hook

- 在每次請求時自動檢查 URL 或 query string 是否帶有 meeting_id。

- 若存在，查詢會議資訊並存入 `g.meeting_ctx`，方便其他 API 使用。

▼表 9-2-8 虛擬碼- permission.py

編號	2-3-7	程式名稱	permission.py				
目的	處理會議過程中不同階段的 成員權限與角色設定。它會根據使用者登入狀態與在會議中的角色，決定顯示的頁面內容與操作權限。主要涵蓋： 1.會議後回顧頁的權限設定（判斷是否可編輯或僅能檢視） 2.會議前、會議中、會議後三個階段的成員列表頁面顯示，包含登入使用者在該會議的角色與相關資訊						
虛擬碼							
1. 會議後回顧權限設定 ◦ 接收參數：meeting_id、topic_id。 ◦ 驗證 meeting_id 是否存在，若缺少則回傳錯誤。 ◦ 驗證使用者登入狀態，未登入則導向登入頁。 ◦ 查詢資料庫： ▪ 取得會議資訊（含 topic_id）。 ▪ 若網址未帶 topic_id，自動補上會議的 topic_id。 ▪ 查詢當前使用者在 meeting_participants 的 role。 ◦ 判斷權限：若角色為主持人或 edit，權限為 edit，否則為 view。 ◦ 回傳會議回顧頁 meeting_review.html，帶入 meeting、meeting_id、topic_id、current_user_id、current_user_role、permission。 <tr><td colspan="4"> 2. 會議前會員列表設定 ◦ 接收參數：meeting_id。 ◦ 取得登入使用者的 user_id。 ◦ 查詢 meeting participants，取得該使用者在該會議的 role。 </td></tr>				2. 會議前會員列表設定 ◦ 接收參數：meeting_id。 ◦ 取得登入使用者的 user_id。 ◦ 查詢 meeting participants，取得該使用者在該會議的 role。			
2. 會議前會員列表設定 ◦ 接收參數：meeting_id。 ◦ 取得登入使用者的 user_id。 ◦ 查詢 meeting participants，取得該使用者在該會議的 role。							

- 若查無資料，預設角色為 view。
- 回傳會員列表頁 meeting_member.html，帶入 meeting_id、role、目前登入者 email。

3. 會議中會員列表設定

- 接收參數：meeting_id。
- 驗證登入狀態，若未登入則導向/signin。
- 取得登入使用者的 user_id 與 email。
- 查詢 meeting_participants，取得該使用者角色 role，若查無則預設為 view。
- 回傳會員列表頁 meeting_member2.html，帶入 meeting_id、role、使用者 email。

4. 會議後會員列表設定

- 接收參數：meeting_id。
- 驗證登入狀態，若未登入則導向/signin。
- 取得登入使用者的 user_id 與 email。
- 查詢 meeting_participants，取得該使用者角色 role，若查無則預設為 view。
- 回傳會員列表頁 meeting_member3.html，帶入 meeting_id、role、使用者 email。

▼表 9-2-9 虛擬碼- qa.py

編號	2-3-8	程式名稱	qa.py
目的	提供會議相關的問答(QA)、摘要(Summary)與歷史紀錄功能。它結合向量檢索(RAG)與 LLM 模型來回答問題與產生會議摘要，並提供介面讓使用者查詢、更新、刪除 QA 與摘要。		
虛擬碼			
1. 上傳與模型設定 ○ 設定檔案上傳目錄 <code>UPLOAD_FOLDER</code>。 ○ 設定 OllamaAPI 相關參數，限制可使用的模型 <code>ALLOWED_OLLAMA_MODELS</code>。 2. RAG 問答(/rag_qa) ○ 接收參數：<code>file_path</code>、<code>question</code>、<code>meeting_id</code>、<code>model</code>、<code>top_k</code>、<code>score_threshold</code>、<code>show_insufficient</code>。 ○ 驗證檔案與 <code>embedding</code> 是否存在，載入向量資料。 ○ 對問題編碼並計算相似度，取最相關的段落。 ○ 生成回答，依需求是否保留「資料不足」訊息。 ○ 紀錄 <code>QALog</code> 至資料庫。 3. 會議摘要產生(/api/llama_summarize) ○ 接收參數：<code>file_path</code>、<code>meeting_id</code>、<code>model</code>、<code>top_k</code>、<code>score_threshold</code>、<code>force_regen</code>。 ○ 優先載入已計算好的 <code>embedding</code>，否則從檔案萃取文字段落。 ○ 切分章節（時間、出席人員、討論事項、決議、待辦事項），並搭配關鍵詞比對。 ○ 對應內容向量化，挑選出關鍵片段，避免摘要缺漏。 ○ 組合成 <code>RAGcontext</code>，請模型輸出五個固定欄位的 JSON： ● 時間、出席人員、討論事項、決議、待辦事項 ○ 缺漏時以章節保底資料補齊。 ○ 寫入資料庫 <code>summaries</code> 與 <code>summary_logs</code>。 4. 摘要查詢與更新 ○ 查詢摘要(/api/summary/<meeting_id>)：依 <code>file_path</code> 查詢摘要。 ○ 更新摘要(/api/summary/<meeting_id>, POST)：需驗證 <code>role</code>（主持人或 <code>edit</code>），更新或新增摘要並記錄歷史。 ○ 查詢摘要歷史(/api/summary_logs/<meeting_id>)：取得指定檔案的修改紀錄。 5. QA 歷史			

- 頁面/qa_history/<meeting_id>，顯示該會議的 QA 歷史紀錄。
- 僅限會議參與者可檢視。
- 6. 會議清單/QALog 查詢
 - /api/meetings：列出所有會議的 id 與 title。
 - /api/qa_logs：列出指定 meeting_id 的 QA 紀錄。
 - /api/qa_logs/search：支援 meeting_id、keyword、date_from、date_to 條件搜尋 QA 紀錄。
- 7. 更新/刪除 QALog
 - 更新(/api/qa_logs/<qa_id>, PUT)：更新 answer，並紀錄最後修改者。
 - 刪除(/api/qa_logs/<log_id>, DELETE)：需為原作者或會議的 host/edit 權限。
- 8. 會議評論頁(/meeting_review_page)
 - 接收參數：meeting_id、topic_id。
 - 查詢會議資訊與使用者角色，決定 permission 為 edit 或 view。
 - 回傳 meeting_review.html。
- 9. 依 topic 查詢會議清單(/api/meetings_by_topic/<topic_id>)
 - 需登入。
 - 回傳該 topic_id 下，使用者有參與的所有會議清單。

▼表 9-2-10 虛擬碼-task.py

編號	2-3-9	程式名稱	task.py
目的	提供任務的建立、查詢、編輯、刪除及通知功能，並整合會議、議題與組織資訊		
虛擬碼			
1. 初始化與設置 ● 載入模組： flask(Blueprint,render_template,request,jsonify,session,redirect,abort,current_app)、db(get_db_cursor)、email_utils(send_email_notification)、datetime ● 定義常數：URGENT_WINDOW_DAYS、資料表名稱與欄位、任務狀態白名單 ● 建立 Blueprint：task_bp ● 定義_send_task_mail_enabled()控制寄信開關 2. 登入與權限工具 ● _current_user_id()：回傳目前登入使用者 ID ● require login()：若未登入則 abort(401)			

- `_is_participant(meeting_id,user_id)`：檢查是否為該會議參與者
- `_norm_id(v)`：清洗輸入 ID，容錯處理
- `_due_status(due_date_str)`：判斷任務是否緊急，計算剩餘天數
- `_infer_org_and_topic(cur,org_id,topic_id,meeting_id)`：由 meeting/topic 推回 org_id 並補齊 topic_id
- `_fetch_ctx_and_lines(cur,org_id,topic_id,meeting_id)`：查詢並組合所屬組織/議題/會議資訊，供信件使用

3. 頁面與 API

- `/tasks(GET)`：任務追蹤主頁，需登入
- `/api/participants(GET)`：取得指定會議的參與者清單（下拉選單）
- `/api/meeting/<mid>/info(GET)`：查詢會議狀態

4. 建立任務

- `/api/task(POST)`：
 - 驗證登入與參與權限
 - 檢查必填欄位（`assignee_id`、`name`、`due_date`）
 - 驗證日期格式，禁止過去日
 - 檢查 assignee 存在且為會議參與者
 - 寫入 DB，建立任務
 - 查詢任務詳細資訊（受指派者、指派者、會議、組織、議題）
 - 若允許寄信，依緊急與否發送 email 給 assignee
 - 同時透過 LINE 推播通知受指派者（`_send_line_notification`）
 - 回傳任務 ID 與相關資訊

5. 查詢任務

- `/api/tasks(GET)`：
 - 支援 `org_id`、`meeting_id`、`topic_id` 查詢
 - 若 `org_id` 缺，會自動推導
 - 驗證會議參與權限
 - 回傳任務列表與解析後的 `org/topic/meeting_id`

6. 刪除任務

- `/api/task/<task_id>(DELETE)`：

- 查詢任務與相關人資訊
- 驗證刪除權限（僅 assigner 可刪除）
- 刪除 DB 資料
- 若允許寄信，寄送刪除通知給 assignee 與 assigner，包含任務內容與所屬資訊
- 同時透過 LINE 推播通知 assignee 與 assigner
（_send_line_notification）

7. 編輯任務

- /api/task/<task_id>(PUT/PATCH)：
 - 權限：assigner 可修改
name/description/due_date/assignee_id/meeting_id/topic_id；assignee 僅能修改 status
 - 驗證輸入格式與日期
 - 若 assignee 或 meeting_id 有變更，檢查參與權限
 - 更新 DB 並取回最新資料
 - 若允許寄信，依更新類型寄送通知：
 - 負責人變更→通知舊 assignee、新 assignee 與 assigner
（Email+LINE）
 - 狀態變更→通知 assigner（Email+LINE）
 - 一般欄位變更→通知 assignee（Email+LINE）

▼表 9-2-11 虛擬碼- rag_pipeline.py

編號	2-4-4	程式名稱	rag_pipeline.py
目的	從會議資料夾蒐集文字內容，經過清洗與合併後，建立或載入 RAG 向量索引，並支援強制重建、檔案過濾與文字裁切		
虛擬碼			
6. 初始化與設置 ● 載入模組：os、logging、hashlib、typing(List,Tuple,Iterable) ● 載入內部模組： ➤ text_extract.extract_text from file			

- `rag_utils.build_or_load_index`
- 定義常數：
 - `INDEX_FILES`：常見索引檔名（`index.faiss`、`index.pkl`、`.faiss`、`.index`）
 - `IGNORE_DIRS`：需忽略的目錄（`faiss_index`、`pycache`、`.cache`、`.git`）
 - `IGNORE_SUFFIX`：需忽略的暫存或快取副檔名（`.tmp`、`.part`、`.swp`、`.swx`、`~`）
- 初始化 logger

7. 定義函數 `iter_files(meeting_dir,exts,recursive=True)`

- 建立允許副檔名集合 `norm_exts`（轉小寫、補上.）
- 定義內部判斷函數 `_want(fname)`：
 - 排除索引檔（在 `INDEX_FILES`）
 - 排除隱藏檔（以.開頭）
 - 排除暫存檔（結尾在 `IGNORE_SUFFIX`）
- 若 `recursive=True`：
 - 使用 `os.walk` 遍歷 `meeting_dir`
 - 過濾 `IGNORE_DIRS` 與隱藏資料夾
 - 過濾副檔名後 yield 檔案完整路徑
- 否則：
 - 只處理最上層檔案並 yield

8. 定義函數 `clean_join(parts : List[str])→str`

- 建立 `seen` 集合與 `cleaned` 清單
- 對每個片段：
 - 去除空行、修剪空白
 - 若非空：計算 SHA1 值，避免重複
 - 新增到 `cleaned`
- 以「雙換行」合併並回傳

9. 定義函數

`build_index_from_meeting_folder(meeting_id,base_dir="uploads",exts=None, rebuild=False,recursive=True,max_chars=500000)`

- 流程：

01. 副檔名設定

- 若 `exts=None`→嘗試讀取環境變數 `RAG_ALLOWED_EXTS`，否則使用預設`[.txt,.docx,.pdf]`

02. 檢查會議資料夾

- 路徑：`base_dir/meeting_{id}`
- 若不存在→`raiseFileNotFoundError`

03. 抽取文字

- 建立 `parts=[]`
- 遍歷 `_iter_files` 找到的檔案：
 - 呼叫 `extract_text_from_file` 抽取文字
 - 若成功→`parts` 加入【檔名】+文字（檔名僅供除錯，不混入語意）
 - 若失敗→`logger.warning`

04. 清理合併文字

- `combined_text=_clean_join(parts)`
- 若為空→`raiseValueError`

05. 處理過長文字

- 若 `len(combined_text)>max_chars`：
 - 裁切前後各一半，中間以`...\n`替代
 - `logger.warning`

06. 建立索引

- 若 `rebuild=True`：
 - 嘗試呼叫 `build_or_load_index(...,rebuild=True)`
 - 若 `TypeError`（舊版不支援）→忽略並繼續
- 否則：直接呼叫 `build_or_load_index`

07. 回傳結果

- `vs`（向量索引物件）
- `idx_dir`（索引資料夾路徑）
- `combined_text`（清理後文字）

▼表 9-2-12 虛擬碼- flask_blueprint.py

編號	2-4-6	程式名稱	flask_blueprint.py
目的	建立 /punct API Blueprint，提供三個端點：SRT 補標點、純文字補標點、健康檢查。		
虛擬碼			

9. 初始化與設置

- 匯入：Blueprint, request, jsonify (Flask) ；List, Any (typing)
- 匯入：
 - punctuate_srt_lines → 用於逐行 SRT 標點
 - srt_utils 模組 (parse_lines, write_entries, list_to_srt_blocks)
 - punctuate_with_gpt → GPT 純文字補標點
- 定義 punct_bp = Blueprint("punct", __name__)
- 定義 _json_error(msg, code) → 回傳統一錯誤 JSON

10. API 路由

- POST /punctuate_srt
 - ◆ 輸入：

```
{  
  "lines": ["00:00:00,000 --> 00:00:05,000 | 大家好今天來測試", ...],  
  "batch_size": 30,  
  "return_standard": true  
}
```
 - ◆ 流程：
 - 取出 lines，檢查格式。
 - 呼叫 punctuate_srt_lines → GPT 批次補標點。
 - 將結果存入 resp["lines"]。
 - 若 return_standard 為真 → 嘗試用 list_to_srt_blocks 轉為標準 SRT 區塊；失敗則 fallback parse_lines + write_entries。
 - ◆ 輸出：

```
{  
  "success": true,  
  "lines": [...],  
  "count": 數量,  
  "srt": "標準 SRT 區塊 (可選)"  
}
```

- POST /punctuate_text

- ◆ 輸入：

```
{  
  "text": "沒有標點的中文長句……",  
  "fallback_basic": true  
}
```

- ◆ 流程：

- 取出 text，檢查非空。
- 呼叫 punctuate_with_gpt → GPT 加標點。
- 若失敗且 fallback_basic = true → 載入 .punctuate 模組：
 - 優先使用 bart_punctuation
 - 若失敗則使用 basic_punctuation
 - 在回傳結果中加 "fallback": "offline" 標記。

- ◆ 輸出：

```
{ "success": true, "result": "加過標點的完整文本" }
```

- GET /health

- ◆ 流程：

- A. 定義測試句："測試一下沒有標點的句子如果有連到應該會變好看一點"
- B. 呼叫 punctuate_with_gpt → 嘗試補標點。
- C. 判斷回傳字串長度是否 $\geq$ 輸入 → 作為 ok 判斷依據。

- ◆ 輸出：

```
{  
  "ok": true/false,  
  "in": "測試句",  
  "out": "加標點結果",  
  "len_in": 13,  
  "len_out": 18  
}
```

▼表 9-2-13 虛擬碼-formal_doc.py

編號	2-4-14	程式名稱	formal_doc.py
目的	依據會議整理檔與選定模板，自動生成正式會議文件(Word)，並存檔與回傳資訊。		
虛擬碼			
10. 初始化與設置 - 載入模組：載入模組：flask(Blueprint, request, jsonify, send_file)、os、re、logging、py pandoc、OpenAI、db(get_db)。 - 定義 Blueprint：formal_doc_bp。 - 建立 logger。 - 設定檔案目錄：INPUT_DIR (meeting_drafts)、OUTPUT_DIR (formal_docs)。 - 初始化 OpenAI client。 11. 工具函式 - get_next_filename(base_name, output_dir, suffix)：依序號產生不重複檔名。 - read_meeting_file_from_db(meeting_id)： - 從 DB 讀取最新「會議紀錄整理」檔案。 - 若不存在或檔案遺失 → 回傳錯誤。 - 支援 .txt、.docx 解析，回傳文字內容與檔名。 _is_participant(meeting_id,user_id)：檢查是否為該會議參與者。 - generate_meeting_doc(content, headings, output_name, template_id, ref_filename)： - 依 template_id 決定參考樣板與圖示。 - 組合 Markdown prompt，送入 OpenAI API 生成正式書面文字。 - 後處理清理格式。 - 使用 py pandoc 轉換為 Word 檔。 - 輸出檔案至 OUTPUT_DIR，回傳檔案路徑。 12. 模板清單 - 定義 TEMPLATES 陣列（大學校務會議、例行性會議、法律效力會議）。 - 每個模板包含 id、name、description、preview_url、headings。 13. API 路由 /templates (GET)：回傳模板清單 JSON。 /generate_formal_doc (POST)： - 接收參數：meeting id、template id、output name、uploaded by。			

- 檢查模板是否存在，不存在則回傳錯誤。
- 讀取會議整理檔，若失敗則回傳錯誤。
- 呼叫 generate_meeting_doc 生成正式文件。
- 產生輸出檔名與檔案路徑。
- 寫入 DB (formal_documents 資料表)，包含會議、檔案、範本、上傳者資訊。
- 回傳 JSON：status、file_path、file_name。
- 若例外 → logger 記錄，回傳錯誤 JSON。查詢任務詳細資訊（受指派者、指派者、會議、組織、議題）。

▼表 9-2-14 虛擬碼- realtime_stt_ws.py

編號	2-5-7	程式名稱	realtime_stt_ws.py
目的	提供 WebSocket 代理服務，讓瀏覽器透過 Flask 與 OpenAI Realtime 雙向通訊，並支援健康檢查。		
虛擬碼			
11. 初始化與設置 ● 匯入模組： ➤ flask → Blueprint ➤ flask_sock.Sock → WebSocket 支援 ➤ websocket → 建立與 OpenAI Realtime 連線 ➤ threading → 多執行緒處理收發 ➤ json, os, traceback → 環境變數、資料處理、錯誤追蹤 ● 主要物件： ■ ws_bp = Blueprint("ws", __name__) ■ sock = Sock() ● 環境變數： ■ OPENAI_API_KEY → OpenAI 金鑰 ■ REALTIME_MODEL → 預設 gpt-4o-realtime-preview-2024-12-17 ■ OPENAI_RT_URL → OpenAI Realtime API WebSocket URL			
12. 函數與功能 ● 公用工具： ➤ _send_json(ws, obj) → 傳送 JSON 給客戶端。 ➤ _send_error(ws, message) → 傳送錯誤訊息給客戶端。 ➤ _connect_upstream() → 建立與 OpenAI Realtime WebSocket 的連線。 ■ 驗證 OPENAI_API_KEY 是否存在。 ■ 設定 headers → 加入金鑰與 Realtime 版本。			

- 成功 → 回傳連線物件。
- 失敗 → 拋出錯誤。
- 健康檢查：
 - @sock.route("/ws/ping") → ws_ping(ws)
 - 持續接收客戶端訊息並回傳。
 - 若為文字 → 原樣回覆。
 - 若為二進位 → 回傳 pong。
 - 可用來測試連線是否正常。
- Realtime 主代理
 - @sock.route("/ws/realtime") → realtime_proxy(ws)
 - 功能：代理瀏覽器與 OpenAI 雙向通訊。
 - 流程：
 - ◆ 連線至 OpenAI Realtime。若失敗 → 回傳錯誤。
 - ◆ 建立 closed 事件旗標。
 - ◆ 啟動三個執行緒：
 - _upstream_keepalive
 - 每 20 秒送一次 ping，保持與 OpenAI 的連線。
 - _client_to_openai
 - 接收來自瀏覽器的訊息並轉發到 OpenAI。
 - 支援文字與二進位。
 - _openai_to_client
 - 接收來自 OpenAI 的訊息並回傳給瀏覽器。
 - 錯誤時回報訊息（如連線關閉、例外）。
 - ◆ join() 等待三個執行緒完成，確保連線過程一致。

13. API 端點總覽

- GET /ws/ping → 健康檢查 Echo Server
- WS /ws/realtime → 雙向代理通道 (Browser ↔ Flask ↔ OpenAI)

▼表 9-2-15 虛擬碼-rag_utils.py

編號	2-4-5	程式名稱	rag_utils.py
目的	提供 RAG(Retrieval-AugmentedGeneration)工具： 建立或載入 FAISS 向量索引，支援動態切分文本、內容雜湊檢查、去重與安全寫入，避免重複或錯誤索引建立		
虛擬碼			
14. 初始化與設置 ● 載入模組：os、hashlib、json、tempfile、shutil、logging、typing、contextlib ● 載入 LangChain 模組： ➢ RecursiveCharacterTextSplitter ➢ HuggingFaceEmbeddings ➢ FAISS ● 環境變數參數： ➢ EMB_NAME：embedding 模型名稱，預設"sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2" ➢ EMB_DEVICE：指定裝置"cuda"/"cpu"/None ➢ ALLOW_DANGEROUS：是否允許危險的反序列化 ● 初始化 logging ● 建立 embedding 單例快取變數			
15. 內部工具函數 ● _get_embeddings() ➢ 若尚未載入→初始化 HuggingFaceEmbeddings ➢ 使用 normalize_embeddings=True ➢ 回傳 embedding 物件 ● _sha256(s) ➢ 計算輸入字串的 SHA256 雜湊值 ● _clean_text(s) ➢ 去除 BOM			

- 移除多餘空白與空行
- 回傳乾淨文本
- `_safe_write_json(path,data)`
 - 將 JSON 先寫入 tmp 檔，再原子替換至目標路徑，避免半寫入
- `_try_load_faiss(idx_dir)`
 - 嘗試載入既有 FAISS 索引 (`index.faiss+index.pkl`)
 - 若成功→回傳向量庫物件
 - 若失敗→回傳 None
- `_file_lock(lock_path)`
 - 提供檔案鎖 (`fcntl`)，避免多進程同時重建索引

16. 主要函數

`build_or_load_index(meeting_id,full_text,base_dir="uploads",rebuild=False)`

- 流程：
 01. 建立索引目錄 `uploads/meeting_{id}/faiss_index`
 - 定義 `index.faiss`、`index.pkl`、`CONTENT_HASH.json`、`.lock`
 02. 清理 `full_text` (去 BOM/空白)，若為空→`raise ValueError`
 03. 計算新內容 SHA256
 04. 使用 `_file_lock` 確保操作安全
 05. 若 `rebuild=False` 且索引檔存在：
 - 嘗試讀取舊 hash
 - 若舊 hash==新 hash→嘗試 `_try_load_faiss`
 - 成功→直接回傳既有索引
 - 失敗→發出警告並進行重建
 06. 重建索引
 - 動態設定 `chunk_size` 與 `overlap`：
 - <2000chars→(500,50)
 - <20000chars→(1000,100)
 - 否則→(1500,200)
 - 使用 `RecursiveCharacterTextSplitter` 切分文本
 - 去重：對每個 chunk 計算 SHA256，避免重複片段

- 建立向量索引：FAISS.from_documents(uniq_docs,embeddings)

07. 安全保存

- 將索引先存入暫存資料夾.tmp_save
- 再移動至正式路徑（index.faiss+index.pkl）
- 清理暫存資料夾

08. 更新內容雜湊

- 呼叫_safe_write_json 更新 CONTENT_HASH.json

09. 回傳(vs,idx_dir)

第 10 章 測試模型

10-1 測試計劃

本系統「會議寶」的設計目的在於協助使用者進行會議管理與智慧化紀錄，核心功能包含即時語音轉錄、檔案上傳轉逐字稿、AI 自動生成摘要、任務追蹤與會議回顧等，並提供跨平台的使用體驗。為確保系統功能的正確性與穩定性，本測試計劃將針對各項主要功能進行驗證，確認使用者在不同環境下均能順利操作與獲取正確的會議資訊。系統執行所需的環境僅需具備可連線至網際網路的智慧型裝置或電腦。完成開發設計後，將於上述環境中依測試項目逐一檢驗，確保功能可完整運作並符合使用需求。

● 測試方法

1. 功能測試：針對每個模組（會員帳號管理、會議管理、檔案上傳、語音轉文字、會議回顧、AI 查詢、任務追蹤、通知提醒等）進行逐項驗證，確保能正確執行。
2. 資料一致性測試：確認資料庫寫入與查詢結果一致，避免遺漏或重複紀錄。

● 進行方式

1. 依據系統架構將測試項目劃分為：會員帳號管理、會議管理、檔案上傳與查詢、語音轉文字、會議回顧、文件生成與任務追蹤、系統管理等七大類模組。
2. 為各模組設計測試案例 (Test Case)，包含測試作業與預期結果。
3. 由測試人員依序執行測試案例，並記錄實際結果與是否通過。
4. 當測試結果與預期不符時，提交至開發團隊修正，再次進行回歸測試 (Regression Testing)。

10-2 測試個案與測試結果

▼表 10-2-1 功能測試-註冊/登入/登出

測試項目	註冊/登入/登出
測試作業	輸入信箱與密碼完成註冊，驗證能正確登入與登出。
預期結果	能成功註冊帳號，登入後顯示首頁，登出後回到登入頁。
實際結果	通過

▼表 10-2-2 功能測試-修改會員資料

測試項目	修改會員資料
測試步驟	編輯會員姓名、單位或密碼，並重新登入檢查更新。
預期結果	修改後資料正確儲存並顯示於會員資訊頁。
實際結果	通過

▼表 10-2-3 功能測試-建立組織/邀請成員

測試項目	建立組織 / 邀請成員
測試步驟	建立新組織並輸入成員信箱進行邀請。
預期結果	組織成功建立，受邀成員能收到邀請並加入。
實際結果	通過

▼表 10-2-4 功能測試-創建議題/創建會議

測試項目	創建議題/創建會議
測試步驟	輸入議題名稱與內容後新增議題，並建立會議。
預期結果	議題與會議正確顯示於清單中。
實際結果	通過

▼表 10-2-5 功能測試-設定會議權限

測試項目	設定會議權限
測試步驟	為不同角色（主持人、與會者、訪客）設定權限。
預期結果	不同角色能正確限制存取與操作功能。
實際結果	通過

▼表 10-2-6 功能測試-發送會議通知

測試項目	發送會議通知
測試步驟	建立會議後發送通知。
預期結果	所有成員能收到會議通知（Email 或 Line）。
實際結果	通過

▼表 10-2-7 功能測試-即時轉錄

測試項目	即時轉錄
測試步驟	於會議中開啟即時錄音並轉逐字稿。
預期結果	逐字稿即時顯示，與發言內容一致。
實際結果	通過

▼表 10-2-8 功能測試-檔案轉錄

測試項目	檔案轉錄
測試步驟	會議後上傳音檔，由系統自動轉逐字稿。
預期結果	逐字稿產出完整，支援下載 TXT / DOCX / SRT。
實際結果	通過

▼表 10-2-9 功能測試-檔案上傳

測試項目	檔案上傳
測試步驟	上傳 PDF、Word、音訊或影片檔案。
預期結果	檔案成功儲存並顯示於檔案清單
實際結果	成功

▼表 10-2-10 功能測試-檔案區瀏覽/智能查詢

測試項目	檔案區瀏覽 / 智能查詢
測試步驟	開啟檔案區，選擇檔案並輸入問題進行 AI 查詢。
預期結果	系統回覆與檔案內容相關的正確答案。
實際結果	通過

▼表 10-2-11 功能測試-生成摘要

測試項目	生成摘要
測試步驟	選擇逐字稿並產生摘要。
預期結果	自動產生摘要，內容與檔案一致
實際結果	通過

▼表 10-2-12 功能測試-修改/複製摘要/檢視修改紀錄

測試項目	修改/複製摘要/檢視修改紀錄
測試步驟	修改摘要並檢視歷史版本。
預期結果	修改能正確儲存，紀錄可追蹤。
實際結果	通過

▼表 10-2-13 功能測試-查詢紀錄檢視與修改

測試項目	查詢紀錄檢視與修改
測試步驟	輸入關鍵字查詢過往問答紀錄，並進行修改。
預期結果	紀錄能正確檢索與更新。
實際結果	通過

▼表 10-2-14 功能測試-逐字稿整理→正式文件

測試項目	逐字稿整理→正式文件
測試步驟	將逐字稿匯出成 PDF 或 Word。
預期結果	正式文件能正確下載與開啟。
實際結果	通過

▼表 10-2-15 功能測試-新增/編輯/刪除任務

測試項目	新增/編輯/刪除任務
測試步驟	建立任務並指定負責人，之後修改或刪除。
預期結果	任務能正確新增、更新與移除。
實際結果	通過

▼表 10-2-16 功能測試-發送任務提醒

測試項目	發送任務提醒
測試步驟	建立任務後發送提醒通知。
預期結果	負責人能正確收到任務提醒。
實際結果	通過

▼表 10-2-17 功能測試-帳號維護/檔案維護

測試項目	帳號維護 / 檔案維護
測試步驟	以管理者身份登入，編輯帳號或刪除檔案。
預期結果	帳號與檔案能正確維護。
實際結果	通過

▼表 10-2-18 功能測試-資料庫管理/AI 模型維護

測試項目	資料庫管理/AI 模型維護
測試步驟	檢查資料庫連線，並更新 AI 模型。
預期結果	DB 正常存取，模型能正確維護。
實際結果	通過

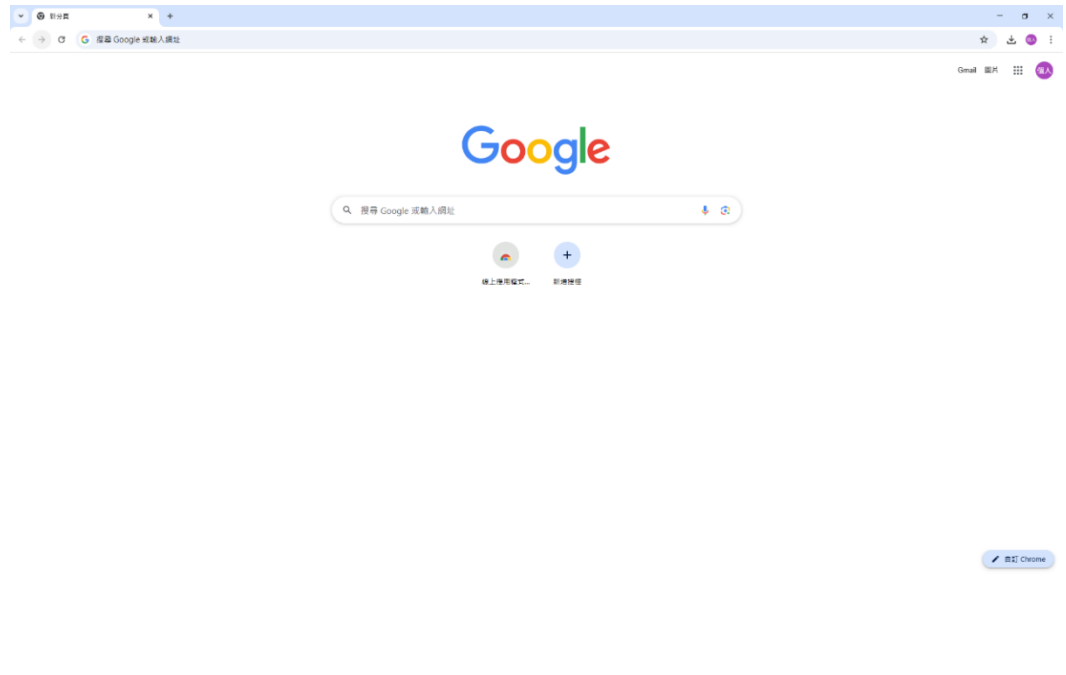
第 11 章 操作手冊

11-1 系統元件

▼表 11-1 系統安裝元件資訊

元件名稱	會議寶 MeetingBae
費用	免費
版本需求	Windows 10
網路需求	Wifi/4G/5G 網路

11-2 系統下載及安裝

步驟一	
	開啟瀏覽器 (Chrome / Safari / Edge / Firefox) 進入 (網址)。

步驟二



[功能簡介](#) [關於我們](#)

會員登入

Email

密碼

[忘記密碼?](#)

[還沒有帳號? 立即註冊](#)

☐ 想接收最新會議通知?

輸入網址：

[http://meetingbae.ipicbox](http://meetingbae.ipicbox.tw)
[.tw](http://meetingbae.ipicbox.tw) 進入主頁開始會員

登入。

會員註冊

姓名

請輸入姓名

Email

請輸入 Email

密碼

需包含英文大小寫與數字，至少8碼

確認密碼

請再次輸入密碼

若未註冊則先到註冊頁
面進行註冊。

步驟三

☰

首頁

使用說明

登出

會議寶

+ 建立組織

我的組織

會議寶

rrr

即可進入會議寶進行會議

第 12 章 使用手冊

登入頁面

使用者尚未登入時，開啟系統網址，有登入以及註冊。點選登入，輸入電子郵件及密碼後即可按下登入按鈕進入 App。

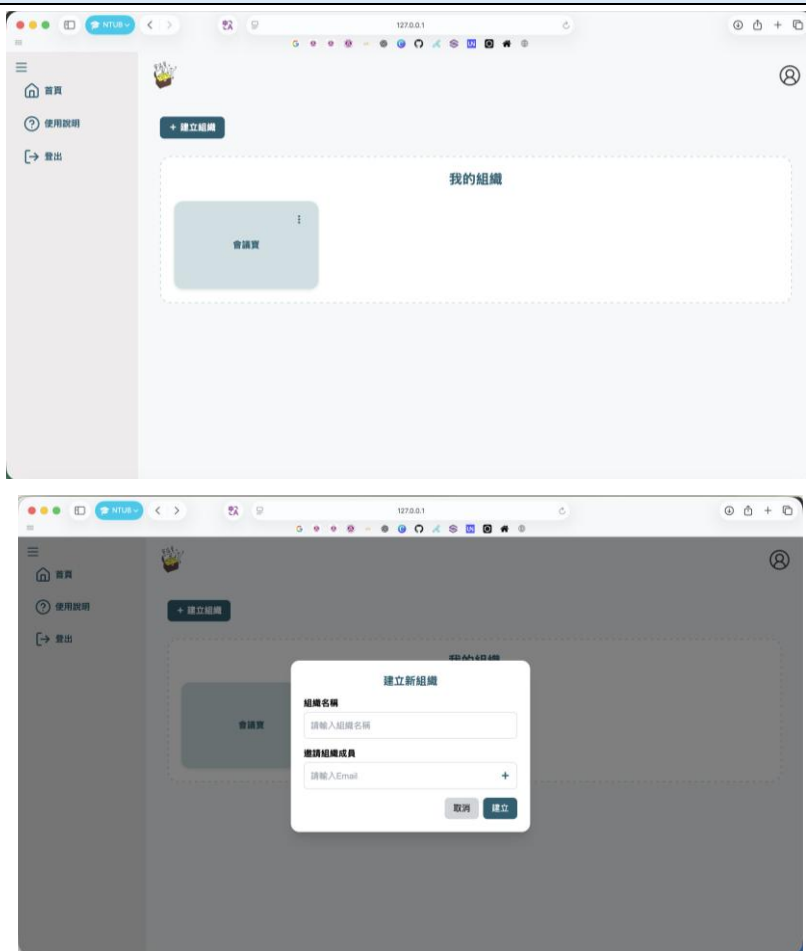
註冊頁面

使用者尚未登入時，開啟系統網址，有登入以及註冊。點選註冊帳號，輸入電子郵件、密碼、個人相關資料以及確認密碼後，即可註冊。

忘記密碼頁面

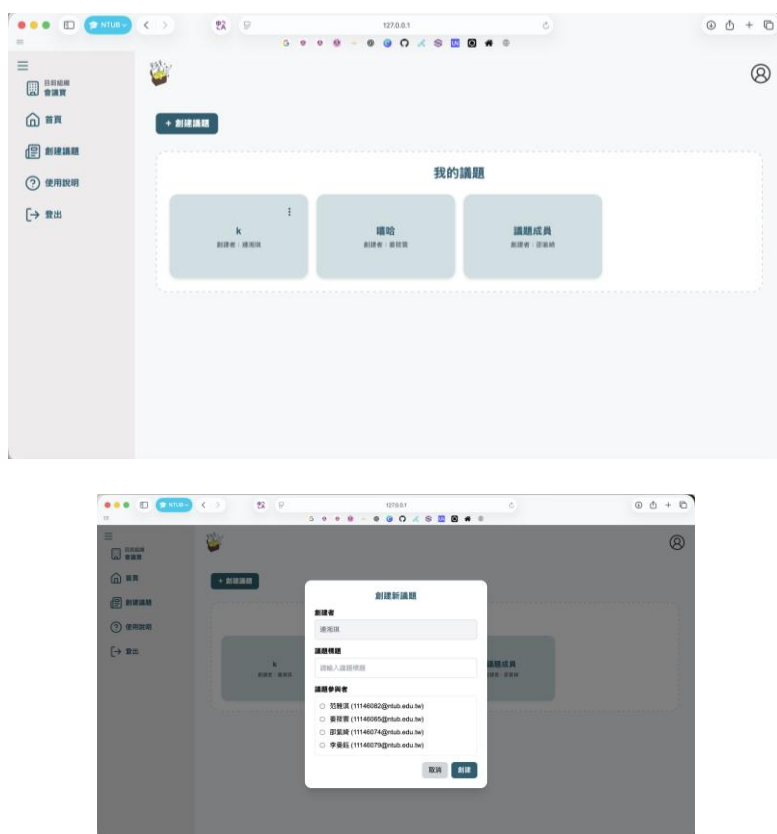
若使用者忘記密碼，在登入頁面中，可以點選忘記密碼，並輸入電子郵件，傳送後即可在信箱收到隨機密碼，輸入隨機密碼進入系統網頁，可在個人資料頁面更改新的密碼。

建立組織頁面



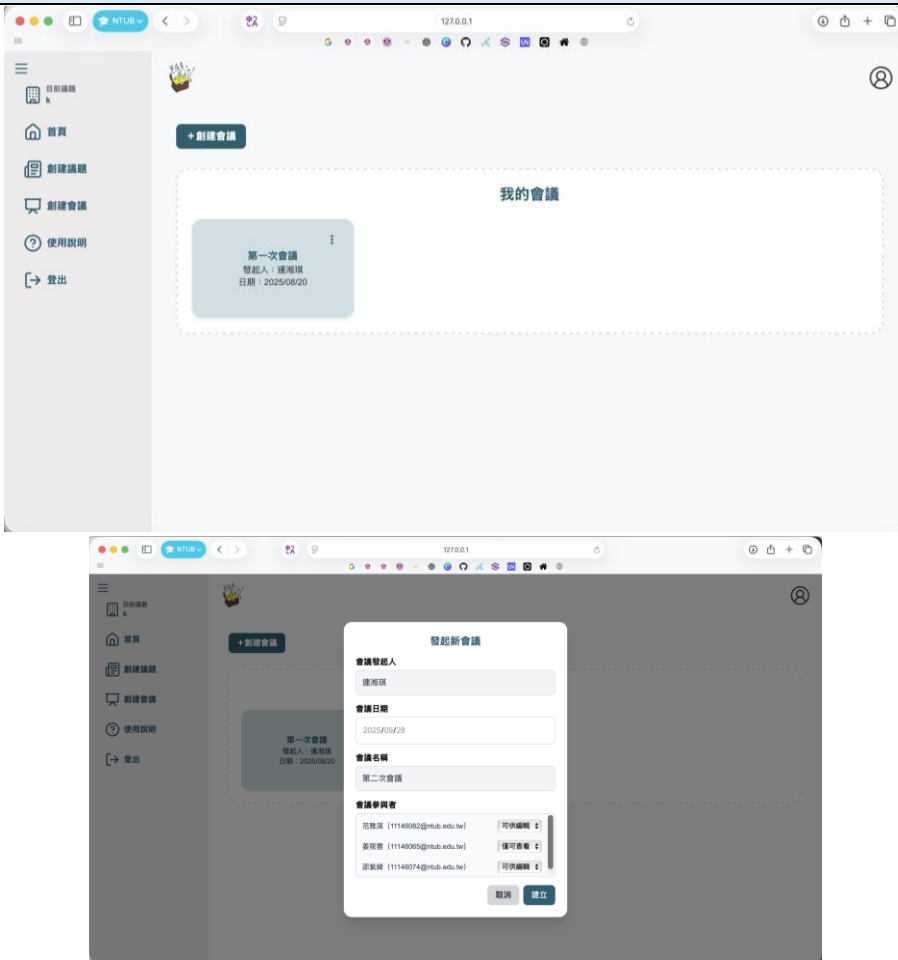
使用者登入後，會進入建立組織頁面，可點選+建立組織，將每一項目填寫完成後，按下建立按鈕，即可於頁面中成功新增該組織卡片。

建立議題頁面



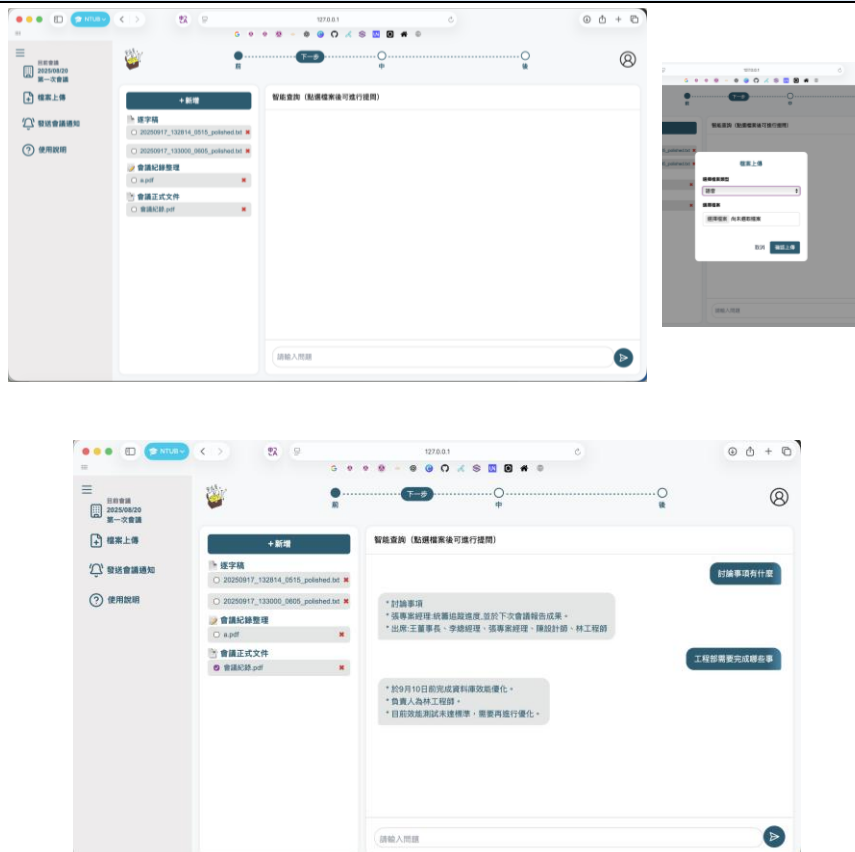
按下組織卡片，進入建立議題頁面，按下+建立議題，將每一項目填寫勾選完成後，按下創建按鈕，即可於頁面中成功新增該議題卡片。

建立會議頁面



按下議題卡片，進入建立會議頁面，按下+創建會議，將每一項目填寫完成後，按下建立按鈕，即可於頁面中成功新增該會議卡片。

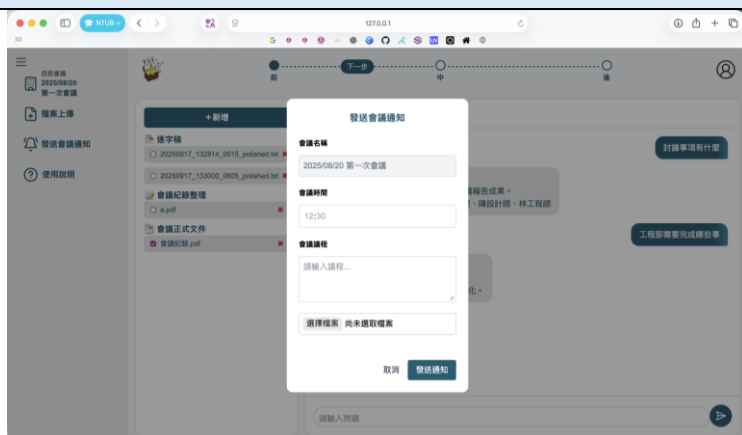
會議前狀態（檔案上傳、智能查詢）頁面



按下會議卡片，進入會議前狀態頁面，點選+新增，可依上傳檔案選擇檔案類型（語音、逐字稿、會議記錄整理、會議正式文件），點選確認上傳後，即可於檔案區成功上傳檔案。

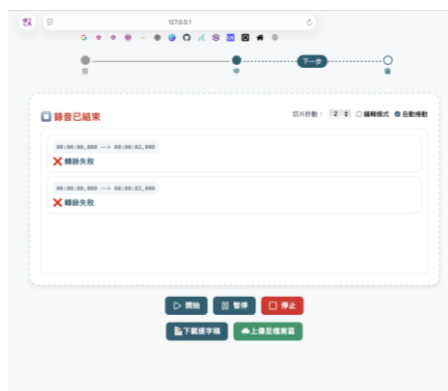
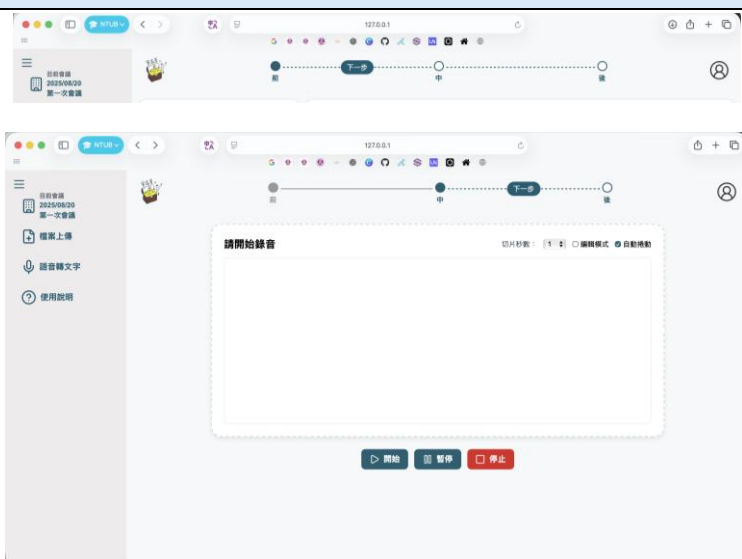
在檔案區中勾選想要詢問的檔案後，即可在智能查詢區提問，獲得及時解答。

會議前狀態（發送會議通知）頁面



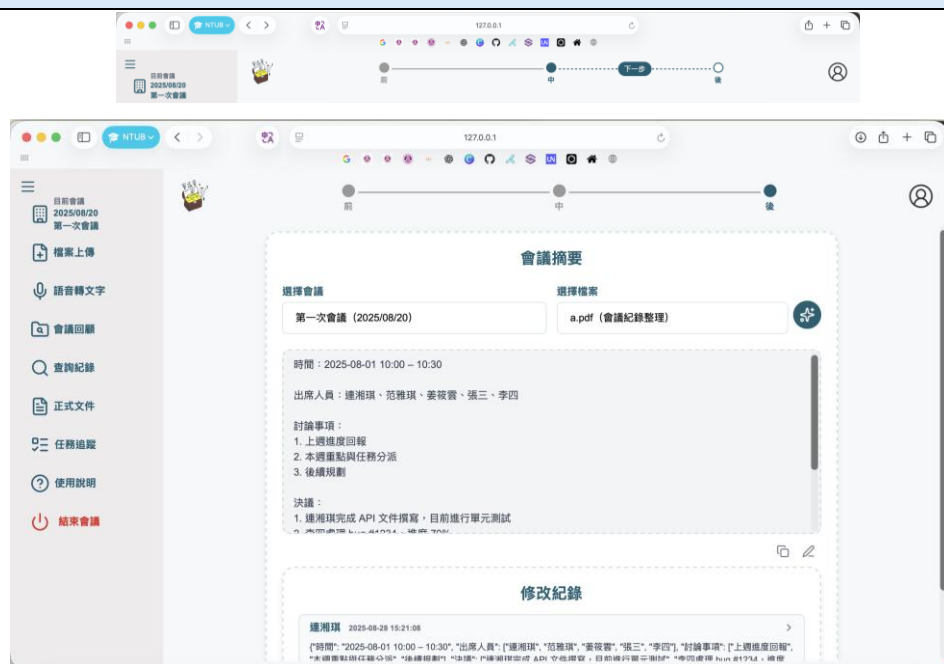
點選側邊欄發送會議通知按鈕，進入發送通知卡片，將每一項目填寫後，按下發送通知按鈕，即發送至參與會議的相關成員郵件，Line Bot 也將同步發送通知。

會議中狀態（即時轉錄）頁面



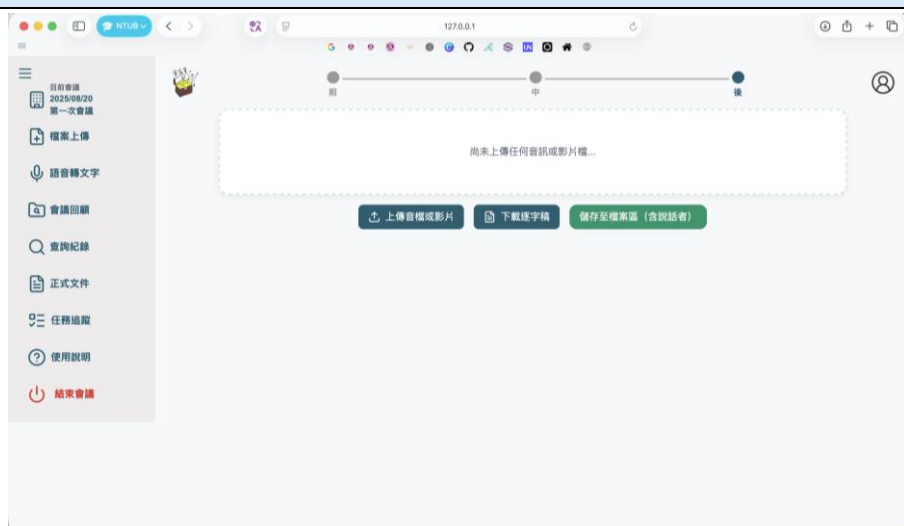
在會議前狀態中點選狀態列的下一步，即跳轉至會議中即時轉錄頁面，點選開始按鈕，即開始錄音，按下停止按鈕後可選擇下載逐字稿及將檔案上傳至檔案區。

會議後狀態（會議回顧）頁面



在會議中狀態列點選下一步按鈕，即跳轉至會議後會議回顧頁面，選擇會議及檔案，按下生成按鈕，利用 AI 生成該會議摘要，並可點選複製與修改按鈕。修改後的紀錄也會在頁面上呈現。

會議後狀態（語音轉文字）頁面



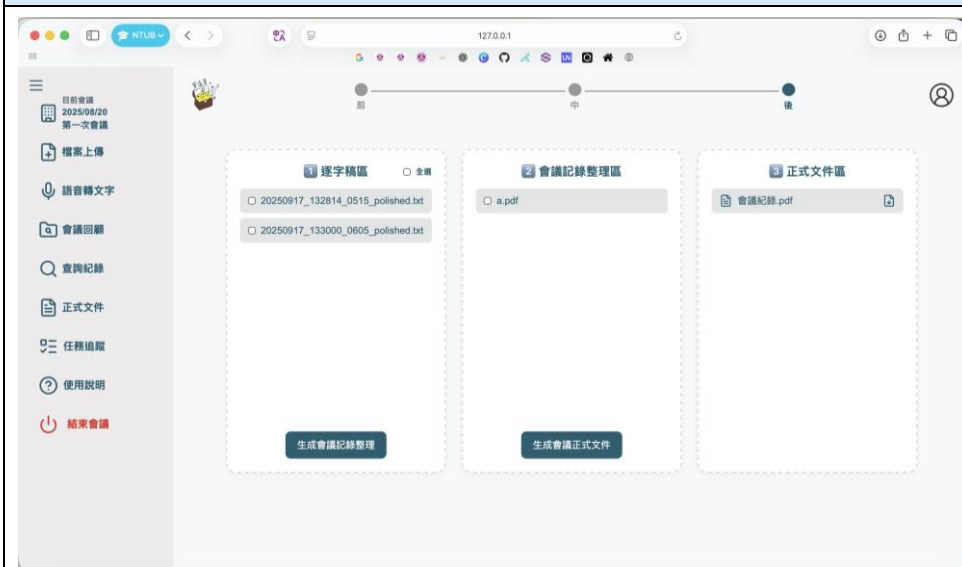
會議後狀態的語音轉文字功能提供使用者上傳音檔轉換為逐字稿，可供下載即上傳至檔案區。

會議後狀態（查詢紀錄）頁面



選擇會議，即可跳出歷史智能查詢中的問答，顯示提問者與時間，也可進行修改或刪除。

會議後狀態（正式文件）頁面

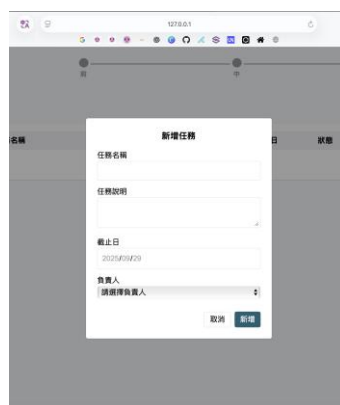


在正式文件區，點選要轉會議紀錄整理的文件後，點選生成會議紀錄整理，即完成會議紀錄整理。勾選需要轉正式文件的檔案，點選生成正式文件按鈕，選擇模板後，利用 AI 生成正式文件。

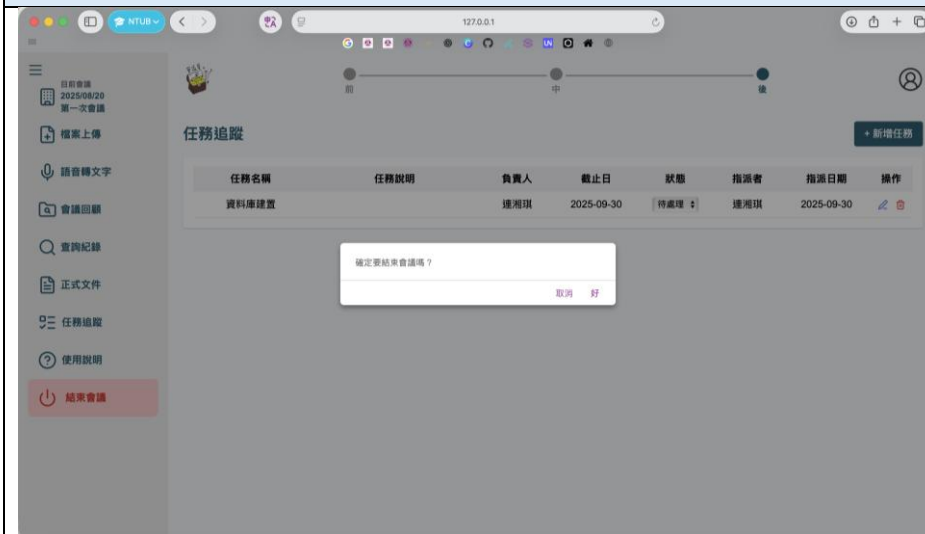
會議後狀態（任務追蹤）頁面



點選+新增任務，將每一項目填寫勾選完成後，按下新增後，即可在頁面中成功新增任務。可由負責人編輯狀態及修改或刪除。在任務新增與截止日前三天，將利用 SMTP 與 LineBot 發送通知。



會議結束狀態



按下側邊欄結束會議，將結束會議流程，只留下會議後狀態，供使用者回顧會議資訊。

第 13 章 感想

➤ 11146065 姜筱雲

在這次「會議寶」專題中，我獲得了許多寶貴的經驗。一開始對題目和進度都有些不安，擔心功能是否能如期完成，也擔心組員之間的想法會不會有落差。不過隨著討論與老師的指導，我們逐漸釐清方向，將抽象的構想一步步落實為具體成果。組員間的合作也非常順利，大家都能各司其職，並在遇到問題時互相協助，讓專題能穩定推進。我很慶幸能和這群夥伴一起努力，過程中也深刻體會到團隊合作的重要性。

我的任務是負責前端頁面設計，主要將後端資料轉化成清楚、直觀又好操作的介面。實作過程中，我不僅應用了 HTML、CSS 和 JavaScript，也嘗試使用框架來提升效率。設計時除了追求介面的美觀，更需要站在使用者的角度去思考體驗。雖然中途遇到過排版不協調、互動不流暢等問題，但透過不斷修改與測試，最後成功完成並與系統整合。這次專題不僅讓我提升了技術能力，也培養了思考與解決問題的能力，更重要的是學會了在團隊中溝通協調的價值。

➤ 11146067 連湘琪

在這次專題剛開始時，我其實對資料庫並不算熟悉，心裡難免有些緊張。不過因為「會議寶」需要處理大量的會議資料，這讓我有機會真正挑戰自己，把課堂上學過的知識應用到實際專案中。隨著討論逐漸深入，我也開始理解到資料庫在整個系統裡的重要性，並且一步步將結構設計得更完整。

我的任務是負責資料庫的設計與建置。過程中，我特別著重在資料表的關聯設計與查詢效能的提升，例如如何讓大量的資料能被快速搜尋，或者避免因結構不佳造成效能下降。雖然中途遇過查詢過慢、結構不合理的問題，但透過測試、調整以及和前後端的配合，最終都找到了解決辦法。這次經驗不僅讓我更加熟悉資料庫的實務應用，也讓我體會到從零開始規劃一個系統的挑戰與成就感。

➤ 11146074 邵紫綺

在這次「會議寶」專題中，我的任務是負責後端程式設計。必須確保各個功能能穩定運作，因此我主要專注在 API 的開發，並處理前端與資料庫之間的資料交換。過程中也設計了檔案上傳與權限管理，確保不同角色能正確使用系統。這部分挑戰不少，特別是權限設定，只要有一點疏忽就可能造成資料存取異常，我花了相當多時間來測試和修正。

透過這次專題，我對後端架構的理解更深入了，也體會到寫程式並不只是完成功能，而是要考慮整體的穩定性與可靠度。能和組員一起把系統完整地建構出來，對我來說是很大的成就感。我也學到如何在團隊裡發揮自己的角色，讓系統能更順利地整合完成。

➤ 11146079 李曼鈺

這次專題的主題是「會議寶」，目的是透過 AI 技術輔助會議記錄與管理，讓整個流程更有效率。我主要負責 SMTP 郵件通知，以及語音轉文字的功能。SMTP 讓系統能自動發送提醒，避免使用者遺漏重要會議；語音轉文字則利用 OpenAI 的 Whisper 技術，將冗長的錄音即時轉換成文字，讓後續的整理和檢索更加方便。這些功能的完成，使系統更貼近實際需求，也提升了整體的完整度。

在實作過程中，我遇到不少挑戰，例如郵件伺服器設定的錯誤排查，或是語音轉文字過程中常出現辨識錯誤、標點符號不準確的情況，導致逐字稿閱讀體驗不佳。每次測試都需要調整流程，甚至重新設計提示方式，才能讓輸出的內容更清楚。雖然過程辛苦，但當我看到錄音能順利轉成條理分明的逐字稿並和其他模組結合時，成就感非常充實。這次專題不僅讓我更熟悉郵件服務與語音辨識的應用，也讓我學會如何把技術落地到真實情境。同時，透過與組員的配合，我深刻體會到跨模組協作的重要，這是課堂上學不到的寶貴經驗。

➤ 11146082 范雅淇

在這次「會議寶」專題中，我的任務是將會議逐字稿轉換成正式文件。逐字稿通常內容冗長、語氣口語化，如果要人工整理需要耗費大量時間，因此我結合了 AI 與大型語言模型 (LLM)，讓系統能自動將原始紀錄整理成條理清晰、語句正式的文件，幫助使用者快速掌握重點並提取行動項目。這個功能大幅提升了系統的實用性，也展現了 AI 在實際應用中的價值。

在開發過程中，我遇到不少挑戰，像是模型輸出的文字有時候標點符號不準確，或同一句話被切得太零碎，導致閱讀體驗不佳。為了改善這些問題，我不斷調整處理流程並修改提示方式，讓結果能更貼近使用者的需求。雖然過程中花了不少心力，但看到原本雜亂的逐字稿能被轉換成正式的會議紀錄時，成就感特別強烈。這次專題不僅讓我更熟悉 AI 文本處理的應用，也讓我體會到如何把技術和實際情境結合，並在團隊協作中發揮自己的角色。

第 14 章 參考資料

[1]RAG（檢索增強生成）入門指南：[SolwenAI-為企業打造實際有價值的 AI 應用](#)

[|SolwenAI](#)

[2]LangChain 由入門到熟練建構 Chatbot 使用 OpenAIAPI：[Day16:langchain 由入門到熟練](#)

[\(建構一個 Chatbot-使用 OpenAI-API\)-iT 邦幫忙::一起幫忙解決難題，拯救 IT 人](#)

[的一天](#)

[3] MCP（多雲策略平台）解析：[mcp 是什麼？mcpserver 是什麼？mcp 中文、意思、實](#)

[例一次看懂|數位時代 BusinessNext](#)

[4]Otter.ai：[OtterMeetingAgent-AINotetaker, Transcription, Insights](#)

[5]用 ChatGPT 搭配 Whisper 輕鬆完成會議紀錄：[做會議紀錄好麻煩？ChatGPT+Whisper 一](#)

[次完成！自動聽打、摘要、寄發都可以|數位時代 BusinessNext](#)

[6]SeaMeet：[SeaMeet](#)

[7]Icons：<https://lucide.dev/>

附錄一 評審建議事項

► 初評

評審建議事項	修正情形
現行類似功能已經非常多，要怎麼做出區隔？	我們不是只做逐字稿或摘要，而是把內容直接變成任務，幫助後續追蹤和專案管理，重點是「讓會議有行動結果」。
評審建議事項	修正情形
針對跨會議工作追蹤是否可以提供主動提醒功能，例如 DEADLINE 延伸通知？	現在系統會自動產生任務，並在到期前提醒相關人員，如果沒完成還會持續通知，避免事情被遺漏。
評審建議事項	修正情形
逐字稿如何轉換成正式會議紀錄？ (包含呈現格式、文字差異)	逐字稿會先過 AI 處理，整理成重點和標準格式，去掉口語和重複內容，直接生成專業會議紀錄。
評審建議事項	修正情形
是否能增加多人協作功能？	我們已經加上多人協作功能，會議產生的摘要或紀錄不只是一個人看，大家可以一起在線上修改並留下紀錄。AI 先幫忙整理出草稿，團隊成員可以再調整，最後的會議紀錄會更完整、更符合實際需求。
評審建議事項	修正情形
系統缺乏合適競品分析？	我們補上了競品比較，說明市面上產品的差異，並凸顯我們在任務追蹤、跨會議脈絡和文件生成的優勢。
評審建議事項	修正情形
任務追蹤功能是否可以進一步加強？	現在任務可以標示進度、設定負責人，也會自動提醒，後續還能跟常見的專案管理工具串接。
評審建議事項	修正情形
系統同時提供即時錄音與上傳錄音功能，流程略顯混亂，是否應明確定位為支援線上或線下會議？	我們把定位弄清楚：線上會議用即時錄音，線下會議用上傳錄音，讓使用情境更直觀。
評審建議事項	修正情形
系統能否有效串聯錄音、逐字稿、會議記錄與正式文件的轉換流程？	現在流程已經打通，從錄音到逐字稿，再到正式會議紀錄，一鍵就能完成，不用再切換不同工具。

評審建議事項	修正情形
前台提供時間點標記，可重新聽錄音檔，任務追蹤是否讓與會者確認，權限核實同意會議內容，權限管理方式，能否有重新生成文件的次數	前台已支援依音軌時間碼標記與文字對應，方便使用者進行內容校對與檢視。 「錄音回放」功能暫未開放，主要因同時進行錄音與回放會增加系統負載，影響即時辨識效能與穩定性。 任務追蹤目前提供記錄與提醒功能，協助掌握執行情況，惟未設人工確認機制。任務執行屬團隊內部協作範疇，若負責人未執行，仍需由使用者自行協調與落實。 文件生成前可先檢視與修改內容，確認無誤後再執行生成，生成作業僅限一次，以確保輸出版本一致與內容正確。
評審建議事項	修正情形
前台須可以根據音軌時間碼編輯文字。文字稿，工作指派的審核或確認機制。會議記錄的編修權限	系統前台已支援依音軌時間碼編輯逐字稿，使用者可針對特定段落進行修改與確認，確保內容正確後再輸出任務與文件，提升資料精準度。 任務追蹤部分目前提供記錄與提醒功能，協助掌握執行情況，但未設人工確認機制。任務執行仍屬團隊協作與管理範疇，若負責人未執行，需由使用者自行協調與落實。 同時會議紀錄的編修權限僅限主持人或授權成員操作，以確保資料一致性與安全性。
評審建議事項	修正情形
語音會議逐字稿應不只靠 openAI，要讓 user 確認並修正確認逐字稿，正確的才派任務，才有價值	系統已具備使用者修正逐字稿的功能。使用者可在語音轉文字生成後進行人工確認與編輯，確認內容正確後才啟動任務派發流程，以確保任務資訊的準確性與可執行性。

附錄二 會議記錄

臺北商業大學 資訊管理系專題 114409 第二次例行會議 會議記錄

會議地點	線上Google Meeting	會議時間	114年03月07日
指導老師	陳信宏	出席人員	全組人員
會議內容			
1、全體人員集合 2、會議開始 3、討論內容 1. 方向： 訂出個人助理要輔助人類做的項目要定義一下 (ex：商用：開會會議提醒 客戶資料 跟一個人開了多少次會 結論是什麼) 決定好後再想一下需要幫助的情境是什麼？ 給機器人資料的集合是什麼？ 要提供給機器人資料（像是整個行事曆） 專題的輔助資料範圍是什麼？ 使用者傳資料的格式要求 （可能行事曆要統一格式再上傳比較好處理） （但不是常態可能必須要接受很多格式） 2. 技術： LangChain可以用 問一下王老師那邊可以架什麼東西 馬斯克 Grok 3免費 rag的方式把資料存在資料庫 大語言模型把向量資料庫的結果再轉成使用者看得懂 要什麼介面（假設如果是商用：使用者提供資料的介面 AI協助畫面 使用介面） 3. 待解決的問題： LLM+RAG+Langchain設計架構 文本分塊(chunking)、再分詞(tokenize)、matadata資料表如何建置 向量檢索(搜尋)技術，就是Embedding Model、向量工程(向量維度是多少?向量空間的建置？prompt轉向量的作法？ 回答生成、Chatbot的設計(介面、prompt提示輔助、prompt提示分析) 下次討論前再去試其他的API 看看會遇到什麼問題（準備好一些檔案 需要被整理的 丟到大語言模型 用API的方式去整理）			

臺北商業大學 資訊管理系專題 114409

第四次例行會議 會議記錄

會議地點	線上Google Meeting	會議時間	114年03月26日
指導老師	陳信宏	出席人員	全組人員

會議內容

- 1、全體人員集合
- 2、會議開始
- 3、討論內容
 1. 主題方向：
 - 學校方面會限縮到比較小的範圍
 - 大方向想好要開始想技術方面的細節
 - 創新度建議：做出區隔 除了大語言模型 政大通借接了很多服務 串LLM沒有特別的點 接近聊天機器人
 2. 技術方面：
 - 大語言模型上可能比GPT弱
 - 自動通知設定來系統 註冊後定期根據使用者資料自動寄信給使用者像是選課要結束了等等
 - 強調資料間接能夠讓使用者資料直接導進
 - 多在大語言模型資料提供 網路爬蟲機器人幫我抓回來去做規則處理
 - 學分選課建議 要自己寫演算法或是問LLM
 - 考試提醒 用爬蟲去抓學校資訊去發通知
 - 先試著抓到了資訊試著給GPT讓他通知我，如果回應的很好可以把東西接回來去做通知
 - 找特定領域的聊天機器人 像是：國泰世華助理阿發，可以做一個SWOT分析
 3. 解決的問題：
 - 根據什麼東西發送？
 - 怎麼把文字轉變成通知發出？
 - 怎麼搜集學生需求？
 - 市場是什麼？
 - 有沒有辦法調整模型回答得比open AI好？
 - 差異表格需要改一些太過抽象不夠準確

臺北商業大學 資訊管理系專題 114409

第八次例行會議 會議記錄

會議地點	線上Google Meeting	會議時間	114年05月13日
指導老師	陳信宏	出席人員	全組人員

會議內容

- 1、全體人員集合
- 2、會議開始
- 3、討論內容
 - 網頁展示建議
 1. 文件上傳：
 - 可以改成「檔案」上傳
 - 文件上傳刪除等等應該設有權限問題，使用者也應該有權限問題
 2. 語音轉文字：
 - 上傳跟錄音行為跟後續操作是不同的，可以分成不同的按鈕
 3. 議題回顧：
 - 不要用議題當名稱，一次開會可能會有很多議題
 4. 成員介紹：
 - 可以放在最外層使用說明在功能裡面
 - 專案成員跟成員介紹名稱要重新定義（可以叫關於我們）
 - 一次會議議題是會議裡的東西，一次會議會有很多份文件
 - 專案裡再去細分好幾次會議，想一下一對多的方式
 - 可以在網頁裡面增加開會時間
 - 開會通知發送的時候可以加上註冊的網址
 - 預計要用的需求整理出來給老師，像是：伺服器、檔案系統或AI等等
 - 使用說明內容應該是簡單的功能介紹，後面才是使用教學（說明）

臺北商業大學 資訊管理系專題 114409

第九次例行會議 會議記錄

會議地點	線上 Google Meeting	會議時間	114年05月26日
指導老師	陳信宏	出席人員	全組人員

會議內容

一、全體人員集合

二、會議開始

三、討論內容

1. 手冊建議：

排版要一致不要有的置中有的靠左

● 第2章 營運計畫

2-4-1 SWOT-TOWS 分析表：

劣勢：可以加知名度不高

機會：我們發現外界有功能沒有串接

威脅：可以先列出威脅再去用優勢去解決

SWOT 要多提到外部

● 第3章 系統規格

3-1-2 系統架構圖：

圖必須分一條條去做區格 字太小

老師建議：<https://vfunction.com/blog/3-tier-appli>

3-2 系統軟、硬體需求與技術平台：

軟體沒有列完全

● 第5章 使用者需求

要跟架構圖能夠呼應

5-2 使用者個案圖：

使用者角色是否可以分類

發送通知 API 可以劃一條就好

5-5 分析類別圖：

也要跟功能相呼應

語音會是會議檔案裡的東西嗎？

● 第6章 設計模型

循序圖：

不是：UI

6-1-1 會員註冊之循序圖：

第一個應該是查驗有沒有人註冊過

老師建議：<https://developer.ibm.com/articles/the-sequence-diagram/>

<https://terryjryeh.blogspot.com/2019/03/uml-sequence-diagrams-8.html>

- 第7章 實作模型

- 7-1-1 佈署圖：

- 中間不是 databass

- 外部的 server 會畫在外部除非是自己建的

- 7-2-1 套件圖：

- 圖必須要連貫前後串聯

- 7-3-1 元件圖：

- 跟系統架構圖是相似的

- 7-4-5 語音轉文字之狀態機：

- 語音辨識才會在顯示轉錄結果 有可能會失敗像是不清楚之類的

- 7-4-6 會議回顧之狀態機：

- 不用寫「等待」使用者選擇會議

- 複製結果有點奇怪應該會有個後續

- 第8章 資料庫設計

- 8-1-1 資料庫關聯表：

- 使用者名稱不是 int

- 一個組織應該包含很多人，要可以一對多

- 一次會議也需要多個使用者

- 可以根據 ER modle 畫

- 8-2 表格及其 Meta data：

- Metadata 是同一個字不可以空格

- 主索引應該叫主鍵

- 2. 問題回覆：

- 1. 那邊能提供之前提到的LLaMA API endpoint、token 模型規格和有呼叫量限制嗎？

- 老師回答：Whisper可以架，gpt不行要一直花錢，有LLaMA呼叫量有蠻大的。

- 2. 關於伺服器環境？

- 老師回答：最好是需要的型號都可以說清楚，然後再過去中研院架設，像是flask db等等。

- 3. 中研院暑假每個禮拜有哪幾天可以讓我們固定過去？

- 老師回答：會幫我們借會議室需要自己帶筆電，想要去幾天都可以 討論完再跟老師說，我們決定星期一二三四。

臺北商業大學 資訊管理系專題 114409

第十次例行會議 會議記錄

會議地點	線上Google Meeting	會議時間	114年07月28日
指導老師	陳信宏	出席人員	全組人員
會議內容			
1、全體人員集合 2、會議開始 3、討論內容 ● 報告進度： ● 老師問題： 1. AI這一塊沒有訓練 速度慢 準確度不高 (llama) 2. 智能問答可以自由下問題去命令嗎？準確度夠不夠？ 3. 逐字稿需要加上標點符號 4. 標點符號是否要用中研院模型，例如斷詞斷句，但可能會有問題因為當初訓練的不確定是否適用於一般 5. 轉換的字有些錯誤（會用同音字取代） ● 解決建議： 1. 利用AI用貼上的方式產生制式的文件格式，會議記錄模板找至少56個模板，可以找不同單位的去做整理 2. 會議回顧不只有會議摘要，要可以生成出文本跟討論內容共識決議等等（AI沒有出來的可以去做補充） 3. 文件轉換套格式不是llama強項，沒有訓練還做不到，可能需要找另外的模型 4. 要去連大型語言模型（但因為還沒訓練）轉正式文件需要 5. 跟notebookLM比較，但他可以把語音錄下來並直接轉換會議內容，但只是對話式的摘要，並不會轉換（我們可以！很棒！） 6. 架構有達到目標讚讚，但AI模型部分要加強做的過程多比較，至少達到notebookLM 八成就完美了			

臺北商業大學 資訊管理系專題 114409

第十二次例行會議 會議記錄

會議地點	線上Google Meeting	會議時間	114年08月20日
指導老師	陳信宏	出席人員	全組人員
會議內容			
1、全體人員集合 2、會議開始 3、討論內容 ● 進度報告 ● 老師建議 1. 檔案轉錄： 速度跟準確度問題解決問題會將模型轉到70B 標點符號還是會不準，後續可能轉到70B試看看 逐字稿文章要去先寫GPT的API之後中研院內部的架好就可以直接套用 2. 正式文件： 這項功能是要幫沒有格式的檔案去做轉錄 會議記錄整理的格式像是文字區塊，逐字稿去掉口語化詞語，誰講了什麼東西等等 會議記錄整理可以改為逐字稿整理 3. 新增功能： 看看能不能加上line bot，如果可以實現後續可以加上會前通知 4. 比賽： 一定要去參加資訊應用競賽（AI工具應用組）產業創新AI比較不符合 越大的比賽加分越高，多找大比賽錢多的			